

SOLIDWORKS API (.NET VBA C#) — Examples (8/50):

Create_Advanced_Hole_Example_VB- Create_Base_Flange_Feature_Example_CShar

SOLIDWORKS API Documentation (converted from CHM)

Create Advanced Hole Feature Example (VBA)

This example shows how to create an Advanced Hole feature.

```

'-----
' Preconditions:
' 1. Verify that the specified part document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Selects near and far side faces of the Advanced Hole.
' 2. Defines an Advanced Hole at a sketch point on the edge of the block
'    with the following:
'    * Countersink near side element
'    * Straight tap near side element
'    * Counterbore far side element
'    * Straight hole far side element
' 3. Gets some near and far side element data for the Advanced Hole.
' 4. Modifies the near side element array to contain a tapered tap element.
' 5. Deletes the Advanced Hole's defining sketch point on the edge of the block.
' 6. Adds two sketch points and creates two Advanced Holes at those locations
'    using the previously defined Advanced Hole.
' 7. Inspect the Immediate window and graphics area.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim feat As Feature
Dim boolstatus As Boolean
Dim swAdvHole_Near_1 As AdvancedHoleElementData
Dim swAdvHole_Near_2 As AdvancedHoleElementData
Dim swAdvHole_Near_3 As AdvancedHoleElementData
Dim swAdvHole_Far_1 As AdvancedHoleElementData
Dim swAdvHole_Far_2 As AdvancedHoleElementData
Dim swCounterSinkNear As CountersinkElementData
Dim swCounterBoreFar As CounterboreElementData
Dim swStraightHoleFar As StraightElementData
Dim swStraightTapNear As StraightTapElementData
Dim swTaperedTapNear As TaperedTapElementData
Dim swFeatureMgr As FeatureManager
Dim ConvFactorAngle, ConvFactorLength As Double
Dim advHoleNearArr(1) As AdvancedHoleElementData
Dim advHoleFarArr(1) As AdvancedHoleElementData
Dim featdata As AdvancedHoleFeatureData
Dim newNearArr(1) As AdvancedHoleElementData
Dim newFarArr(1) As AdvancedHoleElementData
Dim swSketchFeature As Feature
Dim swSelectionManager As SelectionMgr
Dim swSketch As Sketch
Dim swSketchPointArray As Variant
Dim swMaxPointNumber As Long
Dim skPoint As Object
Dim swSketchPoint As Object
Dim swCurrentPointNumber As Long
Dim errors As Long, warnings As Long
Dim jtr As Long
Dim ResultArray As Variant
Dim CalloutString As String
Dim StrDiam As String
Dim strDepth As String
Dim retVal As Boolean

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block20.sldprt",
swDocPART, swOpenDocOptions_Silent, "", errors, warnings)
    Set swFeatureMgr = swModel.FeatureManager

    ConvFactorLength = 25.4 / 1000 'inches to meters
    ConvFactorAngle = (22 / 7) / 180 'degrees to radians

    'Select the near and far side faces
    boolstatus = swModel.Extension.SelectByRay(-5.89202612791269E-02, 2.60626824463657E-02, 5.60000000000969E-02,
-0.400036026779312, -0.515038074910024, -0.758094294050284, 7.22831189342222E-04, 2, False, 256, 0) 'Near side
    boolstatus = swModel.Extension.SelectByRay(-1.10716643645077E-02, 2.11784489308116E-02, -6.39370439421896E-02,
0.18261953966356, -0.612697461661826, 0.76892907618728, 9.36301020408163E-04, 2, False, 512, 0) 'Far side

    'Define near and far side hole elements

    'Near side countersink
    Set swAdvHole_Near_1 = swModel.Extension.CreateAdvancedHoleElementData(swAdvWzdGeneralHoleTypes_e.swAdvWzdCounterSink)
    swAdvHole_Near_1.Orientation = swHoleElementOrientation_e.swHoleElementOrientation_Nearside

```

```

swAdvHole_Near_1.Size = "#4"
swAdvHole_Near_1.Standard = swWzdHoleStandards_e.swStandardAnsiInch
swAdvHole_Near_1.FastenerType = swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchFlatHead100
swAdvHole_Near_1.Diameter = ConvFactorLength * 0.225
swAdvHole_Near_1.BlindDepth = 0.02055794 * ConvFactorLength
swAdvHole_Near_1.EndCondition = swEndConditions_e.swEndCondBlind
Set swCounterSinkNear = swAdvHole_Near_1
swCounterSinkNear.EndConditionOverride = True
swCounterSinkNear.AngleOverride = False

'Near side straight tap
Set swAdvHole_Near_2 = swModel.Extension.CreateAdvancedHoleElementData(swAdvWzdGeneralHoleTypes_e.swAdvWzdStraightTap)

swAdvHole_Near_2.Orientation = swHoleElementOrientation_e.swHoleElementOrientation_Nearside
swAdvHole_Near_2.Size = "#4-40"
swAdvHole_Near_2.Standard = swWzdHoleStandards_e.swStandardAnsiInch
swAdvHole_Near_2.FastenerType = swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchBottomingTappedHole
swAdvHole_Near_2.Diameter = ConvFactorLength * 0.089
swAdvHole_Near_2.EndCondition = swEndConditions_e.swEndCondUpToNext
swAdvHole_Near_2.DiameterOverride = True
Set swStraightTapNear = swAdvHole_Near_2
swStraightTapNear.CustomSizing = swStraightTapHoleCustomSizing_e.swStraightTapHoleCustomSizing_TapDrillDiameter
swStraightTapNear.ThreadClass = swStraightTapHoleThreadClass_e.swStraightTapHoleThreadClass_1B
swStraightTapNear.ThreadClassOverride = True

'Near side tapered tap
Set swAdvHole_Near_3 = swModel.Extension.CreateAdvancedHoleElementData(swAdvWzdGeneralHoleTypes_e.swAdvWzdTaperTap)
swAdvHole_Near_3.Orientation = swHoleElementOrientation_e.swHoleElementOrientation_Nearside
swAdvHole_Near_3.Size = "1/16"
swAdvHole_Near_3.Standard = swWzdHoleStandards_e.swStandardAnsiInch
swAdvHole_Near_3.FastenerType = swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchTaperedPipeTap
swAdvHole_Near_3.Diameter = ConvFactorLength * 0.266
swAdvHole_Near_3.BlindDepth = 0.205 * ConvFactorLength
swAdvHole_Near_3.EndCondition = swEndConditions_e.swEndCondBlind
swAdvHole_Near_3.DiameterOverride = True
Set swTaperedTapNear = swAdvHole_Near_3
swTaperedTapNear.CustomSizing = swTaperedTapCustomSizing_e.swTaperedTapCustomSizing_MinorDiameterwithCosmeticThread
swTaperedTapNear.ThreadClass = swTaperedTapThreadClass_e.swTaperedTapThreadClass_1
swTaperedTapNear.ThreadClassOverride = True
swTaperedTapNear.EndConditionOverride = True

'Far side counterbore
Set swAdvHole_Far_1 = swModel.Extension.CreateAdvancedHoleElementData(swAdvWzdGeneralHoleTypes_e.swAdvWzdCounterBore)
swAdvHole_Far_1.Orientation = swHoleElementOrientation_e.swHoleElementOrientation_Farside
swAdvHole_Far_1.Size = "#8"
swAdvHole_Far_1.Standard = swWzdHoleStandards_e.swStandardAnsiInch
swAdvHole_Far_1.FastenerType = swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchBinding
swAdvHole_Far_1.Diameter = ConvFactorLength * 0.375
swAdvHole_Far_1.BlindDepth = 0.105 * ConvFactorLength
swAdvHole_Far_1.EndCondition = swEndConditions_e.swEndCondBlind
swAdvHole_Far_1.DiameterOverride = True
Set swCounterBoreFar = swAdvHole_Far_1
swCounterBoreFar.EndConditionOverride = True

'Far side straight hole
Set swAdvHole_Far_2 = swModel.Extension.CreateAdvancedHoleElementData(swAdvWzdGeneralHoleTypes_e.swAdvWzdStraight)

swAdvHole_Far_2.Orientation = swHoleElementOrientation_e.swHoleElementOrientation_Farside
swAdvHole_Far_2.Size = "1/16"
swAdvHole_Far_2.Standard = swWzdHoleStandards_e.swStandardAnsiInch
swAdvHole_Far_2.FastenerType = swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchAllDrillSizes
swAdvHole_Far_2.Diameter = ConvFactorLength * 0.0625
swAdvHole_Far_2.BlindDepth = 0.2711 * ConvFactorLength
swAdvHole_Far_2.EndCondition = swEndConditions_e.swEndCondBlind
swAdvHole_Far_2.DiameterOverride = True

'Customize the hole callout for this straight element
StrDiam = swModel.Extension.GetCalloutVariableString(swCalloutVariable_AH_Straight_Diameter)
StrDepth = swModel.Extension.GetCalloutVariableString(swCalloutVariable_AH_Straight_Depth)
CalloutString = "<MOD-DIAM> " & StrDiam & " " & "<HOLE-DEPTH> " & StrDepth
swAdvHole_Far_2.CalloutString = CalloutString
Set swStraightHoleFar = swAdvHole_Far_2

'Set the near and far side arrays; stack ordering is from the near and far side faces
Set advHoleNearArr(0) = swCounterSinkNear
Set advHoleNearArr(1) = swStraightTapNear
Set advHoleFarArr(0) = swCounterBoreFar
Set advHoleFarArr(1) = swStraightHoleFar

Set feat = swFeatureMgr.AdvancedHole2((advHoleNearArr), (advHoleFarArr), False, True, False, ResultArray)

'Get the near and far side element data
Set featdata = feat.GetDefinition
featdata.AccessSelections swModel, Nothing
Debug.Print "Number of near side hole elements: " & featdata.NearSideElementsCount
Debug.Print "Number of far side hole elements: " & featdata.FarSideElementsCount
Set swCounterSinkNear = featdata.GetNearSideElements(0)
Set swCounterBoreFar = featdata.GetFarSideElements(0)
Debug.Print "Near side countersink:"
Debug.Print "   Hole element type as defined in swAdvWzdGeneralHoleTypes_e: " & swCounterSinkNear.ElementType
Debug.Print "   Size as defined on the Advanced Hole PropertyManager page: " & swCounterSinkNear.Size
Debug.Print "   Standard as defined in swWzdHoleStandards_e: " & swCounterSinkNear.Standard

```

```

Debug.Print " Fastener type as defined in swWzdHoleStandardFastenerTypes_e: " & swCounterSinkNear.FastenerType
Debug.Print " Diameter in m: " & swCounterSinkNear.Diameter
Debug.Print " Blind depth in m: " & swCounterSinkNear.BlindDepth
Debug.Print " Orientation as defined in swHoleElementOrientation_e: " & swCounterSinkNear.Orientation
Debug.Print " End condition as defined in swEndConditions_e: " & swCounterSinkNear.EndCondition

```

```

Debug.Print "Far side straight:"
Debug.Print " Hole element type as defined in swAdvWzdGeneralHoleTypes_e: " & swStraightHoleFar.ElementType
Debug.Print " Size as defined on the Advanced Hole PropertyManager page: " & swStraightHoleFar.Size
Debug.Print " Standard as defined in swWzdHoleStandards_e: " & swStraightHoleFar.Standard
Debug.Print " Fastener type as defined in swWzdHoleStandardFastenerTypes_e: " & swStraightHoleFar.FastenerType
Debug.Print " Diameter in m: " & swStraightHoleFar.Diameter
Debug.Print " Diameter override? " & swStraightHoleFar.DiameterOverride
Debug.Print " Blind depth in m: " & swStraightHoleFar.BlindDepth
Debug.Print " Orientation as defined in swHoleElementOrientation_e: " & swStraightHoleFar.Orientation
Debug.Print " End condition as defined in swEndConditions_e: " & swStraightHoleFar.EndCondition
Debug.Print " Customized hole callout: " & swStraightHoleFar.CalloutString

```

```

'Modify the near and far side element arrays
Set newNearArr(0) = swTaperedTapNear
Set newNearArr(1) = swStraightTapNear
'Set newFarArr(0) = swCounterBoreFar
'Set newFarArr(1) = swStraightHoleFar
featdata.SetNearSideElements newNearArr
'featdata.SetFarSideElements newFarArr
feat.ModifyDefinition featdata, swModel, Nothing

```

```

'Delete the first point used to define the Advanced Hole
Set swSketchFeature = feat.GetFirstSubFeature
swSketchFeature.Select2 False, 0
swModel.EditSketch
Set swSelectionManager = swModel.SelectionManager
Set swSketch = swSketchFeature.GetSpecificFeature2()
swSketchPointArray = swSketch.GetSketchPoints2()
swMaxPointNumber = UBound(swSketchPointArray)
For swCurrentPointNumber = 0 To swMaxPointNumber Step 1
    Set swSketchPoint = swSketchPointArray(swCurrentPointNumber)
    retVal = swSelectionManager.AddSelectionListObject(swSketchPoint, Nothing)
    swModel.EditDelete
Next swCurrentPointNumber

```

```

'Create points for multiple Advanced Hole locations
Set skPoint = swModel.SketchManager.CreatePoint(-3.19158789518497E-02, 3.44489966898323E-02, 0.0560000000000004)
Set skPoint = swModel.SketchManager.CreatePoint(-4.94104502066557E-02, 8.0156770060853E-03, 5.60000000000969E-02)

```

```

swModel.SketchManager.InsertSketch True

```

```

End Sub

```

Create Advanced Hole Feature Example (VB.NET)

This example shows how to create an Advanced Hole feature.

```

'-----
' Preconditions:
' 1. Verify that the specified part document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Selects near and far side faces of the Advanced Hole.
' 2. Defines an Advanced Hole at a sketch point on the edge of the block
'    with the following:
'    * Countersink near side element
'    * Straight tap near side element
'    * Counterbore far side element
'    * Straight hole far side element
' 3. Gets some near and far side element data for the Advanced Hole.
' 4. Modifies the near side element array to contain a tapered tap element.
' 5. Deletes the Advanced Hole's defining sketch point on the edge of the block.
' 6. Adds two sketch points and creates two Advanced Holes at those locations
'    using the previously defined Advanced Hole.
' 7. Inspect the Immediate window and graphics area.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices

```

Partial Class SolidWorksMacro

Public Sub main()

```

    Dim swModel As ModelDoc2
    Dim feat As Feature
    Dim boolstatus As Boolean
    Dim swAdvHole_Near_1 As AdvancedHoleElementData
    Dim swAdvHole_Near_2 As AdvancedHoleElementData
    Dim swAdvHole_Near_3 As AdvancedHoleElementData
    Dim swAdvHole_Far_1 As AdvancedHoleElementData
    Dim swAdvHole_Far_2 As AdvancedHoleElementData
    Dim swCounterSinkNear As CountersinkElementData
    Dim swCounterBoreFar As CounterboreElementData
    Dim swStraightHoleFar As StraightElementData
    Dim swStraightTapNear As StraightTapElementData
    Dim swTaperedTapNear As TaperedTapElementData
    Dim swFeatureMgr As FeatureManager
    Dim ConvFactorLength As Double
    Dim advHoleNearArr(1) As AdvancedHoleElementData
    Dim advHoleFarArr(1) As AdvancedHoleElementData
    Dim featdata As AdvancedHoleFeatureData
    Dim newNearArr(1) As AdvancedHoleElementData
    Dim newFarArr(1) As AdvancedHoleElementData
    Dim swSketchFeature As Feature
    Dim swSelectionManager As SelectionMgr
    Dim swSketch As Sketch
    Dim swSketchPointArray As Object
    Dim swMaxPointNumber As Integer
    Dim skPoint As Object
    Dim swSketchPoint As Object
    Dim swCurrentPointNumber As Integer
    Dim errors As Long, warnings As Integer
    Dim ResultArray As Object = Nothing
    Dim CalloutString As String
    Dim StrDiam As String
    Dim strDepth As String

```

Dim retval As Boolean

```

swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block20.sldprt", swDocum
swFeatureMgr = swModel.FeatureManager

```

```

'Conversion from inches to meters
ConvFactorLength = 25.4 / 1000

```

```

'Select two faces for the near side and far side hole elements
boolstatus = swModel.Extension.SelectByRay(-0.0589202612791269, 0.0260626824463657, 0.05600000000000969, -0.400036026779312,
boolstatus = swModel.Extension.SelectByRay(-0.0110716643645077, 0.0211784489308116, -0.0639370439421896, 0.18261953966356, -

```

```

'Define near and far side hole elements

```

```

'Near side countersink
swAdvHole_Near_1 = swModel.Extension.CreateAdvancedHoleElementData(swAdvWzdGeneralHoleTypes_e.swAdvWzdCounterSink)

```

```

swAdvHole_Near_1.Orientation = swHoleElementOrientation_e.swHoleElementOrientation_Nearside
swAdvHole_Near_1.Size = "#4"

```

```

swAdvHole_Near_1.Standard = 0
swAdvHole_Near_1.FastenerType = swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchFlatHead100
swAdvHole_Near_1.Diameter = ConvFactorLength * 0.225
swAdvHole_Near_1.BlindDepth = 0.02055794 * ConvFactorLength
swAdvHole_Near_1.EndCondition = swEndConditions_e.swEndCondBlind
swCounterSinkNear = swAdvHole_Near_1
swCounterSinkNear.EndConditionOverride = True
swCounterSinkNear.AngleOverride = False

'Near side straight tap
swAdvHole_Near_2 = swModel.Extension.CreateAdvancedHoleElementData(swAdvWzdGeneralHoleTypes_e.swAdvWzdStraightTap)

swAdvHole_Near_2.Size = "#4-40"
swAdvHole_Near_2.Standard = swWzdHoleStandards_e.swStandardAnsiInch
swAdvHole_Near_2.FastenerType = swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchBottomingTappedHole
swAdvHole_Near_2.Diameter = ConvFactorLength * 0.089
swAdvHole_Near_2.EndCondition = swEndConditions_e.swEndCondUpToNext
swAdvHole_Near_2.DiameterOverride = True
swStraightTapNear = swAdvHole_Near_2
swStraightTapNear.CustomSizing = swStraightTapHoleCustomSizing_e.swStraightTapHoleCustomSizing_TapDrillDiameter
swStraightTapNear.ThreadClass = swStraightTapHoleThreadClass_e.swStraightTapHoleThreadClass_1B
swStraightTapNear.ThreadClassOverride = True

'Near side tapered tap
swAdvHole_Near_3 = swModel.Extension.CreateAdvancedHoleElementData(swAdvWzdGeneralHoleTypes_e.swAdvWzdTaperTap)

swAdvHole_Near_3.Orientation = swHoleElementOrientation_e.swHoleElementOrientation_Nearside
swAdvHole_Near_3.Size = "1/16"
swAdvHole_Near_3.Standard = swWzdHoleStandards_e.swStandardAnsiInch
swAdvHole_Near_3.FastenerType = swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchTaperedPipeTap
swAdvHole_Near_3.Diameter = ConvFactorLength * 0.266
swAdvHole_Near_3.BlindDepth = 0.205 * ConvFactorLength
swAdvHole_Near_3.EndCondition = swEndConditions_e.swEndCondBlind
swAdvHole_Near_3.DiameterOverride = True
swTaperedTapNear = swAdvHole_Near_3
swTaperedTapNear.CustomSizing = swTaperedTapCustomSizing_e.swTaperedTapCustomSizing_MinorDiameterWithCosmeticThread
swTaperedTapNear.ThreadClass = swTaperedTapThreadClass_e.swTaperedTapThreadClass_1
swTaperedTapNear.ThreadClassOverride = True
swTaperedTapNear.EndConditionOverride = True

'Far side counterbore
swAdvHole_Far_1 = swModel.Extension.CreateAdvancedHoleElementData(swAdvWzdGeneralHoleTypes_e.swAdvWzdCounterBore)

swAdvHole_Far_1.Orientation = swHoleElementOrientation_e.swHoleElementOrientation_Farside
swAdvHole_Far_1.Size = "#8"
swAdvHole_Far_1.Standard = swWzdHoleStandards_e.swStandardAnsiInch
swAdvHole_Far_1.FastenerType = swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchBinding
swAdvHole_Far_1.Diameter = ConvFactorLength * 0.375
swAdvHole_Far_1.BlindDepth = 0.105 * ConvFactorLength
swAdvHole_Far_1.EndCondition = swEndConditions_e.swEndCondBlind
swAdvHole_Far_1.DiameterOverride = True
swCounterBoreFar = swAdvHole_Far_1
swCounterBoreFar.EndConditionOverride = True

'Far side straight
swAdvHole_Far_2 = swModel.Extension.CreateAdvancedHoleElementData(swAdvWzdGeneralHoleTypes_e.swAdvWzdStraight)

swAdvHole_Far_2.Size = "1/16"
swAdvHole_Far_2.Standard = 0
swAdvHole_Far_2.FastenerType = swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchAllDrillSizes
swAdvHole_Far_2.Diameter = ConvFactorLength * 0.0625
swAdvHole_Far_2.BlindDepth = 0.2711 * ConvFactorLength
swAdvHole_Far_2.EndCondition = swEndConditions_e.swEndCondBlind
swAdvHole_Far_2.DiameterOverride = True

'Customize the hole callout for this straight element
StrDiam = swModel.Extension.GetCalloutVariableString(swCalloutVariable_e.swCalloutVariable_AH_Straight_Diameter)
StrDepth = swModel.Extension.GetCalloutVariableString(swCalloutVariable_e.swCalloutVariable_AH_Straight_Depth)
CalloutString = "<MOD-DIAM> " & StrDiam & " " & "<HOLE-DEPTH> " & StrDepth
swAdvHole_Far_2.CalloutString = CalloutString

swStraightHoleFar = swAdvHole_Far_2

'Set near and far side element arrays
advHoleNearArr(0) = swCounterSinkNear
advHoleNearArr(1) = swStraightTapNear
advHoleFarArr(0) = swCounterBoreFar
advHoleFarArr(1) = swStraightHoleFar

Dim dispArray As DispatchWrapper() = ObjectArrayToDispatchWrapperArray(New Object() {advHoleNearArr(0), advHoleNearArr(1)})
Dim dispArray2 As DispatchWrapper() = ObjectArrayToDispatchWrapperArray(New Object() {advHoleFarArr(0), advHoleFarArr(1)})

'Create the Advanced Hole using the near and far side element arrays; specify to not use baseline dimensions; customize Hole
feat = swFeatureMgr.AdvancedHole2(dispArray, dispArray2, False, True, False, ResultArray)

'Get some near and far side element data
featdata = feat.GetDefinition
featdata.AccessSelections(swModel, Nothing)
Debug.Print("Number of near side hole elements: " & featdata.NearSideElementsCount)
Debug.Print("Number of far side hole elements: " & featdata.FarSideElementsCount)
swCounterSinkNear = featdata.GetNearSideElements(0)
swCounterBoreFar = featdata.GetFarSideElements(0)
swStraightHoleFar = featdata.GetFarSideElements(1)
Debug.Print("Near side countersink:")

```

```

Debug.Print(" Hole element type as defined in swAdvWzdGeneralHoleTypes_e: " & swCounterSinkNear.ElementType)
Debug.Print(" Size as defined on the Advanced Hole PropertyManager page: " & swCounterSinkNear.Size)
Debug.Print(" Standard as defined in swWzdHoleStandards_e: " & swCounterSinkNear.Standard)
Debug.Print(" Fastener type as defined in swWzdHoleStandardFastenerTypes_e: " & swCounterSinkNear.FastenerType)
Debug.Print(" Diameter in m: " & swCounterSinkNear.Diameter)
Debug.Print(" Blind depth in m: " & swCounterSinkNear.BlindDepth)
Debug.Print(" Orientation as defined in swHoleElementOrientation_e: " & swCounterSinkNear.Orientation)
Debug.Print(" End condition as defined in swEndConditions_e: " & swCounterSinkNear.EndCondition)
Debug.Print("Far side straight:")
Debug.Print(" Hole element type as defined in swAdvWzdGeneralHoleTypes_e: " & swStraightHoleFar.ElementType)
Debug.Print(" Size as defined on the Advanced Hole PropertyManager page: " & swStraightHoleFar.Size)
Debug.Print(" Standard as defined in swWzdHoleStandards_e: " & swStraightHoleFar.Standard)
Debug.Print(" Fastener type as defined in swWzdHoleStandardFastenerTypes_e: " & swStraightHoleFar.FastenerType)
Debug.Print(" Diameter in m: " & swStraightHoleFar.Diameter)
Debug.Print(" Diameter override? " & swStraightHoleFar.DiameterOverride)
Debug.Print(" Blind depth in m: " & swStraightHoleFar.BlindDepth)
Debug.Print(" End condition as defined in swEndConditions_e: " & swStraightHoleFar.EndCondition)
Debug.Print(" Customized hole callout: " & swStraightHoleFar.CalloutString)

```

```

'Modify the near and far side element arrays
newNearArr(0) = swTaperedTapNear
newNearArr(1) = swStraightTapNear
'newFarArr(0) = swCounterBoreFar
'newFarArr(1) = swStraightHoleFar
featdata.SetNearSideElements(newNearArr)
'featdata.SetFarSideElements(newFarArr)
feat.ModifyDefinition(featdata, swModel, Nothing)

```

```

'Delete the first point used to define the Advanced Hole
swSketchFeature = feat.GetFirstSubFeature
swSketchFeature.Select2(False, 0)
swModel.EditSketch
swSelectionManager = swModel.SelectionManager
swSketch = swSketchFeature.GetSpecificFeature2()
swSketchPointArray = swSketch.GetSketchPoints2()
swMaxPointNumber = UBound(swSketchPointArray)
For swCurrentPointNumber = 0 To swMaxPointNumber Step 1
    swSketchPoint = swSketchPointArray(swCurrentPointNumber)
    retval = swSelectionManager.AddSelectionListObject(swSketchPoint, Nothing)
    swModel.EditDelete
Next swCurrentPointNumber

```

```

'Create points for multiple Advanced Hole locations
skPoint = swModel.SketchManager.CreatePoint(-0.0319158789518497, 0.0344489966898323, 0.0560000000000004)
skPoint = swModel.SketchManager.CreatePoint(-0.0494104502066557, 0.0080156770060853, 0.05600000000000969)

swModel.SketchManager.InsertSketch(True)

```

End Sub

```

Public Function ObjectArrayToDispatchWrapperArray(ByVal Objects As Object()) As DispatchWrapper()
    Dim ArraySize As Integer = 0
    ArraySize = Objects.GetUpperBound(0)
    Dim d As DispatchWrapper() = New DispatchWrapper(ArraySize + 1 - 1) {}
    Dim ArrayIndex As Integer = 0
    For ArrayIndex = 0 To ArraySize
        d(ArrayIndex) = New DispatchWrapper(Objects(ArrayIndex))
    Next
    Return d
End Function

```

```

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Create and Access Curve-driven Pattern Feature Example (C#)

This example shows how to create a curve-driven pattern feature and access its data.

```
//-----
// Preconditions:
// 1. Verify that the specified part document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part document.
// 2. Creates a cut extrude feature.
// 3. Creates a curve-driven pattern feature using the
//    the cut extrude feature.
// 4. Gets curve-driven pattern feature data.
// 5. Examine the FeatureManager design tree, graphics area,
//    and Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace CurveDrivenPatternFeatureCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchMgr = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            FeatureManager swFeatureMgr = default(FeatureManager);
            Feature swFeature = default(Feature);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            CurveDrivenPatternFeatureData swCurveDrivenPatternFeatureData = default(CurveDrivenPatternFeatureData);
            Entity swEntity = default(Entity);
            object patternDirection = null;
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2019\\samples\\tutorial\\api\\bagel.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);

            //Sketch a circle and create a cut extrude
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("", "FACE", 0.0118560192339032, 0, 0.0566664535234693, false, 0, null, 0);
            swSketchMgr = (SketchManager)swModel.SketchManager;
            swSketchMgr.InsertSketch(true);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateCircle(-0.059172, -0.048012, 0.0, -0.044189, -0.040533, 0.0);
            swSketchMgr.InsertSketch(true);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Sketch6", "SKETCH", 0, 0, 0, false, 0, null, 0);
            swFeatureMgr = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureMgr.FeatureCut3(true, false, false, 1, 0, 0.00254, 0.00254, false, false, false,
            false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, false, true, true,
            true, true, false, 0, 0, false);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swSelectionMgr.EnableContourSelection = false;
            swModel.ActivateSelectedFeature();
            status = swModelDocExt.SelectByID2("", "EDGE", 0.0115207253109588, -8.89643058599177E-06, 0.0754182969300832, true, 0, null, 0);
            swModel.ClearSelection2(true);

            //Create curve-driven pattern feature
            status = swModelDocExt.SelectByID2("Cut-Extrude2", "BODYFEATURE", 0, 0, 0, false, 4, null, 0);
            status = swModelDocExt.SelectByID2("", "EDGE", 0.0115207253109588, -8.89643058599177E-06, 0.0754182969300832, true, 1, null, 0);

            swCurveDrivenPatternFeatureData =
            (CurveDrivenPatternFeatureData)swFeatureMgr.CreateDefinition((int)swFeatureNameID_e.swFmCurvePattern);

            swCurveDrivenPatternFeatureData.D1AlignmentMethod = 0;
            swCurveDrivenPatternFeatureData.D1CurveMethod = 0;
            swCurveDrivenPatternFeatureData.D1InstanceCount = 3;
            swCurveDrivenPatternFeatureData.D1IsEqualSpaced = true;
            swCurveDrivenPatternFeatureData.D1ReverseDirection = false;
            swCurveDrivenPatternFeatureData.D1Spacing = 0.0254;

            swFeature = (Feature)swFeatureMgr.CreateFeature(swCurveDrivenPatternFeatureData);

            //Access the curve-driven pattern feature
            status = swModelDocExt.SelectByID2("CrvPattern1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            swCurveDrivenPatternFeatureData = (CurveDrivenPatternFeatureData)swFeature.GetDefinition();
            status = swCurveDrivenPatternFeatureData.AccessSelections(swModel, null);
            Debug.Print("Number of pattern instances in Direction 1: " + swCurveDrivenPatternFeatureData.D1InstanceCount);
            Debug.Print("Alignment method of Direction 1: " + swCurveDrivenPatternFeatureData.D1AlignmentMethod);
            Debug.Print("Curve method of Direction 1: " + swCurveDrivenPatternFeatureData.D1CurveMethod);
            patternDirection = swCurveDrivenPatternFeatureData.D1Direction;
            swEntity = (Entity)patternDirection;
            Debug.Print("Pattern direction object type of Direction 1: " + swEntity.GetType());
            Debug.Print("Pattern instances spaced equally in Direction 1? " + swCurveDrivenPatternFeatureData.D1IsEqualSpaced);
            Debug.Print("Pattern direction reversed in Direction 1? " + swCurveDrivenPatternFeatureData.D1ReverseDirection);
            Debug.Print("Number of seed bodies in pattern: " + swCurveDrivenPatternFeatureData.GetPatternBodyCount());
            Debug.Print("Number of seed features in pattern: " + swCurveDrivenPatternFeatureData.GetPatternFeatureCount());
            swCurveDrivenPatternFeatureData.ReleaseSelectionAccess();
        }
    }
}
```

```
}  
/// <summary>  
/// The Sldworks swApp variable is pre-assigned for you.  
/// </summary>  
public SldWorks swApp;  
}
```

Create and Access Curve-driven Pattern Feature Example (VBA)

This example shows how to create a curve-driven pattern feature and access its data.

```

'-----
' Preconditions:
' 1. Verify that the specified part document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Creates a cut extrude feature.
' 3. Creates a curve-driven pattern feature using the
'   the cut extrude feature.
' 4. Gets curve-driven pattern feature data.
' 5. Examine the FeatureManager design tree, graphics area,
'   and Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchMgr As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swFeatureMgr As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swCurveDrivenPatternFeatureData As SldWorks.CurveDrivenPatternFeatureData
Dim swEntity As SldWorks.Entity
Dim patternDirection As Object
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\bagel.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Sketch a circle and create a cut extrude
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID("", "FACE", 1.18560192339032E-02, 0, 5.66664535234693E-02, False, 0, Nothing, 0)
    Set swSketchMgr = swModel.SketchManager
    swSketchMgr.InsertSketch True
    Set swSketchSegment = swSketchMgr.CreateCircle(-0.059172, -0.048012, 0#, -0.044189, -0.040533, 0#)
    swSketchMgr.InsertSketch True
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID("Sketch6", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeatureMgr = swModel.FeatureManager
    Set swFeature = swFeatureMgr.FeatureCut3(True, False, False, 1, 0, 0.00254, 0.00254, False, False, False, False, 1.74532925199433E-02, 1.74532925199433E-02, False, False, False, False, True, True, True, True, False, 0, 0, False)
    Set swSelectionMgr = swModel.SelectionManager
    swSelectionMgr.EnableContourSelection = False
    swModel.ActivateSelectedFeature
    status = swModelDocExt.SelectByID("", "EDGE", 1.15207253109588E-02, -8.89643058599177E-06, 7.54182969300832E-02, True, 0, Nothing, 0)
    swModel.ClearSelection2 True

    'Create curve-driven pattern feature
    status = swModelDocExt.SelectByID("Cut-Extrude2", "BODYFEATURE", 0, 0, 0, False, 4, Nothing, 0)
    status = swModelDocExt.SelectByID("", "EDGE", 1.15207253109588E-02, -8.89643058599177E-06, 7.54182969300832E-02, True, 1, Nothing, 0)

    Set swCurveDrivenPatternFeatureData = swFeatureMgr.CreateDefinition(swFmCurvePattern)

    swCurveDrivenPatternFeatureData.D1AlignmentMethod = 0
    swCurveDrivenPatternFeatureData.D1CurveMethod = 0
    swCurveDrivenPatternFeatureData.D1InstanceCount = 3
    swCurveDrivenPatternFeatureData.D1IsEqualSpaced = True
    swCurveDrivenPatternFeatureData.D1ReverseDirection = False
    swCurveDrivenPatternFeatureData.D1Spacing = 0.0254

    Set swFeature = swFeatureMgr.CreateFeature(swCurveDrivenPatternFeatureData)

    'Access the curve-driven pattern feature
    status = swModelDocExt.SelectByID("CrvPattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swCurveDrivenPatternFeatureData = swFeature.GetDefinition
    status = swCurveDrivenPatternFeatureData.AccessSelections(swModel, Nothing)
    Debug.Print "Number of pattern instances in Direction 1: " & swCurveDrivenPatternFeatureData.D1InstanceCount
    Debug.Print "Alignment method of Direction 1: " & swCurveDrivenPatternFeatureData.D1AlignmentMethod
    Debug.Print "Curve method of Direction 1: " & swCurveDrivenPatternFeatureData.D1CurveMethod
    Set patternDirection = swCurveDrivenPatternFeatureData.D1Direction
    Set swEntity = patternDirection
    Debug.Print "Pattern direction object type of Direction 1: " & swEntity.GetType
    Debug.Print "Pattern instances spaced equally in Direction 1? " & swCurveDrivenPatternFeatureData.D1IsEqualSpaced
    Debug.Print "Pattern direction reversed in Direction 1? " & swCurveDrivenPatternFeatureData.D1ReverseDirection
    Debug.Print "Number of seed bodies in pattern: " & swCurveDrivenPatternFeatureData.GetPatternBodyCount
    Debug.Print "Number of seed features in pattern: " & swCurveDrivenPatternFeatureData.GetPatternFeatureCount
    swCurveDrivenPatternFeatureData.ReleaseSelectionAccess

```

End Sub

Create and Access Curve-driven Pattern Feature Example (VB.NET)

This example shows how to create a curve-driven pattern feature and access its data.

```

-----
Preconditions:
1. Verify that the specified part to open exists.
2. Open the Immediate window.

Postconditions:
1. Opens the specified part document.
2. Creates a cut extrude feature.
3. Creates a curve-driven pattern feature using the
  the cut extrude feature.
4. Gets curve-driven pattern feature data.
5. Examine the FeatureManager design tree, graphics area,
  and Immediate window.

NOTE: Because the part is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchMgr As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swFeatureMgr As FeatureManager
        Dim swFeature As Feature
        Dim swSelectionMgr As SelectionMgr
        Dim swCurveDrivenPatternFeatureData As CurveDrivenPatternFeatureData
        Dim swEntity As Entity
        Dim patternDirection As Object
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\bagel.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        'Sketch a circle and create a cut extrude
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID("", "FACE", 0.0118560192339032, 0, 0.0566664535234693, False, 0, Nothing, 0)
        swSketchMgr = swModel.SketchManager
        swSketchMgr.InsertSketch(True)
        swSketchSegment = swSketchMgr.CreateCircle(-0.059172, -0.048012, 0.0#, -0.044189, -0.040533, 0.0#)
        swSketchMgr.InsertSketch(True)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID("Sketch6", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        swFeatureMgr = swModel.FeatureManager
        swFeature = swFeatureMgr.FeatureCut3(True, False, False, 1, 0, 0.00254, 0.00254, False, False, False, 0.0174532925199433, 0.017453292519943
3, False, False, False, False, True, True, True, True, False, 0, 0, False)
        swSelectionMgr = swModel.SelectionManager
        swSelectionMgr.EnableContourSelection = False
        swModel.ActivateSelectedFeature()
        status = swModelDocExt.SelectByID("", "EDGE", 0.0115207253109588, -0.00000889643058599177, 0.0754182969300832, True, 0, Nothing, 0)
        swModel.ClearSelection2(True)

        'Create curve-driven pattern feature
        status = swModelDocExt.SelectByID("Cut-Extrude2", "BODYFEATURE", 0, 0, 0, False, 4, Nothing, 0)
        status = swModelDocExt.SelectByID("", "EDGE", 0.0115207253109588, -0.00000889643058599177, 0.0754182969300832, True, 1, Nothing, 0)

        swCurveDrivenPatternFeatureData = swFeatureMgr.CreateDefinition(swFeatureNameID_e.swFmCurvePattern)

        swCurveDrivenPatternFeatureData.D1AlignmentMethod = 0
        swCurveDrivenPatternFeatureData.D1CurveMethod = 0
        swCurveDrivenPatternFeatureData.D1InstanceCount = 3
        swCurveDrivenPatternFeatureData.D1IsEqualSpaced = True
        swCurveDrivenPatternFeatureData.D1ReverseDirection = False
        swCurveDrivenPatternFeatureData.D1Spacing = 0.0254

        swFeature = swFeatureMgr.CreateFeature(swCurveDrivenPatternFeatureData)

        'Access the curve-driven pattern feature
        status = swModelDocExt.SelectByID("CrvPattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swCurveDrivenPatternFeatureData = swFeature.GetDefinition
        status = swCurveDrivenPatternFeatureData.AccessSelections(swModel, Nothing)
        Debug.Print("Number of pattern instances in Direction 1: " & swCurveDrivenPatternFeatureData.D1InstanceCount)
        Debug.Print("Alignment method of Direction 1: " & swCurveDrivenPatternFeatureData.D1AlignmentMethod)
        Debug.Print("Curve method of Direction 1: " & swCurveDrivenPatternFeatureData.D1CurveMethod)
        patternDirection = swCurveDrivenPatternFeatureData.D1Direction
        swEntity = patternDirection
        Debug.Print("Pattern direction object type of Direction 1: " & swEntity.GetType)
        Debug.Print("Pattern instances spaced equally in Direction 1? " & swCurveDrivenPatternFeatureData.D1IsEqualSpaced)
        Debug.Print("Pattern direction reversed in Direction 1? " & swCurveDrivenPatternFeatureData.D1ReverseDirection)
        Debug.Print("Number of seed bodies in pattern: " & swCurveDrivenPatternFeatureData.GetPatternBodyCount)
        Debug.Print("Number of seed features in pattern: " & swCurveDrivenPatternFeatureData.GetPatternFeatureCount)
        swCurveDrivenPatternFeatureData.ReleaseSelectionAccess()

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Create and Arrange Windows Example (VBA)

This example shows how to create a new window containing the active document object and how to arrange all open windows.

```
'-----  
Option Explicit  
Sub main()  
    Dim swApp As SldWorks.SldWorks  
    Set swApp = Application.SldWorks  
    swApp.CreateNewWindow  
    ' Arrange:  
    ' 0 = Cascade  
    ' 1 = Tile horizontally  
    ' 2 = Tile vertically  
    swApp.ArrangeWindows 1  
End Sub  
'-----
```

Create and Convert Non-manifold Bodies Example (C#)

This example shows how to create non-manifold bodies, which by default are not allowed in SOLIDWORKS Design, and then convert the non-manifold bodies to manifold bodies.

```
//-----
// Preconditions:
// 1. Verify that the specified part document
//    template exists.
// 2. Add a reference to Microsoft.VisualBasic (right-click
//    the project name in the Project Explorer and click
//    Add Reference > Microsoft.VisualBasic on the .NET tab >
//    OK.
//
// Postconditions:
// 1. Set a breakpoint at this line:
//    swModeler.GeneralTopology = false;
// 2. Step through the macro by pressing F10
//    while observing the graphics area.
// 3. Creates and tessellates non-manifold bodies
//    and converts them to manifold bodies.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using Microsoft.VisualBasic;

namespace GeneralTopologyCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        ModelDoc2 swModel = default(ModelDoc2);

        public void Main()
        {
            Modeler swModeler = default(Modeler);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchManager = default(SketchManager);
            Feature swFeature = default(Feature);
            SelectionMgr swSelMgr = default(SelectionMgr);
            SketchSegment swSketchSegment = default(SketchSegment);
            FeatureManager swFeatureManager = default(FeatureManager);
            Tessellation tess = default(Tessellation);
            Body2 tool = default(Body2);
            Body2 tgt1 = default(Body2);
            Body2 tgt0 = default(Body2);
            object[] sketchLines = null;
            object[] resVar = null;
            object[] resvar2 = null;
            object[] manifVar = null;
            int[] vFacetId = null;
            int[] vFinId = null;
            int[] vVertexId = null;
            double[] vVertex1 = null;
            double[] vVertex2 = null;
            Face2 f = default(Face2);
            bool boolstatus = false;
            int i = 0;
            int j = 0;
            int[] clr = new int[2];

            swModeler = (Modeler)swApp.GetModeler();

            //SOLIDWORKS requires this option to be
            //false, so make sure it is set to false
            swModeler.GeneralTopology = false;

            //Create part having a tool and target bodies
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2016\\templates\\Part.prt", 0, 0, 0);
            swModelDocExt = swModel.Extension;
            boolstatus = swModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstToRectEntity, (int)swUserPreferenceOption_e.swDetailingNoOptionSpecified, false);
            boolstatus = swModelDocExt.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, (int)swUserPreferenceOption_e.swDetailingNoOptionSpecified, true);
            swSketchManager = swModel.SketchManager;
            sketchLines = (object[])swSketchManager.CreateCornerRectangle(0, 0, 0, 0.13786334229408, 0.069192775961991, 0);
            boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            boolstatus = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            boolstatus = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            boolstatus = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            boolstatus = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, false, 4, null, 0);
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureManager.FeatureExtrusion2(true, false, false, (int)swEndConditions_e.swEndCondBlind, 0, 0.01524, 0.00254, false, false, false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, true, true, true, (int)swStartConditions_e.swStartSketchPlane, 0, false);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swSelMgr.EnableContourSelection = false;
            swModel.ClearSelection2(true);
            swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, (int)swSelectOption_e.swSelectOptionDefault);
            swSketchManager.InsertSketch(true);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(0.0, 0.034596, 0.0, 0.137863, 0.034596, 0.0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(0.068932, 0.069193, 0.0, 0.068932, 0.0, 0.0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(0.0, 0.0, 0.0, 0.137863, 0.0, 0.0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(0.137863, 0.0, 0.0, 0.137863, 0.069193, 0.0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(0.137863, 0.069193, 0.0, 0.0, 0.069193, 0.0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(0.0, 0.069193, 0.0, 0.0, 0.0, 0.0);
            boolstatus = swModelDocExt.SelectByID2("Sketch2", "SKETCHREGION", 0.0295651111315002, 0.0562122082077101, 0.00761999999999985, true, 4, null, 0);
            boolstatus = swModelDocExt.SelectByID2("Sketch2", "SKETCHREGION", 0.0812426680973543, 0.0181340083381333, 0.00762000000000011, true, 4, null, 0);
            swModel.ClearSelection2(true);
            boolstatus = swModelDocExt.SelectByID2("Line9", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            boolstatus = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0.0295651111315002, 0.0562122082077101, 0.00761999999999985, true, 4, null, 0);
            swSelMgr.EnableContourSelection = true;
            boolstatus = swModelDocExt.SelectByID2("Sketch2", "SKETCHREGION", 0.0295651111315002, 0.0562122082077101, 0.00761999999999985, true, 4, null, 0);
            boolstatus = swModelDocExt.SelectByID2("Sketch2", "SKETCHREGION", 0.0812426680973543, 0.0181340083381333, 0.00762000000000011, true, 4, null, 0);
        }
    }
}
```

```

swFeature = (Feature)swFeatureManager.FeatureExtrusion2(true, false, false, (int)swEndConditions_e.swEndCondBlind, 0, 0.01524, 0.01524, false,
false, false,
false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, false, true, true,
(int)swStartConditions_e.swStartSketchPlane, 0, false);
swSelMgr.EnableContourSelection = false;

//Hide the boss-extrude and sketch features
boolstatus = swModelDocExt.SelectByID2("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
swFeatureManager.HideBodies();
boolstatus = swModelDocExt.SelectByID2("Boss-Extrude2", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
swFeatureManager.HideBodies();

//Make selections of tool and target bodies;
//Boss-Extrude1 is larger cube, whereas Boss-Extrude2[1]
//and Boss-Extrude2[2] are 1/4 the size of Boss-Extrude1,
//so (Boss-Extrude - Boss-Extrude2[1]) - Boss-Extrude2[2])
//results in non-manifold bodies; under normal conditions,
//i.e., when non-manifold bodies are not allowed,
//such an operation results in two bodies;
//when creation of non-manifold bodies is allowed,
//then one general body is the result
boolstatus = swModelDocExt.SelectByID2("Boss-Extrude1", "SOLIDBODY", 0, 0, 0, false, 0, null, 0);
boolstatus = swModelDocExt.SelectByID2("Boss-Extrude2[1]", "SOLIDBODY", 0, 0, 0, true, 0, null, 0);
boolstatus = swModelDocExt.SelectByID2("Boss-Extrude2[2]", "SOLIDBODY", 0, 0, 0, true, 0, null, 0);
tool = (Body2)swSelMgr.GetSelectedObject6(1, -1);
tgt0 = (Body2)swSelMgr.GetSelectedObject6(2, -1);
tgt1 = (Body2)swSelMgr.GetSelectedObject6(3, -1);

//Create temporary bodies
tool = (Body2)tool.Copy();
tgt0 = (Body2)tgt0.Copy();
tgt1 = (Body2)tgt1.Copy();

swModel.ClearSelection2(true);

//First cut operation : Boss-Extrude1 - Boss-Extrude2[1]
int errCode = 0;
resVar = (object[])tool.Operations2((int)swBodyOperationType_e.SWBODYCUT, tgt0, out errCode);

//SOLIDWORKS requires this option
//to be false; thus, switch it back to false
//as soon as your intended operations complete
swModeler.GeneralTopology = true;

//Second cut operation: (Boss-Extrude1 - Boss-Extrude2[1]) - Boss-Extrude2[2])
Body2 swBody = default(Body2);
swBody = (Body2)resVar[0];
resvar2 = (object[])swBody.Operations2((int)swBodyOperationType_e.SWBODYCUT, tgt1, out errCode);

//Reset the option back to false
swModeler.GeneralTopology = false;
clr[0] = Information.RGB(0, 0, 255);
clr[1] = Information.RGB(255, 0, 0);
for (i = 0; i < resvar2.Length; i++)
{
    DisplayBody(resvar2[i], clr[i]);
}

//Hide the displayed bodies
for (i = 0; i < resvar2.Length; i++)
{
    HideBody(resvar2[i]);
}

//Try tessellation

//Add sketch for this face
swModel.Insert3DSketch2(false);

//Add lines directly to sketch to increase performance
swModel.SetAddToDB(true);
Body2 swBody2 = default(Body2);
swBody2 = (Body2)resvar2[0];
tess = (Tessellation)swBody2.GetTessellation(null);
tess.NeedFaceFacetMap = true;
tess.MatchType = (int)swTessellationMatchType_e.swTessellationMatchFacetGeometry;
boolstatus = tess.Tessellate();
Face2 swFace = default(Face2);
f = (Face2)swBody2.GetFirstFace();
while ((f != null))
{
    vFacetId = (int[])tess.GetFaceFacets(f);
    for (i = 0; i < vFacetId.Length; i++)
    {
        vFinId = (int[])tess.GetFacetFins(vFacetId[i]);
        for (j = 0; j <= 2; j++)
        {
            //Should always be three fins per facet
            vVertexId = (int[])tess.GetFinVertices(vFinId[j]);
            //Should always be two vertices per fin
            vVertex1 = (double[])tess.GetVertexPoint(vVertexId[0]);
            vVertex2 = (double[])tess.GetVertexPoint(vVertexId[1]);
            swModel.CreateLine2(vVertex1[0], vVertex1[1], vVertex1[2], vVertex2[0], vVertex2[1], vVertex2[2]);
        }
    }
    f = (Face2)f.GetNextFace();
}

//Convert non-manifold bodies to manifold bodies
manifVar = (object[])swModeler.MakeManifoldBodies(swBody2);
for (i = 0; i < manifVar.Length; i++)
{
    DisplayBody(manifVar[i], Information.RGB(0, 255, 0));
}

swModel.ClearSelection2(true);

for (i = 0; i < manifVar.Length; i++)
{
    HideBody(manifVar[i]);
}
}

public void DisplayBody(object b, int col)
{
    Body2 body = null;
    body = (Body2)b;
    PartDoc swPart = default(PartDoc);
    swPart = (PartDoc)swModel;
}

```

```

        body.Display2(swPart, col, (int)swTempBodySelectOptions_e.swTempBodySelectable);
    }
    public void HideBody(object b)
    {
        Body2 body = null;
        body = (Body2)b;
        body.Hide(swModel);
    }

    /// <summary>
    ///   The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}

```

Create and Convert Non-manifold Bodies Example (VBA)

This example shows how to create non-manifold bodies, which by default are not allowed in SOLIDWORKS Design, and then convert the non-manifold bodies to manifold bodies.

```

'-----
' Preconditions: Verify that the specified part document
' template exists.
'
' Postconditions:
' 1. Click Debug > Step Into in the IDE.
' 2. Step through the rest of the macro by pressing F8
'    while observing the graphics area.
' 3. Creates and tessellates non-manifold bodies
'    and converts them to manifold bodies.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModeler As SldWorks.Modeler
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchManager As SldWorks.SketchManager
Dim swFeature As SldWorks.Feature
Dim swSelMgr As SldWorks.SelectionMgr
Dim swSketchSegment As SldWorks.SketchSegment
Dim swFeatureManager As SldWorks.FeatureManager
Dim tess As SldWorks.Tessellation
Dim tool As SldWorks.Body2
Dim tgt1 As SldWorks.Body2
Dim tgt0 As SldWorks.Body2
Dim sketchLines As Variant
Dim resVar As Variant
Dim resvar2 As Variant
Dim manifVar As Variant
Dim vFacetId As Variant
Dim vFinId As Variant
Dim vVertexId As Variant
Dim vVertex1 As Variant
Dim vVertex2 As Variant
Dim f As Object
Dim boolstatus As Boolean
Dim i As Long
Dim j As Long
Dim clr(0 To 1) As Long

Sub DisplayBody(ByVal b As Object, col As Long)
    Call b.Display2(swModel, col, swTempBodySelectOptions_e.swTempBodySelectable)
End Sub

Sub HideBody(ByVal b As Object)
    Call b.Hide(swModel)
End Sub

Sub main()

    Set swApp = Application.SldWorks

    Set swModeler = swApp.GetModeler
    'SOLIDWORKS requires this option to be
    'false, so make sure it is set to false
    swModeler.GeneralTopology = False

    'Create part having a tool and target bodies
    Set swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
    Set swModelDocExt = swModel.Extension
    boolstatus = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, False)
    boolstatus = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, True)
    Set swSketchManager = swModel.SketchManager
    sketchLines = swSketchManager.CreateCornerRectangle(0, 0, 0, 0.13786334229408, 0.069192775961991, 0)
    boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    boolstatus = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    boolstatus = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    boolstatus = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    boolstatus = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)
    Set swFeatureManager = swModel.FeatureManager
    Set swFeature = swFeatureManager.FeatureExtrusion2(True, False, False, swEndConditions_e.swEndCondBlind, 0, 0.01524, 0.00254, False, False, False,
False, 1.74532925199433E-02, 1.74532925199433E-02, False, False, False, False, True, True, True, swStartConditions_e.swStartSketchPlane, 0, False)
    Set swSelMgr = swModel.SelectionManager
    swSelMgr.EnableContourSelection = False
    swModel.ClearSelection2 True
    swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    swSketchManager.InsertSketch (True)
    Set swSketchSegment = swSketchManager.CreateLine(0#, 0.034596, 0#, 0.137863, 0.034596, 0#)
    Set swSketchSegment = swSketchManager.CreateLine(0.068932, 0.069193, 0#, 0.068932, 0#, 0#)
    Set swSketchSegment = swSketchManager.CreateLine(0#, 0#, 0#, 0.137863, 0#, 0#)
    Set swSketchSegment = swSketchManager.CreateLine(0.137863, 0#, 0#, 0.137863, 0.069193, 0#)
    Set swSketchSegment = swSketchManager.CreateLine(0.137863, 0.069193, 0#, 0#, 0.069193, 0#)
    Set swSketchSegment = swSketchManager.CreateLine(0#, 0.069193, 0#, 0.069193, 0#, 0#)
    boolstatus = swModelDocExt.SelectByID2("Sketch2", "SKETCHREGION", 2.95651111315002E-02, 5.62122082077101E-02, 7.61999999999985E-03, True, 4, Nothing,
0)
    boolstatus = swModelDocExt.SelectByID2("Sketch2", "SKETCHREGION", 8.12426680973543E-02, 1.81340083381333E-02, 7.62000000000011E-03, True, 4, Nothing,
0)
    swModel.ClearSelection2 True
    boolstatus = swModelDocExt.SelectByID2("Line9", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)

```

```

boolstatus = swModelDocExt.SelectByID("Sketch2", "SKETCH", 2.95651111315002E-02, 5.62122082077101E-02, 7.61999999999985E-03, True, 4, Nothing, 0)
swSelMgr.EnableContourSelection = True
boolstatus = swModelDocExt.SelectByID("Sketch2", "SKETCHREGION", 2.95651111315002E-02, 5.62122082077101E-02, 7.61999999999985E-03, True, 4, Nothing, 0)
boolstatus = swModelDocExt.SelectByID("Sketch2", "SKETCHREGION", 8.12426680973543E-02, 1.81340083381333E-02, 7.62000000000011E-03, True, 4, Nothing, 0)
Set swFeature = swFeatureManager.FeatureExtrusion2(True, False, False, swEndConditions_e.swEndCondBlind, 0, 0.01524, 0.01524, False, False, False, False, 1.74532925199433E-02, 1.74532925199433E-02, False, False, False, False, False, True, True, swStartConditions_e.swStartSketchPlane, 0, False)
swSelMgr.EnableContourSelection = False

'Hide the boss-extrude and sketch features
boolstatus = swModelDocExt.SelectByID("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
swFeatureManager.HideBodies
boolstatus = swModelDocExt.SelectByID("Boss-Extrude2", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
swFeatureManager.HideBodies

'Make selections of tool and target bodies;
'Boss-Extrude1 is larger cube, whereas Boss-Extrude2[1]
'and Boss-Extrude2[2] are 1/4 the size of Boss-Extrude1,
'so (Boss-Extrude - Boss-Extrude2[1])-Boss-Extrude2[2])
'results in non-manifold bodies; under normal conditions,
'i.e., when non-manifold bodies are not allowed,
'such an operation results in two bodies;
'when creation of non-manifold bodies is allowed,
'then one general body is the result
boolstatus = swModelDocExt.SelectByID("Boss-Extrude1", "SOLIDBODY", 0, 0, 0, False, 0, Nothing, 0)
boolstatus = swModelDocExt.SelectByID("Boss-Extrude2[1]", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
boolstatus = swModelDocExt.SelectByID("Boss-Extrude2[2]", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)

Set tool = swSelMgr.GetSelectedObject6(1, -1)
Set tgt0 = swSelMgr.GetSelectedObject6(2, -1)
Set tgt1 = swSelMgr.GetSelectedObject6(3, -1)

'Create temporary bodies
Set tool = tool.Copy
Set tgt0 = tgt0.Copy
Set tgt1 = tgt1.Copy

swModel.ClearSelection2 True

'First cut operation : Boss-Extrude1 - Boss-Extrude2[1]
Dim errCode As Long
resVar = tool.Operations2(swBodyOperationType_e.SWBODYCUT, tgt0, errCode)

'SOLIDWORKS requires this option
'to be false; thus, switch it back to false
'as soon as your intended operations complete
swModeler.GeneralTopology = True

'Second cut operation: (Boss-Extrude1 - Boss-Extrude2[1])- Boss-Extrude2[2])
resvar2 = resVar(0).Operations2(swBodyOperationType_e.SWBODYCUT, tgt1, errCode)

'Reset the option back to false
swModeler.GeneralTopology = False

clr(0) = RGB(0, 0, 255)
clr(1) = RGB(255, 0, 0)
For i = LBound(resvar2) To UBound(resvar2)
    Call DisplayBody(resvar2(i), clr(i))
Next i

'Hide the displayed bodies
For i = LBound(resvar2) To UBound(resvar2)
    HideBody (resvar2(i))
Next i

'Try tessellation

'Add sketch for this face
swModel.Insert3DSketch2 False

'Add lines directly to sketch to increase performance
swModel.SetAddToDB True
Set tess = resvar2(0).GetTessellation(Empty)
tess.NeedFaceFacetMap = True
tess.MatchType = swTessellationMatchFacetGeometry
boolstatus = tess.Tessellate
Set f = resvar2(0).GetFirstFace
While Not f Is Nothing
    vFacetId = tess.GetFaceFacets(f)
    For i = 0 To UBound(vFacetId)
        vFinId = tess.GetFacetFins(vFacetId(i))
        For j = 0 To 2
            'Should always be three fins per facet
            vVertexId = tess.GetFinVertices(vFinId(j))
            'Should always be two vertices per fin
            vVertex1 = tess.GetVertexPoint(vVertexId(0))
            vVertex2 = tess.GetVertexPoint(vVertexId(1))
            Call swModel.CreateLine2(vVertex1(0), vVertex1(1), vVertex1(2), vVertex2(0), vVertex2(1), vVertex2(2))
        Next j
    Next i
    Set f = f.GetNextFace
Wend

'Convert non-manifold bodies to manifold bodies
manifVar = swModeler.MakeManifoldBodies(resvar2(0))
For i = LBound(manifVar) To UBound(manifVar)
    Call DisplayBody(manifVar(i), RGB(0, 255, 0))
Next i

swModel.ClearSelection2 True

For i = LBound(manifVar) To UBound(manifVar)
    HideBody (manifVar(i))

```

Next i

End Sub

Create and Convert Non-manifold Bodies Example (VB.NET)

This example shows how to create non-manifold bodies, which by default are not allowed in SOLIDWORKS Design, and then convert the non-manifold bodies to manifold bodies.

```

-----
Preconditions: Verify that the specified part document
template exists.
Postconditions:
1. Set a breakpoint at this line:
   swModeler.GeneralTopology = False
2. Step through the macro by pressing F10
   while observing the graphics area.
3. Creates and tessellates non-manifold bodies
   and converts them to manifold bodies.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2

    Public Sub main()

        Dim swModeler As Modeler
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchManager As SketchManager
        Dim swFeature As Feature
        Dim swSelMgr As SelectionMgr
        Dim swSketchSegment As SketchSegment
        Dim swFeatureManager As FeatureManager
        Dim tess As Tessellation
        Dim tool As Body2
        Dim tgt1 As Body2
        Dim tgt0 As Body2
        Dim sketchLines As Object
        Dim resVar As Object
        Dim resvar2 As Object
        Dim manifVar As Object
        Dim vFacetId As Object
        Dim vFinId As Object
        Dim vVertexId As Object
        Dim vVertex1 As Object
        Dim vVertex2 As Object
        Dim f As Object
        Dim boolstatus As Boolean
        Dim i As Integer
        Dim j As Integer
        Dim clr(0 To 1) As Integer

        swModeler = swApp.GetModeler

        'SOLIDWORKS requires this option to be
        'false, so make sure it is set to false
        swModeler.GeneralTopology = False

        'Create part having a tool and target bodies
        swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\templates\Part.prt", 0, 0, 0)
        swModelDocExt = swModel.Extension
        boolstatus = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity, swUserPreferenceOption_e.swDetailingNoOp
tionSpecified, False)
        boolstatus = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType, swUserPreferenceOption_e.swDetailing
NoOptionSpecified, True)
        swSketchManager = swModel.SketchManager
        sketchLines = swSketchManager.CreateCornerRectangle(0, 0, 0, 0.13786334229408, 0.069192775961991, 0)
        boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
        boolstatus = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        boolstatus = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        boolstatus = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        boolstatus = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0, 0, 0, False, 4, Nothing, 0)
        swFeatureManager = swModel.FeatureManager
        swFeature = swFeatureManager.FeatureExtrusion2(True, False, False, swEndConditions_e.swEndCondBlind, 0, 0.01524, 0.00254, False, False, False, Fal
se, 0.0174532925199433, 0.0174532925199433, False, False, False, False, True, True, True, swStartConditions_e.swStartSketchPlane, 0, False)
        swSelMgr = swModel.SelectionManager
        swSelMgr.EnableContourSelection = False
        swModel.ClearSelection2(True)
        swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swSketchManager.InsertSketch(True)
        swSketchSegment = swSketchManager.CreateLine(0.0#, 0.034596, 0.0#, 0.137863, 0.034596, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(0.068932, 0.069193, 0.0#, 0.068932, 0.0#, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(0.0#, 0.0#, 0.0#, 0.137863, 0.0#, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(0.137863, 0.0#, 0.0#, 0.137863, 0.069193, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(0.137863, 0.069193, 0.0#, 0.0#, 0.069193, 0.0#)
        swSketchSegment = swSketchManager.CreateLine(0.0#, 0.069193, 0.0#, 0.0#, 0.0#, 0.0#)
        boolstatus = swModelDocExt.SelectByID2("Sketch2", "SKETCHREGION", 0.0295651111315002, 0.0562122082077101, 0.00761999999999985, True, 4, Nothing, 0)
    )
    boolstatus = swModelDocExt.SelectByID2("Sketch2", "SKETCHREGION", 0.0812426680973543, 0.0181340083381333, 0.00762000000000011, True, 4, Nothing, 0)
    )
    swModel.ClearSelection2(True)
    boolstatus = swModelDocExt.SelectByID2("Line9", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    boolstatus = swModelDocExt.SelectByID2("Sketch2", "SKETCH", 0.0295651111315002, 0.0562122082077101, 0.00761999999999985, True, 4, Nothing, 0)
    swSelMgr.EnableContourSelection = True
    boolstatus = swModelDocExt.SelectByID2("Sketch2", "SKETCHREGION", 0.0295651111315002, 0.0562122082077101, 0.00761999999999985, True, 4, Nothing, 0)
    )
    boolstatus = swModelDocExt.SelectByID2("Sketch2", "SKETCHREGION", 0.0812426680973543, 0.0181340083381333, 0.00762000000000011, True, 4, Nothing, 0)
    )
    swFeature = swFeatureManager.FeatureExtrusion2(True, False, False, swEndConditions_e.swEndCondBlind, 0, 0.01524, 0.01524, False, False, False, Fal
se, 0.0174532925199433, 0.0174532925199433, False, False, False, False, True, True, True, swStartConditions_e.swStartSketchPlane, 0, False)
    swSelMgr.EnableContourSelection = False

    'Hide the boss-extrude and sketch features
    boolstatus = swModelDocExt.SelectByID2("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    swFeatureManager.HideBodies()
    boolstatus = swModelDocExt.SelectByID2("Boss-Extrude2", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    swFeatureManager.HideBodies()

```

```

'Make selections of tool and target bodies;
'Boss-Extrude1 is larger cube, whereas Boss-Extrude2[1]
'and Boss-Extrude2[2] are 1/4 the size of Boss-Extrude1,
'so (Boss-Extrude - Boss-Extrude2[1]) - Boss-Extrude2[2])
'results in non-manifold bodies; under normal conditions,
'i.e., when non-manifold bodies are not allowed,
'such an operation results in two bodies;
'when creation of non-manifold bodies is allowed,
'then one general body is the result
boolstatus = swModelDocExt.SelectByID2("Boss-Extrude1", "SOLIDBODY", 0, 0, 0, False, 0, Nothing, 0)
boolstatus = swModelDocExt.SelectByID2("Boss-Extrude2[1]", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
boolstatus = swModelDocExt.SelectByID2("Boss-Extrude2[2]", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
tool = swSelMgr.GetSelectedObject6(1, -1)
tgt0 = swSelMgr.GetSelectedObject6(2, -1)
tgt1 = swSelMgr.GetSelectedObject6(3, -1)

'Create temporary bodies
tool = tool.Copy
tgt0 = tgt0.Copy
tgt1 = tgt1.Copy

swModel.ClearSelection2(True)

'First cut operation : Boss-Extrude1 - Boss-Extrude2[1]
Dim errCode As Integer
resVar = tool.Operations2(swBodyOperationType_e.SWBODYCUT, tgt0, errCode)

'SOLIDWORKS requires this option
'to be false; thus, switch it back to false
'as soon as your intended operations complete
swModeler.GeneralTopology = True

'Second cut operation: (Boss-Extrude1 - Boss-Extrude2[1]) - Boss-Extrude2[2])
Dim swBody As Body2
swBody = resVar(0)
resvar2 = swBody.Operations2(swBodyOperationType_e.SWBODYCUT, tgt1, errCode)

'Reset the option back to false
swModeler.GeneralTopology = False
clr(0) = RGB(0, 0, 255)
clr(1) = RGB(255, 0, 0)
For i = LBound(resvar2) To UBound(resvar2)
    Call DisplayBody(resvar2(i), clr(i))
Next i

'Hide the displayed bodies
For i = LBound(resvar2) To UBound(resvar2)
    HideBody(resvar2(i))
Next i

'Try tessellation
'Add sketch for this face
swModel.Insert3DSketch2(False)

'Add lines directly to sketch to increase performance
swModel.SetAddToDB(True)
tess = resvar2(0).GetTessellation(Nothing)
tess.NeedFaceFacetMap = True
tess.MatchType = swTessellationMatchType_e.swTessellationMatchFacetGeometry
boolstatus = tess.Tessellate
f = resvar2(0).GetFirstFace
While Not f Is Nothing
    vFacetId = tess.GetFaceFacets(f)
    For i = 0 To UBound(vFacetId)
        vFinId = tess.GetFacetFins(vFacetId(i))
        For j = 0 To 2
            'Should always be three fins per facet
            vVertexId = tess.GetFinVertices(vFinId(j))
            'Should always be two vertices per fin
            vVertex1 = tess.GetVertexPoint(vVertexId(0))
            vVertex2 = tess.GetVertexPoint(vVertexId(1))
            Call swModel.CreateLine2(vVertex1(0), vVertex1(1), vVertex1(2), vVertex2(0), vVertex2(1), vVertex2(2))
        Next j
    Next i
    f = f.GetNextFace
End While

'Convert non-manifold bodies to manifold bodies
manifVar = swModeler.MakeManifoldBodies(resvar2(0))
For i = LBound(manifVar) To UBound(manifVar)
    Call DisplayBody(manifVar(i), RGB(0, 255, 0))
Next i

swModel.ClearSelection2(True)

For i = LBound(manifVar) To UBound(manifVar)
    HideBody(manifVar(i))
Next i

End Sub

Sub DisplayBody(ByVal b As Object, ByVal col As Integer)
    Dim body As Body2 = Nothing
    body = b
    Call body.Display2(swModel, col, swTempBodySelectOptions_e.swTempBodySelectable)
End Sub

Sub HideBody(ByVal b As Object)
    Dim body As Body2 = Nothing
    body = b
    Call body.Hide(swModel)
End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Create and Delete Selection Sets Example (C#)

This example shows how to:

- create selection sets.
- get the Selection Sets folder.
- get the selection sets in the Selection Sets folder.
- get and then delete the items in the selection sets, which deletes the selection sets.

```
//-----
// Preconditions:
// 1. Verify that the assembly to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the assembly.
// 2. Creates two selection sets.
// 3. Traverses the FeatureManager design tree. If the feature is the
// Selection Sets folder, then:
// a. Gets its name.
// b. Gets each selection set in the Selection Sets folder.
// c. Gets and then deletes each selection set item in the
// selection set, which deletes the selection set.
// 4. Examine the Immediate window.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExtension = default(ModelDocExtension);
            SelectionSet swSelectionSet1 = default(SelectionSet);
            SelectionSet swSelectionSet2 = default(SelectionSet);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\wrench.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref e
            rrors, ref warnings);
            Debug.Print("File = " + swModel.GetPathName());
            swModelDocExtension = swModel.Extension;

            //Create a selection set
            status = swModelDocExtension.SelectByID2("arm1-1@wrench", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            status = swModelDocExtension.SelectByID2("arm2-1@wrench", "COMPONENT", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExtension.SelectByID2("clamp2-1@wrench", "COMPONENT", 0, 0, 0, true, 0, null, 0);
            swSelectionSet1 = (SelectionSet)swModelDocExtension.SaveSelection(out errors);
            Debug.Print(" First selection set created (1 = succeeded; 0 = failed)? " + errors);
            swModel.ClearSelection2(true);

            //Create another selection set
            status = swModelDocExtension.SelectByID2("centerlink-1@wrench", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            status = swModelDocExtension.SelectByID2("screw-1@wrench", "COMPONENT", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExtension.SelectByID2("screw-1@wrench", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            swSelectionSet2 = (SelectionSet)swModelDocExtension.SaveSelection(out errors);
            Debug.Print(" Second selection set created (1 = succeeded; 0 = failed)? " + errors);
            swModel.ClearSelection2(true);

            //Traverse the model to get Selection Sets folder
            TraverseModelFeatures(swModel);
        }

        public void TraverseFeatureFeatures(Feature swFeat)
        {
            SelectionSetFolder swSelectionSetFolder = default(SelectionSetFolder);
            object[] selectionSetArray = null;
            object[] selectionSetItemArray = null;
            int[] selectionSetItemArrayTypes = null;
            SelectionSet swSelectionSet = default(SelectionSet);
            SelectionSetItem swSelectionSetItem = default(SelectionSetItem);
            int i = 0;
            int j = 0;

            while ((swFeat != null))
            {
                if (swFeat.Name == "Selection Sets")
                {
                    Debug.Print(" " + swFeat.Name + " [" + swFeat.GetTypeName() + "]");
                    //Get Selection Sets folder
                    swSelectionSetFolder = (SelectionSetFolder)swFeat.GetSpecificFeature2();
                    //Get selection sets in Selection Sets folder
                    selectionSetArray = (object[])swSelectionSetFolder.GetSelectionSets();
                    for (i = 0; i < selectionSetArray.GetLength(0); i++)
                    {
                        swSelectionSet = (SelectionSet)selectionSetArray[i];
                        Debug.Print(" Selection set[" + i + "] name: " + swSelectionSet.GetName());
                        //Get the items in this selection set
                        selectionSetItemArray = (object[])swSelectionSet.GetSelectionSetItems();
                        selectionSetItemArrayTypes = (int[])swSelectionSet.GetSelectionSetItemTypes();
                        for (j = 0; j < selectionSetItemArray.GetLength(0); j++)
                        {
                            swSelectionSetItem = (SelectionSetItem)selectionSetItemArray[j];
                            Debug.Print(" Item[" + j + "] in this selection set is this entity type as defined in swSelectType_e: " +
                            selectionSetItemArrayTypes[j]);
                            //Delete each item in this selection set
                            swSelectionSetItem.RemoveFromSelectionSet();
                            Debug.Print(" Selection set item[" + j + "] deleted");
                        }
                    }
                }
            }
        }
    }
}
```

```

        }
    }
    }
    swFeat = (Feature)swFeat.GetNextFeature();
}

}

public void TraverseModelFeatures(ModelDoc2 swModel)
{
    Feature swFeat = default(Feature);

    swFeat = (Feature)swModel.FirstFeature();
    TraverseFeatureFeatures(swFeat);
}

/// <summary>
///   The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Create and Delete Selection Sets Example (VBA)

This example shows how to:

- create selection sets.
- get the Selection Sets folder.
- get the selection sets in the Selection Sets folder.
- get and then delete the items in the selection sets, which deletes the selection sets.

```

'-----
' Preconditions:
' 1. Verify that the assembly to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the assembly.
' 2. Creates two selection sets.
' 3. Traverses the FeatureManager design tree. If the feature is the
' Selection Sets folder, then:
'   a. Gets its name.
'   b. Gets each selection set in the Selection Sets folder.
'   c. Gets and then deletes each selection set item in the
'       selection set, which deletes the selection set.
' 4. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Option Explicit

Sub TraverseFeatureFeatures(swFeat As SldWorks.Feature)

    Dim swSelectionSetFolder As SldWorks.SelectionSetFolder
    Dim selectionSetArray As Variant
    Dim selectionSetItemArray As Variant
    Dim selectionSetItemArrayTypes As Variant
    Dim swSelectionSet As SldWorks.SelectionSet
    Dim swSelectionSetItem As SldWorks.SelectionSetItem
    Dim i As Long
    Dim j As Long

    While Not swFeat Is Nothing
        If swFeat.Name = "Selection Sets" Then
            Debug.Print "    " & swFeat.Name & " [" & swFeat.GetTypeName & "]"
            'Get Selection Sets folder
            Set swSelectionSetFolder = swFeat.GetSpecificFeature2
            'Get selection sets in Selection Sets folder
            selectionSetArray = swSelectionSetFolder.GetSelectionSets
            For i = 0 To UBound(selectionSetArray)
                Set swSelectionSet = selectionSetArray(i)
                Debug.Print "        Selection set[" & i & "] name: " & swSelectionSet.GetName
                'Get the items in this selection set
                selectionSetItemArray = swSelectionSet.GetSelectionSetItems
                selectionSetItemArrayTypes = swSelectionSet.GetSelectionSetItemTypes
                For j = 0 To UBound(selectionSetItemArray)
                    Set swSelectionSetItem = selectionSetItemArray(j)
                    Debug.Print "            Item[" & j & "] in this selection set is this entity type as defined in swSelectType_e: " &
selectionSetItemArrayTypes(j)
                    'Delete each item in this selection set
                    swSelectionSetItem.RemoveFromSelectionSet
                    Debug.Print ("                Selection set item[" & j & "] deleted")
                Next
            Next
        End If
        Set swFeat = swFeat.GetNextFeature
    Wend

End Sub

Sub TraverseModelFeatures(swModel As SldWorks.ModelDoc2)

    Dim swFeat As SldWorks.Feature

    Set swFeat = swModel.FirstFeature
    TraverseFeatureFeatures swFeat

End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExtension As SldWorks.ModelDocExtension
    Dim swSelectionSet1 As SldWorks.SelectionSet
    Dim swSelectionSet2 As SldWorks.SelectionSet
    Dim status As Boolean
    Dim errors As Long
    Dim warnings As Long
    Dim fileName As String

    Set swApp = CreateObject("SldWorks.Application")

    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\wrench.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Debug.Print "File = " & swModel.GetPathName
    Set swModelDocExtension = swModel.Extension

```

```

'Create a selection set
status = swModelDocExtension.SelectByID("arm1-1@wrench", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
status = swModelDocExtension.SelectByID("arm2-1@wrench", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
status = swModelDocExtension.SelectByID("clamp2-1@wrench", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
Set swSelectionSet1 = swModelDocExtension.SaveSelection(errors)
Debug.Print " First selection set created (1 = succeeded; 0 = failed)? " & errors
swModel.ClearSelection2 True

'Create another selection set
status = swModelDocExtension.SelectByID("centerlink-1@wrench", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
status = swModelDocExtension.SelectByID("screw-1@wrench", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
status = swModelDocExtension.SelectByID("screw-1@wrench", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
Set swSelectionSet2 = swModelDocExtension.SaveSelection(errors)
Debug.Print " Second selection set created (1 = succeeded; 0 = failed)? " & errors
swModel.ClearSelection2 True

'Traverse the model to get Selection Sets folder
TraverseModelFeatures swModel

```

End Sub

Create and Delete Selection Sets Example (VB.NET)

This example shows how to:

- create selection sets.
- get the Selection Sets folder.
- get the selection sets in the Selection Sets folder.
- get and then delete the items in the selection sets, which deletes the selection sets.

```

'-----
' Preconditions:
' 1. Verify that the assembly to open exists.
' 2. Open the Immediate window.
'-----
' Postconditions:
' 1. Opens the assembly.
' 2. Creates two selection sets.
' 3. Traverses the FeatureManager design tree. If the feature is the
'    Selection Sets folder, then:
'    a. Gets its name.
'    b. Gets each selection set in the Selection Sets folder.
'    c. Gets and then deletes each selection set item in the
'       selection set, which deletes the selection set.
' 4. Examine the Immediate window.
'-----
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExtension As ModelDocExtension
        Dim swSelectionSet1 As SelectionSet
        Dim swSelectionSet2 As SelectionSet
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\wrench.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        Debug.Print("File = " & swModel.GetPathName)
        swModelDocExtension = swModel.Extension

        'Create a selection set
        status = swModelDocExtension.SelectByID2("arm1-1@wrench", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        status = swModelDocExtension.SelectByID2("arm2-1@wrench", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExtension.SelectByID2("clamp2-1@wrench", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
        swSelectionSet1 = swModelDocExtension.SaveSelection(errors)
        Debug.Print(" First selection set created (1 = succeeded; 0 = failed)? " & errors)
        swModel.ClearSelection2(True)

        'Create another selection set
        status = swModelDocExtension.SelectByID2("centerlink-1@wrench", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        status = swModelDocExtension.SelectByID2("screw-1@wrench", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExtension.SelectByID2("screw-1@wrench", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionSet2 = swModelDocExtension.SaveSelection(errors)
        Debug.Print(" Second selection set created (1 = succeeded; 0 = failed)? " & errors)
        swModel.ClearSelection2(True)

        'Traverse the model to get Selection Sets folder
        TraverseModelFeatures(swModel)

    End Sub

    Sub TraverseFeatureFeatures(ByVal swFeat As Feature)

        Dim swSelectionSetFolder As SelectionSetFolder
        Dim selectionSetArray As Object
        Dim selectionSetItemArray As Object
        Dim selectionSetItemArrayTypes As Object
        Dim swSelectionSet As SelectionSet
        Dim swSelectionSetItem As SelectionSetItem
        Dim i As Integer
        Dim j As Integer

        While Not swFeat Is Nothing
            If swFeat.Name = "Selection Sets" Then
                Debug.Print(" " & swFeat.Name & " [" & swFeat.GetTypeName & "]")
                'Get Selection Sets folder
                swSelectionSetFolder = swFeat.GetSpecificFeature2
                'Get selection sets in Selection Sets folder
                selectionSetArray = swSelectionSetFolder.GetSelectionSets
                For i = 0 To UBound(selectionSetArray)
                    swSelectionSet = selectionSetArray(i)
                    Debug.Print(" Selection set[" & i & "] name: " & swSelectionSet.GetName)
                    'Get the items in this selection set
                    selectionSetItemArray = swSelectionSet.GetSelectionSetItems
                    selectionSetItemArrayTypes = swSelectionSet.GetSelectionSetItemTypes
                    For j = 0 To UBound(selectionSetItemArray)
                        swSelectionSetItem = selectionSetItemArray(j)
                        Debug.Print(" Item[" & j & "] in this selection set is this entity type as defined in swSelectType_e: " &
                        selectionSetItemArrayTypes(j))
                        'Delete each item in this selection set
                        swSelectionSetItem.RemoveFromSelectionSet()
                        Debug.Print(" Selection set item[" & j & "] deleted")
                    Next
                Next
            End If
            swFeat = swFeat.GetNextFeature
        End While

    End Sub

End Sub

```

```
Sub TraverseModelFeatures(ByVal swModel As ModelDoc2)

    Dim swFeat As Feature

    swFeat = swModel.FirstFeature
    TraverseFeatureFeatures(swFeat)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class
```

Create and Edit Circular Sketch Pattern Example (C#)

This example shows how to create and edit a circular sketch pattern.

```
//-----
// Preconditions: Verify that the specified part document template
// exists.
//
// Postconditions:
// 1. Opens a new part document and creates a sketch.
// 2. Inserts a circular sketch pattern of four instances.
// 3. Closes the sketch.
// 4. Opens the circular sketch pattern for editing.
// 5. Deletes an instance of the circular sketch pattern, leaving
//    three instances.
// 6. Examine the graphics area.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchMgr = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            object vSkLines = null;
            bool boolstatus = false;
            int longstatus = 0;

            // Reset the counts for untitled documents for this macro
            swApp.ResetUntitledCount(0, 0, 0);

            // Create a part document
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2016\\templates\\Part.prtdot", 0, 0,
0);
            swApp.ActivateDoc2("Part1", false, ref longstatus);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSketchMgr = swModel.SketchManager;
            swModelDocExt = swModel.Extension;

            // Sketch a circle
            swSketchSegment = swSketchMgr.CreateCircle(0.0, 0.0, 0.0, 0.045549, 0.013926, 0.0);

            // Clear any selections and change
            // the view orientation to Front
            swModel.ClearSelection2(true);
            swModel.ShowNamedView2("**Front", 1);

            // Create a rectangle
            vSkLines = swSketchMgr.CreateCornerRectangle(-0.005867589431389, 0.03694408160504, 0, 0.004563680668858,
0.02673012963188, 0);
            // Create a circular sketch pattern
            // using the rectangle
            boolstatus = swSketchMgr.CreateCircularSketchStepAndRepeat(0.03184378021964, 4.732863934409, 4, 1.570796326795, true,
"", true, true, true);
            swModel.ClearSelection2(true);

            // Close the sketch and rebuild
            swSketchMgr.InsertSketch(true);

            // Select an entity in the circular sketch pattern
            // and open the circular sketch pattern to edit it
            boolstatus = swModelDocExt.SelectByID2("Line1@Sketch1", "EXTSKETCHSEGMENT", -0.002390499397973, 0.03694408160504, 0,
false, 0, null, 0);

            swModel.EditSketch();

            // Delete an instance of the circular
            // sketch pattern and close the sketch
            boolstatus = swSketchMgr.EditCircularSketchStepAndRepeat(0.03184378021964, 4.732863934409, 3, 1.570796326795, true,
"", true, true, true, "Line2_Line1_Line4_Line3");
            swModel.ClearSelection2(true);
            swSketchMgr.InsertSketch(true);
        }
    }
}

public SldWorks swApp;
}
```

Create and Edit Circular Sketch Pattern Example (VBA)

This example shows how to create and edit a circular sketch pattern.

```

'-----
' Preconditions: Verify that the specified part document template
' exists.
'
' Postconditions:
' 1. Opens a new part document and creates a sketch.
' 2. Inserts a circular sketch pattern of four instances.
' 3. Closes the sketch.
' 4. Opens the circular sketch pattern for editing.
' 5. Deletes an instance of circular sketch pattern, leaving
'    three instances.
' 6. Examine the graphics area.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchMgr As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim vSkLines As Variant
Dim boolstatus As Boolean
Dim longstatus As Long

Sub main()

    Set swApp = Application.SldWorks

    ' Reset the counts for untitled documents for this macro
    swApp.ResetUntitledCount 0, 0, 0

    ' Create part document
    Set swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
    swApp.ActivateDoc2 "Part1", False, longstatus
    Set swModel = swApp.ActiveDoc
    Set swSketchMgr = swModel.SketchManager
    Set swModelDocExt = swModel.Extension

    ' Sketch a circle
    Set swSketchSegment = swSketchMgr.CreateCircle(0#, 0#, 0#, 0.045549, 0.013926, 0#)

    ' Clear any selections and change
    ' the view orientation to Front
    swModel.ClearSelection2 True
    swModel.ShowNamedView2 "**Front", 1

    ' Create a rectangle
    vSkLines = swSketchMgr.CreateCornerRectangle(-0.005867589431389, 0.03694408160504, 0, 0.004563680668858, 0.02673012963188, 0)

    ' Create a circular sketch pattern
    ' using the rectangle
    boolstatus = swSketchMgr.CreateCircularSketchStepAndRepeat(0.03184378021964, 4.732863934409, 4, 1.570796326795, True, "", True, True, True)
    swModel.ClearSelection2 True

    ' Close the sketch and rebuild
    swSketchMgr.InsertSketch True

    ' Select an entity in the circular sketch pattern
    ' and open the circular sketch pattern to edit it
    boolstatus = swModelDocExt.SelectByID("Line1@Sketch1", "EXTSKETCHSEGMENT", -0.002390499397973, 0.03694408160504, 0, False, 0, Nothing, 0)
    swModel.EditSketch

    ' Delete an instance of the circular
    ' sketch pattern and close the sketch
    boolstatus = swSketchMgr.EditCircularSketchStepAndRepeat(0.03184378021964, 4.732863934409, 3, 1.570796326795, True, "", True, True, True,
"Line2_Line1_Line4_Line3")
    swModel.ClearSelection2 True
    swSketchMgr.InsertSketch True

End Sub

```

Create and Edit Circular Sketch Pattern Example (VB.NET)

This example shows how to create and edit a circular sketch pattern.

```

'-----
' Preconditions: Verify that the specified part document template
' exists.
'
' Postconditions:
' 1. Opens a new part document and creates a sketch.
' 2. Inserts a circular sketch pattern of four instances.
' 3. Closes the sketch.
' 4. Opens the circular sketch pattern for editing.
' 5. Deletes an instance of circular sketch pattern, leaving
'    three instances.
' 6. Examine the graphics area.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

Partial Class SolidWorksMacro
    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchMgr As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim vSkLines As Object
        Dim boolstatus As Boolean
        Dim longstatus As Long

        ' Reset the counts for untitled documents for this macro
        swApp.ResetUntitledCount(0, 0, 0)

        ' Create and activate a part document
        swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2008\templates\Part.prtdot", 0, 0, 0)
        swApp.ActivateDoc2("Part1", False, longstatus)
        swModel = swApp.ActiveDoc

        swSketchMgr = swModel.SketchManager
        swModelDocExt = swModel.Extension

        ' Sketch a circle
        swSketchSegment = swSketchMgr.CreateCircle(0.0#, 0.0#, 0.0#, 0.045549, 0.013926, 0.0#)

        ' Clear any selections and change
        ' the view orientation to Front
        swModel.ClearSelection2(True)
        swModel.ShowNamedView2("Front", 1)

        ' Create a rectangle
        vSkLines = swSketchMgr.CreateCornerRectangle(-0.005867589431389, 0.03694408160504, 0, 0.004563680668858, 0.02673012963188,
0)

        ' Create a circular sketch pattern
        ' using the rectangle
        boolstatus = swSketchMgr.CreateCircularSketchStepAndRepeat(0.03184378021964, 4.732863934409, 4, 1.570796326795, True, "",
True, True, True)
        swModel.ClearSelection2(True)

        ' Close the sketch and rebuild
        swSketchMgr.InsertSketch(True)

        ' Select an entity in the circular sketch pattern
        ' and open the circular sketch pattern to edit it
        boolstatus = swModelDocExt.SelectByID2("Line1@Sketch1", "EXTSKETCHSEGMENT", -0.002390499397973, 0.03694408160504, 0, False,
0, Nothing, 0)
        swModel.EditSketch()

        ' Delete an instance of the circular
        ' sketch pattern and close the sketch
        boolstatus = swSketchMgr.EditCircularSketchStepAndRepeat(0.03184378021964, 4.732863934409, 3, 1.570796326795, True, "",
True, True, True, "Line2_Line1_Line4_Line3_")
        swModel.ClearSelection2(True)
        swSketchMgr.InsertSketch(True)

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>

```

Public swApp As SldWorks

End Class

Create and Edit Linear Sketch Pattern Example (C#)

This example shows how to create and edit a linear sketch pattern.

```
//-----
// Preconditions: Verify that the specified part template exists.
//
// Postconditions:
// 1. Creates an extrude feature.
// 2. Opens a sketch on the front face of the extrude feature.
// 3. Creates a linear sketch pattern.
// 4. Closes the sketch.
// 5. Opens the linear sketch pattern.
// 6. Deletes an entity (i.e., Line3) in each instance of the linear
// sketch pattern.
// 7. Examine the graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchMgr = default(SketchManager);
            Feature swFeature = default(Feature);
            FeatureManager swFeatureMgr = default(FeatureManager);
            object vSkLines = null;
            bool boolstatus = false;
            int longstatus = 0;

            // Reset the counts for untitled documents for this macro
            swApp.ResetUntitledCount(0, 0, 0);

            // Create part document
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2016\\templates\\Part.prtdot", 0, 0, 0);
            swApp.ActivateDoc2("Part1", false, ref longstatus);
            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Select the Front plane
            swModelDocExt = swModel.Extension;
            boolstatus = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);

            // Open a sketch and sketch a rectangle
            swSketchMgr = swModel.SketchManager;
            swSketchMgr.InsertSketch(true);
            swModel.ClearSelection2(true);
            vSkLines = swSketchMgr.CreateCornerRectangle(-0.07606134448097, 0.04933121484909, 0, 0.07649598073515, -0.0510697598658, 0);

            // Change view orientation and clear all selections
            swModel.ShowNamedView2("Trimetric", 8);
            swModel.ClearSelection2(true);

            // Select the sketch entities to extrude
            swModelDocExt = swModel.Extension;
            boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            boolstatus = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            boolstatus = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);
            boolstatus = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, true, 0, null, 0);

            // Create the extrude feature
            swFeatureMgr = swModel.FeatureManager;
            swFeature = swFeatureMgr.FeatureExtrusion2(true, false, false, 0, 0, 0.0508, 0.381, false, false, false,
            false, 0.01745329251994, 0.01745329251994, false, false, false, false, true, true, true, 0, 0, false);

            // Fit the model in the graphics area
            swModel.ViewZoomtofit2();

            // Select the face on the extrude feature and sketch the entities to pattern
            swModel.ShowNamedView2("Front", 1);
            boolstatus = swModelDocExt.SelectByID2("", "FACE", -0.05428715407583, 0.03314479661076, 0.050799999999998, false, 0, null, 0);
            vSkLines = swSketchMgr.CreateCornerRectangle(-0.00838865116811, 0.00609746454014, 0, 0.00738895920642, -0.007221297464333, 0);

            // Create a linear sketch pattern using the newly
            // sketched rectangle as the linear pattern seed feature
            boolstatus = swSketchMgr.CreateLinearSketchStepAndRepeat(2, 2, 0.0254, 0.0254, 0.296705972839, 1.134464013796, "", true, true, false, true, true, true);

            swModel.ClearSelection2(true);

            // Close the sketch
            swSketchMgr.InsertSketch(true);

            // Select an entity in the linear sketch seed
            // pattern and open the linear sketch pattern to edit
            boolstatus = swModelDocExt.SelectByID2("Line3@Sketch2", "EXTSKETCHSEGMENT", -0.002651338304644, -0.007221297464333, 0, false, 0, null, 0);
            swModel.EditSketch();

            // Delete the Line3 sketch entity from each instance
            // of the linear sketch pattern
            boolstatus = swSketchMgr.EditLinearSketchStepAndRepeat(3, 2, 0.0254, 0.0254, 0.296705972839, 1.134464013796, "", false, false, false, true, true, true);

            swModel.EditSketch();
        }
    }
}

public SldWorks swApp;
```

Create and Edit Linear Sketch Pattern Example (VBA)

This example shows how to create and edit a linear sketch pattern.

```

'-----
' Preconditions: Verify that the specified part template exists.
'
' Postconditions:
' 1. Creates an extrude feature.
' 2. Opens a sketch on the front face of the extrude feature.
' 3. Creates a linear sketch pattern.
' 4. Closes the sketch.
' 5. Opens the linear sketch pattern.
' 6. Deletes an entity (i.e., Line3) in each instance of the linear
'    sketch pattern.
' 7. Examine the graphics area.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchMgr As SldWorks.SketchManager
Dim swFeature As SldWorks.Feature
Dim swFeatureMgr As SldWorks.FeatureManager
Dim vSkLines As Variant
Dim boolstatus As Boolean
Dim longstatus As Long

Sub main()

    Set swApp = Application.SldWorks

    ' Reset the counts for untitled documents for this macro
    swApp.ResetUntitledCount 0, 0, 0

    ' Create part document
    Set swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
    swApp.ActivateDoc2 "Part1", False, longstatus
    Set swModel = swApp.ActiveDoc

    ' Select the Front plane
    Set swModelDocExt = swModel.Extension
    boolstatus = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)

    ' Open a sketch and sketch a rectangle
    Set swSketchMgr = swModel.SketchManager
    swSketchMgr.InsertSketch True
    swModel.ClearSelection2 True
    vSkLines = swSketchMgr.CreateCornerRectangle(-0.07606134448097, 0.04933121484909, 0, 0.07649598073515, -0.0510697598658, 0)

    ' Change view orientation and clear all selections
    swModel.ShowNamedView2 "Trimetric", 8
    swModel.ClearSelection2 True

    ' Select the sketch entities to extrude
    Set swModelDocExt = swModel.Extension
    boolstatus = swModelDocExt.SelectByID("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    boolstatus = swModelDocExt.SelectByID("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    boolstatus = swModelDocExt.SelectByID("Line4", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
    boolstatus = swModelDocExt.SelectByID("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)

    ' Create the extrude feature
    Set swFeatureMgr = swModel.FeatureManager
    Set swFeature = swFeatureMgr.FeatureExtrusion2(True, False, False, 0, 0, 0.0508, 0.381, False, False, False, False, 0.01745329251994, 0.01745329251994, False, False, False, True, True, True, 0, 0, False)

    ' Fit the model in the graphics area
    swModel.ViewZoomtofit2

    ' Select the face on the extrude feature
    ' and sketch the entities to pattern
    swModel.ShowNamedView2 "Front", 1
    boolstatus = swModelDocExt.SelectByID("", "FACE", -0.05428715407583, 0.03314479661076, 0.05079999999998, False, 0, Nothing, 0)
    vSkLines = swSketchMgr.CreateCornerRectangle(-0.00838865116811, 0.00609746454014, 0, 0.00738895920642, -0.007221297464333, 0)

    ' Create a linear sketch pattern using the newly
    ' sketched rectangle as the linear pattern
    ' seed feature
    boolstatus = swSketchMgr.CreateLinearSketchStepAndRepeat(2, 2, 0.0254, 0.0254, 0.296705972839, 1.134464013796, "", True, True, False, True, True)
    swModel.ClearSelection2 True

    ' Close the sketch
    swSketchMgr.InsertSketch True

    ' Select an entity in the linear sketch seed
    ' pattern and open the linear sketch pattern to edit
    boolstatus = swModelDocExt.SelectByID("Line3@Sketch2", "EXTSKETCHSEGMENT", -0.002651338304644, -0.007221297464333, 0, False, 0, Nothing, 0)
    swModel.EditSketch

    ' Delete the Line3 sketch entity from each instance
    ' of the linear sketch pattern
    boolstatus = swSketchMgr.EditLinearSketchStepAndRepeat(3, 2, 0.0254, 0.0254, 0.296705972839, 1.134464013796, "", False, False, False, True, True, "Line2_Line1_Line4_")

End Sub

```

Create and Edit Linear Sketch Pattern Example (VB.NET)

This example shows how to create and edit a linear sketch pattern.

```

-----
' Preconditions: Verify that the specified part template exists.
-----
' Postconditions:
' 1. Creates an extrude feature.
' 2. Opens a sketch on the front face of the extrude feature.
' 3. Creates a linear sketch pattern.
' 4. Closes the sketch.
' 5. Opens the linear sketch pattern.
' 6. Deletes an entity (i.e., Line3) in each instance of the linear
'    sketch pattern.
' 7. Examine the graphics area.
-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchMgr As SketchManager
        Dim swFeature As Feature
        Dim swFeatureMgr As FeatureManager
        Dim vSkLines As Object
        Dim boolstatus As Boolean
        Dim longstatus As Integer

        ' Reset the counts for untitled documents for this macro
        swApp.ResetUntitledCount(0, 0, 0)

        ' Create part document
        swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
        swApp.ActivateDoc2("Part1", False, longstatus)
        swModel = swApp.ActiveDoc

        ' Select the Front plane
        swModelDocExt = swModel.Extension
        boolstatus = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)

        ' Open a sketch and sketch a rectangle
        swSketchMgr = swModel.SketchManager
        swSketchMgr.InsertSketch(True)
        swModel.ClearSelection2(True)
        vSkLines = swSketchMgr.CreateCornerRectangle(-0.07606134448097, 0.04933121484909, 0, 0.07649598073515, -0.0510697598658, 0)

        ' Change view orientation and clear all selections
        swModel.ShowNamedView2("Trimetric", 0)
        swModel.ClearSelection2(True)

        ' Select the sketch entities to extrude
        swModelDocExt = swModel.Extension
        boolstatus = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
        boolstatus = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        boolstatus = swModelDocExt.SelectByID2("Line4", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)
        boolstatus = swModelDocExt.SelectByID2("Line3", "SKETCHSEGMENT", 0, 0, 0, True, 0, Nothing, 0)

        ' Create the extrude feature
        swFeatureMgr = swModel.FeatureManager
        swFeature = swFeatureMgr.FeatureExtrusion2(True, False, False, 0, 0, 0.0508, 0.381, False, False, False, False, 0.01745329251994, 0.01745329251994, False, False, False, False, True, True, True, 0, 0, False)

        ' Fit the model in the graphics area
        swModel.ViewZoomtofit2()

        ' Select the face on the extrude feature and sketch the entities to pattern
        swModel.ShowNamedView2("Front", 1)
        boolstatus = swModelDocExt.SelectByID2("", "FACE", -0.05428715407583, 0.03314479661076, 0.050799999999998, False, 0, Nothing, 0)
        vSkLines = swSketchMgr.CreateCornerRectangle(-0.00838865116811, 0.00609746454014, 0, 0.00738895920642, -0.007221297464333, 0)

        ' Create a linear sketch pattern using the newly
        ' sketched rectangle as the linear pattern seed feature
        boolstatus = swSketchMgr.CreateLinearSketchStepAndRepeat(2, 2, 0.0254, 0.0254, 0.296705972839, 1.134464013796, "", True, True, False, True, True)
        swModel.ClearSelection2(True)

        ' Close the sketch
        swSketchMgr.InsertSketch(True)

        ' Select an entity in the linear sketch seed
        ' pattern and open the linear sketch pattern to edit
        boolstatus = swModelDocExt.SelectByID2("Line3@Sketch2", "EXTSKETCHSEGMENT", -0.002651338304644, -0.007221297464333, 0, False, 0, Nothing, 0)
        swModel.EditSketch()

        ' Delete the Line3 sketch entity from each instance
        ' of the linear sketch pattern
        boolstatus = swSketchMgr.EditLinearSketchStepAndRepeat(3, 2, 0.0254, 0.0254, 0.296705972839, 1.134464013796, "", False, False, False, True, True, "Line2_Line1_Line4_")

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Create and Flip Flat-Pattern View of Sheet Metal Part Example (VB)

This example shows how to create and flip a flat-pattern view of a sheet metal part.

```
'-----
'
' Problem:
'   The drawing view must be of the part in a flattened
'   state and without bend lines. This is necessary so the
'   file is suitable for export to DXF for subsequent import
'   into laser cutting software, which normally only requires
'   the outline of the profile to be cut.
'
' Preconditions:
'   1) Sheet metal part is open.
'   2) Adjust paper template size, height, and width.
'
' Postconditions:
'   A new A1 sized drawing is generated with
'   a flattened view of the sheet metal part
'   with no bend lines showing.
'-----
```

Option Explicit

```
Public Enum swDwgPaperSizes_e
    swDwgPaperASize = 0
    swDwgPaperASizeVertical = 1
    swDwgPaperBSize = 2
    swDwgPaperCSize = 3
    swDwgPaperDSize = 4
    swDwgPaperESize = 5
    swDwgPaperA4size = 6
    swDwgPaperA4sizeVertical = 7
    swDwgPaperA3size = 8
    swDwgPaperA2size = 9
    swDwgPaperA1size = 10
    swDwgPaperA0size = 11
    swDwgPapersUserDefined = 12
End Enum
```

```
Public Enum swDwgTemplates_e
    swDwgTemplateASize = 0
    swDwgTemplateASizeVertical = 1
    swDwgTemplateBSize = 2
    swDwgTemplateCSize = 3
    swDwgTemplateDSize = 4
    swDwgTemplateESize = 5
    swDwgTemplateA4size = 6
    swDwgTemplateA4sizeVertical = 7
    swDwgTemplateA3size = 8
    swDwgTemplateA2size = 9
    swDwgTemplateA1size = 10
    swDwgTemplateA0size = 11
    swDwgTemplateCustom = 12
    swDwgTemplateNone = 13
End Enum
```

```
' Paper size in millimeters
' A   216 x 279
' B   279 x 432
' C   432 x 559
' D   559 x 864
' E   864 x 1118
'
' A0   841 x 1189
' A1   594 x 841
```

```
' A2  420 x 594
' A3  297 x 420
' A4  210 x 297
```

```
Const TemplateSize As Long = swDwgTemplateA1size
Const PaperSize As Long = swDwgPaperA1size
Const PaperWidth As Double = 0.841 ' Meters
Const PaperHeight As Double = 0.594 ' Meters
```

```
Sub main()
```

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDraw As SldWorks.DrawingDoc
Dim swView As SldWorks.View
Dim bRet As Boolean
```

```
Set swApp = CreateObject("SldWorks.Application")
Set swModel = swApp.ActiveDoc
Set swDraw = swApp.NewDrawing2(TemplateSize, "", PaperSize, _
    PaperWidth, PaperHeight)
```

```
Set swView = swDraw.CreateFlatPatternViewFromModelView3( _
    swModel.GetPathName, "", _
    PaperWidth / 2, PaperHeight / 2, 0#, True, True)
```

```
Debug.Print swView.GetName2
Debug.Print swView.FlipView
```

```
swView.FlipView = False
Debug.Print swView.FlipView
```

```
End Sub
```

Create and Modify Cavity Feature Example (C#)

This example shows how to create and modify a cavity feature.

```
//-----
// Preconditions:
// 1. Create a new folder and copy public_documents\samples\tutorial\molds\telephone.sldprt to that
// folder.
// 2. Click File > New > Part and OK to create a new part document.
// a. Insert a sketch of a rectangle with a length of 240 mm and width of 350 mm.
// b. Using the sketch, create a boss extrude feature with a depth of 160 mm.
// c. Save the part as telephoneMoldBase.sldprt in the folder created in step 1.
// 3. Click File > New > Assembly and OK to create a new assembly document.
// a. Click telephoneMoldBase in Part/Assembly to Insert in the PropertyManager page.
// b. Click OK.
// c. Click Insert Components on the Assembly toolbar, click Browse in
// Part/Assembly to Insert, click telephone.sldprt located in the folder created in step 1,
// and click Open.
// d. Drop telephone.sldprt in the graphics area.
// e. Click View > Display > Hidden Lines Visible.
// f. In the FeatureManager design tree, click telephone<1> and click Move Component
// in the Assembly toolbar.
// g. Move telephone<1> into the center of telephoneMoldBase<1> and click OK.
// h. Click File > Save All > Rebuild and save the document (recommended), navigate to the
// folder created in step 1, type AssemInterim.sldasm in File name, and click Save.
// i. In the FeatureManager design tree, click telephoneMoldBase<1> and click Edit Component
// in the Assembly toolbar.
// j. Click Cavity on the Mold Tools toolbar.
// k. Click telephone<1> in the FeatureManager design tree for the design component.
// l. Clear Scale uniformly and set X, Y, and Z to 0, if necessary.
// m. Click OK to create the cavity feature and close the PropertyManager page.
// 4. Click File > Save All.
// 5. Click File > Close two times.
// 6. Click File > AssemInterim.sldasm.
// 7. Open the Immediate window.
// 8. Run the macro.
//
// Postconditions:
// 1. Accesses Cavity<1> and gets and sets scale-related properties.
// 2. When the macro finishes executing, click File > Close and Save All.
// 3. Open AssemInterim.sldasm, click Yes to roll forward, and click Rebuild to rebuild.
// 4. Expand telephoneMoldBase<1> in the FeatureManager design tree.
// 5. Right-click the Cavity1 feature, click Edit Feature, click OK if a message box displays, and
// compare the values in the PropertyManager page with the new values shown in the
// Immediate window.
// 6. Click OK.
// 7. Close File > Close > Save All.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System.Diagnostics;

namespace CavityCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Feature swFeature = default(Feature);
            CavityFeatureData swCavityFeatureData = default(CavityFeatureData);
            bool status = false;
            double xScale = 0;
            double yScale = 0;
            double zScale = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Select the Cavity1 feature and get and set its properties
            status = swModelDocExt.SelectByID2("Cavity1@telephoneMoldBase-1@AssemInterim", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            swCavityFeatureData = (CavityFeatureData)swFeature.GetDefinition();
            status = swCavityFeatureData.AccessSelections(swModel, null);
            Debug.Print("Scale type: " + swCavityFeatureData.ScaleType);
            Debug.Print("Scale uniformly value: " + swCavityFeatureData.UseUniformScale);
            swCavityFeatureData.GetScale(out xScale, out yScale, out zScale);
            Debug.Print("Scale values:");
            Debug.Print(" X: " + xScale);
            Debug.Print(" Y: " + yScale);
            Debug.Print(" Z: " + zScale);
            Debug.Print("Value by which to scale this cavity feature in all directions: " + swCavityFeatureData.UniformScale);
            Debug.Print("");
            swCavityFeatureData.UseUniformScale = true;
            Debug.Print("New scale uniformly value: " + swCavityFeatureData.UseUniformScale);
            swCavityFeatureData.UniformScale = 2;
            Debug.Print("New value by which to scale this cavity feature in all directions: " + swCavityFeatureData.UniformScale);
            status = swFeature.ModifyDefinition(swCavityFeatureData, swModel, null);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create and Modify Cavity Feature Example (VBA)

This example shows how to create and modify a cavity feature.

```

'-----
' Preconditions:
' 1. Create a new folder and copy public_documents\samples\tutorial\molds\telephone.sldprt to that
'    folder.
' 2. Click File > New > Part and OK to create a new part document.
'    a. Insert a sketch of a rectangle with a length of 240 mm and width of 350 mm.
'    b. Using the sketch, create a boss extrude feature with a depth of 160 mm.
'    c. Save the part as telephoneMoldBase.sldprt in the folder created in step 1.
' 3. Click File > New > Assembly and OK to create a new assembly document.
'    a. Click telephoneMoldBase in Part/Assembly to Insert in the PropertyManager page.
'    b. Click OK.
'    c. Click Insert Components on the Assembly toolbar, click Browse in
'       Part/Assembly to Insert, click telephone.sldprt located in the folder created in step 1,
'       and click Open.
'    d. Drop telephone.sldprt in the graphics area.
'    e. Click View > Display > Hidden Lines Visible.
'    f. In the FeatureManager design tree, click telephone<1> and click Move Component
'       in the Assembly toolbar.
'    g. Move telephone<1> into the center of telephoneMoldBase<1> and click OK.
'    h. Click File > Save All > Rebuild and save the document (recommended), navigate to the
'       folder created in step 1, type AssemInterim.sldasm in File name, and click Save.
'    i. In the FeatureManager design tree, click telephoneMoldBase<1> and click Edit Component
'       in the Assembly toolbar.
'    j. Click Cavity on the Mold Tools toolbar.
'    k. Click telephone<1> in the FeatureManager design tree for the design component.
'    l. Clear Scale uniformly and set X, Y, and Z to 0, if necessary.
'    m. Click OK to create the cavity feature and close the PropertyManager page.
' 4. Click File > Save All.
' 5. Click File > Close two times.
' 6. Click File > AssemInterim.sldasm.
' 7. Open the Immediate window.
' 8. Run the macro.
'
' Postconditions:
' 1. Accesses Cavity<1> and gets and sets scale-related properties.
' 2. When the macro finishes executing, click File > Close and Save All.
' 3. Open AssemInterim.sldasm, click Yes to roll forward, and click Rebuild to rebuild.
' 4. Expand telephoneMoldBase<1> in the FeatureManager design tree.
' 5. Right-click the Cavity1 feature, click Edit Feature, click OK if a message box displays, and
'    compare the values in the PropertyManager page with the new values shown in the
'    Immediate window.
' 6. Click OK.
' 7. Close File > Close > Save All.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim swCavityFeatureData As SldWorks.CavityFeatureData
Dim status As Boolean
Dim xScale As Double
Dim yScale As Double
Dim zScale As Double

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    'Select the Cavity1 feature and get and set its properties
    status = swModelDocExt.SelectByID2("Cavity1@telephoneMoldBase-1@AssemInterim", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swCavityFeatureData = swFeature.GetDefinition
    status = swCavityFeatureData.AccessSelections(swModel, Nothing)
    Debug.Print ("Scale type: " & swCavityFeatureData.ScaleType)
    Debug.Print ("Scale uniformly value: " & swCavityFeatureData.UseUniformScale)
    swCavityFeatureData.GetScale xScale, yScale, zScale
    Debug.Print ("Scale values:")
    Debug.Print (" X: " & xScale)
    Debug.Print (" Y: " & yScale)
    Debug.Print (" Z: " & zScale)
    Debug.Print ("Value by which to scale this cavity feature in all directions: " & swCavityFeatureData.UniformScale)
    Debug.Print ("")
    swCavityFeatureData.UseUniformScale = True
    Debug.Print ("New scale uniformly value: " & swCavityFeatureData.UseUniformScale)
    swCavityFeatureData.UniformScale = 2
    Debug.Print ("New value by which to scale this cavity feature in all directions: " & swCavityFeatureData.UniformScale)
    status = swFeature.ModifyDefinition(swCavityFeatureData, swModel, Nothing)

End Sub

```

Create and Modify Cavity Feature Example (VB.NET)

This example shows how to create and modify a cavity feature.

```

'-----
' Preconditions:
' 1. Create a new folder and copy public_documents\samples\tutorial\molds\telephone.sldprt to that
'    folder.
' 2. Click File > New > Part and OK to create a new part document.
'    a. Insert a sketch of a rectangle with a length of 240 mm and width of 350 mm.
'    b. Using the sketch, create a boss extrude feature with a depth of 160 mm.
'    c. Save the part as telephoneMoldBase.sldprt in the folder created in step 1.
' 3. Click File > New > Assembly and OK to create a new assembly document.
'    a. Click telephoneMoldBase in Part/Assembly to Insert in the PropertyManager page.
'    b. Click OK.
'    c. Click Insert Components on the Assembly toolbar, click Browse in
'       Part/Assembly to Insert, click telephone.sldprt located in the folder created in step 1,
'       and click Open.
'    d. Drop telephone.sldprt in the graphics area.
'    e. Click View > Display > Hidden Lines Visible.
'    f. In the FeatureManager design tree, click telephone<1> and click Move Component
'       in the Assembly toolbar.
'    g. Move telephone<1> into the center of telephoneMoldBase<1> and click OK.
'    h. Click File > Save All > Rebuild and save the document (recommended), navigate to the
'       folder created in step 1, type AssemInterim.sldasm in File name, and click Save.
'    i. In the FeatureManager design tree, click telephoneMoldBase<1> and click Edit Component
'       in the Assembly toolbar.
'    j. Click Cavity on the Mold Tools toolbar.
'    k. Click telephone<1> in the FeatureManager design tree for the design component.
'    l. Clear Scale uniformly and set X, Y, and Z to 0, if necessary.
'    m. Click OK to create the cavity feature and close the PropertyManager page.
' 4. Click File > Save All.
' 5. Click File > Close two times.
' 6. Click File > AssemInterim.sldasm.
' 7. Open the Immediate window.
' 8. Run the macro.
'
' Postconditions:
' 1. Accesses Cavity<1> and gets and sets scale-related properties.
' 2. When the macro finishes executing, click File > Close and Save All.
' 3. Open AssemInterim.sldasm, click Yes to roll forward, and click Rebuild to rebuild.
' 4. Expand telephoneMoldBase<1> in the FeatureManager design tree.
' 5. Right-click the Cavity1 feature, click Edit Feature, click OK if a message box displays, and
'    compare the values in the PropertyManager page with the new values shown in the
'    Immediate window.
' 6. Click OK.
' 7. Close File > Close > Save All.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelectionMgr As SelectionMgr
        Dim swFeature As Feature
        Dim swCavityFeatureData As CavityFeatureData
        Dim status As Boolean
        Dim xScale As Double
        Dim yScale As Double
        Dim zScale As Double

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        'Select the Cavity1 feature and get and set its properties
        status = swModelDocExt.SelectByID2("Cavity1@telephoneMoldBase-1@AssemInterim", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swCavityFeatureData = swFeature.GetDefinition
        status = swCavityFeatureData.AccessSelections(swModel, Nothing)
        Debug.Print("Scale type: " & swCavityFeatureData.ScaleType)
        Debug.Print("Scale uniformly value: " & swCavityFeatureData.UseUniformScale)
        swCavityFeatureData.GetScale(xScale, yScale, zScale)
        Debug.Print("Scale values:")
        Debug.Print(" X: " & xScale)
        Debug.Print(" Y: " & yScale)
        Debug.Print(" Z: " & zScale)
        Debug.Print("Value by which to scale this cavity feature in all directions: " & swCavityFeatureData.UniformScale)
        Debug.Print("")
        swCavityFeatureData.UseUniformScale = True
        Debug.Print("New scale uniformly value: " & swCavityFeatureData.UseUniformScale)
        swCavityFeatureData.UniformScale = 2
    
```

```
Debug.Print("New value by which to scale this cavity feature in all directions: " & swCavityFeatureData.UniformScale)  
status = swFeature.ModifyDefinition(swCavityFeatureData, swModel, Nothing)
```

```
End Sub
```

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

```
End Class
```

Create and Modify Closed Corner Feature Example (C#)

This example shows how to create and modify a closed corner feature in a sheet metal part.

```
//-----
// Preconditions:
// 1. Verify that the specified sheet metal part document to
//    open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified sheet metal part document.
// 2. Selects two faces.
// 3. Inserts a closed corner feature.
// 4. Modifies properties of the closed corner feature.
// 5. Examine the graphics area and Immediate window.
//
// NOTE: Because this part document is used elsewhere,
// do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace ClosedCornerFeatureDataCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Feature swFeature = default(Feature);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            ClosedCornerFeatureData swClosedCornerFeatureData = default(ClosedCornerFeatureData);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;
            object[] faces = null;
            Face2 swFace = default(Face2);
            int i = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\sheetmetal\\formtools\\cover.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;

            //Select the faces for the closed corner feature
            status = swModelDocExt.SelectByID2("", "FACE", 0.0110595835492404, 0.0280018934407167, 0.0497348782814129, true, 1, null, 0);
            status = swModelDocExt.SelectByID2("", "FACE", 0.0181424245698736, 0.0110595835492404, 0.0495671179450028, true, 1073741824, null, 0);

            //Insert the closed corner feature
            swModel.InsertSheetMetalClosedCorner();

            //Select the closed corner feature
            status = swModelDocExt.SelectByID2("Closed Corner1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            swClosedCornerFeatureData = (ClosedCornerFeatureData)swFeature.GetDefinition();

            //Access closed corner feature
            status = swClosedCornerFeatureData.AccessSelections(swModel, null);
            Debug.Print("Original properties: ");
            Debug.Print("  Corner type: " + swClosedCornerFeatureData.CornerType);
            Debug.Print("  Gap distance: " + swClosedCornerFeatureData.GapDistance);
            Debug.Print("  Open bend region? " + swClosedCornerFeatureData.OpenBendRegion);
            Debug.Print("  Overlap/underlap ratio: " + swClosedCornerFeatureData.OverlapUnderlapRatio);
            faces = (object[])swClosedCornerFeatureData.Faces;
            for (i = 0; i < faces.Length; i++)
            {
                swFace = (Face2)faces[i];
                Debug.Print("  Area of face " + i + ": " + swFace.GetArea() * 1000 + " mm");
            }
            Debug.Print("Modified properties: ");
            swClosedCornerFeatureData.CornerType = (int)swClosedCornerTypes_e.swClosedCornerTypeUnderlap;
            swClosedCornerFeatureData.GapDistance = 0.005;
            swClosedCornerFeatureData.OpenBendRegion = true;
            swClosedCornerFeatureData.OverlapUnderlapRatio = 0.5;
            Debug.Print("  Corner type: " + swClosedCornerFeatureData.CornerType);
            Debug.Print("  Gap distance: " + swClosedCornerFeatureData.GapDistance);
            Debug.Print("  Open bend region? " + swClosedCornerFeatureData.OpenBendRegion);
            Debug.Print("  Overlap/underlap ratio: " + swClosedCornerFeatureData.OverlapUnderlapRatio);

            status = swFeature.ModifyDefinition(swClosedCornerFeatureData, swModel, null);
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}
```

Create and Modify Closed Corner Feature Example (VBA)

This example shows how to create and modify a closed corner feature in a sheet metal part.

```
'-----
' Preconditions:
' 1. Verify that the specified sheet metal part document to
'    open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified sheet metal part document.
' 2. Selects two faces.
' 3. Inserts a closed corner feature.
' 4. Modifies properties of the closed corner feature.
' 5. Examine the graphics area and Immediate window.
'
' NOTE: Because this part document is used elsewhere,
' do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeature As SldWorks.Feature
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swClosedCornerFeatureData As SldWorks.ClosedCornerFeatureData
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String
Dim faces As Variant
Dim swFace As SldWorks.Face2
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\sheetmetal\formtools\cover.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    Set swSelectionMgr = swModel.SelectionManager

    'Select the faces for the closed corner feature
    status = swModelDocExt.SelectByID("", "FACE", 1.10595835492404E-02, 2.80018934407167E-02, 4.97348782814129E-02, True, 1, Nothing, 0)
    status = swModelDocExt.SelectByID("", "FACE", 1.81424245698736E-02, 1.10595835492404E-02, 4.95671179450028E-02, True, 1073741824, Nothing, 0)

    'Insert the closed corner feature
    swModel.InsertSheetMetalClosedCorner

    'Select the closed corner feature
    status = swModelDocExt.SelectByID("Closed Corner1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swClosedCornerFeatureData = swFeature.GetDefinition

    'Access closed corner feature
    status = swClosedCornerFeatureData.AccessSelections(swModel, Nothing)
    Debug.Print ("Original properties: ")
    Debug.Print (" Corner type: " & swClosedCornerFeatureData.CornerType)
    Debug.Print (" Gap distance: " & swClosedCornerFeatureData.GapDistance)
    Debug.Print (" Open bend region? " & swClosedCornerFeatureData.OpenBendRegion)
    Debug.Print (" Overlap/underlap ratio: " & swClosedCornerFeatureData.OverlapUnderlapRatio)
    faces = swClosedCornerFeatureData.faces
    For i = 0 To UBound(faces)
        Set swFace = faces(i)
        Debug.Print (" Area of face " & i & ": " & swFace.GetArea * 1000 & " mm")
    Next i

    Debug.Print ("Modified properties: ")
    swClosedCornerFeatureData.CornerType = swClosedCornerTypes_e.swClosedCornerTypeUnderlap
    swClosedCornerFeatureData.GapDistance = 0.005
    swClosedCornerFeatureData.OpenBendRegion = True
    swClosedCornerFeatureData.OverlapUnderlapRatio = 0.5
    Debug.Print (" Corner type: " & swClosedCornerFeatureData.CornerType)
    Debug.Print (" Gap distance: " & swClosedCornerFeatureData.GapDistance)
    Debug.Print (" Open bend region? " & swClosedCornerFeatureData.OpenBendRegion)
    Debug.Print (" Overlap/underlap ratio: " & swClosedCornerFeatureData.OverlapUnderlapRatio)

    status = swFeature.ModifyDefinition(swClosedCornerFeatureData, swModel, Nothing)

End Sub
```

Create and Modify Closed Corner Feature Example (VB.NET)

This example shows how to create and modify a closed corner feature in a sheet metal part.

```

'-----
' Preconditions:
' 1. Verify that the specified sheet metal part document to
'    open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified sheet metal part document.
' 2. Selects two faces.
' 3. Inserts a closed corner feature.
' 4. Modifies properties of the closed corner feature.
' 5. Examine the graphics area and Immediate window.
'
' NOTE: Because this part document is used elsewhere,
' do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeature As Feature
        Dim swSelectionMgr As SelectionMgr
        Dim swClosedCornerFeatureData As ClosedCornerFeatureData
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String
        Dim faces() As Object
        Dim swFace As Face2
        Dim i As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\sheetmetal\formtools\cover.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warning)
        swModelDocExt = swModel.Extension
        swSelectionMgr = swModel.SelectionManager

        'Select the faces for the closed corner feature
        status = swModelDocExt.SelectByID2("", "FACE", 0.0110595835492404, 0.0280018934407167, 0.0497348782814129, True, 1, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "FACE", 0.0181424245698736, 0.0110595835492404, 0.0495671179450028, True, 1073741824, No

        'Insert the closed corner feature
        swModel.InsertSheetMetalClosedCorner()

        'Select the closed corner feature
        status = swModelDocExt.SelectByID2("Closed Corner1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swClosedCornerFeatureData = swFeature.GetDefinition

        'Access closed corner feature
        status = swClosedCornerFeatureData.AccessSelections(swModel, Nothing)
        Debug.Print("Original properties: ")
        Debug.Print("  Corner type: " & swClosedCornerFeatureData.CornerType)
        Debug.Print("  Gap distance: " & swClosedCornerFeatureData.GapDistance)
        Debug.Print("  Open bend region? " & swClosedCornerFeatureData.OpenBendRegion)
        Debug.Print("  Overlap/underlap ratio: " & swClosedCornerFeatureData.OverlapUnderlapRatio)
        faces = swClosedCornerFeatureData.Faces
        For i = 0 To UBound(faces)
            swFace = faces(i)
            Debug.Print("  Area of face " & i & ": " & swFace.GetArea * 1000 & " mm")
        Next i
        Debug.Print("Modified properties: ")
        swClosedCornerFeatureData.CornerType = swClosedCornerTypes_e.swClosedCornerTypeUnderlap
        swClosedCornerFeatureData.GapDistance = 0.005
        swClosedCornerFeatureData.OpenBendRegion = True
        swClosedCornerFeatureData.OverlapUnderlapRatio = 0.5
        Debug.Print("  Corner type: " & swClosedCornerFeatureData.CornerType)
        Debug.Print("  Gap distance: " & swClosedCornerFeatureData.GapDistance)
        Debug.Print("  Open bend region? " & swClosedCornerFeatureData.OpenBendRegion)
        Debug.Print("  Overlap/underlap ratio: " & swClosedCornerFeatureData.OverlapUnderlapRatio)

        status = swFeature.ModifyDefinition(swClosedCornerFeatureData, swModel, Nothing)

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.

```

```
''' </summary>  
Public swApp As SldWorks
```

```
End Class
```

Create and Modify Distance Chain Pattern Feature Example (C#)

This example shows how to create and modify a distance chain pattern feature.

```
//-----
// Preconditions:
// 1. Verify that the assembly to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified assembly.
// 2. Selects entities to create a distance chain pattern feature.
// 3. Creates the distance chain pattern feature.
// 4. After examining the graphics area and FeatureManager design
//    tree, press F5.
// 5. Modifies the distance chain pattern feature.
// 6. Examine the graphics area to verify step 5.
// 7. Examine the Immediate window.
//
// NOTE: Because the assembly is used elsewhere, do not save
// changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureManager = default(FeatureManager);
            Feature swFeature = default(Feature);
            ChainPatternFeatureData swChainPatternFeatureData = default(ChainPatternFeatureData);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2019\\samples\\tutorial\\api\\distance linkage.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref e
rrors, ref warnings);

            //Select entities to create distance chain pattern feature
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Belt Drive Path", "SKETCH", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Mount Base-1@distance linkage", "COMPONENT", 0, 0, 0, true, 0, null, 0);
            swModel.ViewZoomTo2(1.03935160466665, -0.0995602294424154, 0.846177052265487, 1.16810340243326, -0.261325308687639, 0.846177052265488);
            status = swModelDocExt.SelectByID2("", "FACE", 0.187298652259415, -0.193039676915248, 0.113419833821467, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Right Plane@Mount Base-1@distance linkage", "PLANE", 0, 0, 0, true, 0, null, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Belt Drive Path", "SKETCH", 0, 0, 0, false, 2, null, 0);
            status = swModelDocExt.SelectByID2("Mount Base-1@distance linkage", "COMPONENT", 0, 0, 0, true, 1, null, 0);
            status = swModelDocExt.SelectByID2("", "FACE", 0.187298652259415, -0.193039676915248, 0.113419833821467, true, 256, null, 0);
            status = swModelDocExt.SelectByID2("Right Plane@Mount Base-1@distance linkage", "PLANE", 0, 0, 0, true, 16384, null, 0);

            //Create distance chain pattern feature
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swChainPatternFeatureData = (ChainPatternFeatureData)swFeatureManager.CreateDefinition((int)swFeatureNameID_e.swFmLocalChainPattern);
            swFeature = (Feature)swFeatureManager.CreateFeature(swChainPatternFeatureData);
            swModel.ClearSelection2(true);
            swModel.ViewZoomtofit2();

            System.Diagnostics.Debugger.Break();
            //Examine the graphics area and FeatureManager design tree
            //Press F5

            //Modify distance chain pattern feature
            swChainPatternFeatureData = (ChainPatternFeatureData)swFeature.GetDefinition();
            status = swChainPatternFeatureData.AccessSelections(swModel, null);
            swChainPatternFeatureData.Spacing = 0.078;
            swChainPatternFeatureData.FillPath = true;
            Debug.Print("Number of instances calculated by SOLIDWORKS to fill the path: " + swChainPatternFeatureData.InstanceCount);
            swFeature.ModifyDefinition(swChainPatternFeatureData, swModel, null);

            //Examine the graphics area
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Create and Modify Distance Chain Pattern Feature Example (VBA)

This example shows how to create and modify a distance chain pattern feature.

```

'-----
' Preconditions:
' 1. Verify that the assembly to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly.
' 2. Selects entities to create a distance chain pattern feature.
' 3. Creates the distance chain pattern feature.
' 4. After examining the graphics area and FeatureManager design
'    tree, press F5.
' 5. Modifies the distance chain pattern feature.
' 6. Examine the graphics area to verify step 5.
' 7. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save
' changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureManager As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim swChainPatternFeatureData As SldWorks.ChainPatternFeatureData
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\distance linkage.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Select entities to create distance chain pattern feature
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("Belt Drive Path", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Mount Base-1@distance linkage", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
    swModel.ViewZoomTo2 1.03935160466665, -9.95602294424154E-02, 0.846177052265487, 1.16810340243326, -0.261325308687639, 0.846177052265488
    status = swModelDocExt.SelectByID2("", "FACE", 0.187298652259415, -0.193039676915248, 0.113419833821467, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Right Plane@Mount Base-1@distance linkage", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Belt Drive Path", "SKETCH", 0, 0, 0, False, 2, Nothing, 0)
    status = swModelDocExt.SelectByID2("Mount Base-1@distance linkage", "COMPONENT", 0, 0, 0, True, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "FACE", 0.187298652259415, -0.193039676915248, 0.113419833821467, True, 256, Nothing, 0)
    status = swModelDocExt.SelectByID2("Right Plane@Mount Base-1@distance linkage", "PLANE", 0, 0, 0, True, 16384, Nothing, 0)

    'Create distance chain pattern feature
    Set swFeatureManager = swModel.FeatureManager

    Set swChainPatternFeatureData = swFeatureManager.CreateDefinition(swFmLocalChainPattern)
    Set swFeature = swFeatureManager.CreateFeature(swChainPatternFeatureData)

    swModel.ClearSelection2 True
    swModel.ViewZoomtofit2

    Stop
    'Examine the graphics area and FeatureManager design tree
    'Press F5

    'Modify distance chain pattern feature
    Set swChainPatternFeatureData = swFeature.GetDefinition
    status = swChainPatternFeatureData.AccessSelections(swModel, Nothing)
    swChainPatternFeatureData.Spacing = 0.078
    swChainPatternFeatureData.FillPath = True
    Debug.Print "Number of instances calculated by SOLIDWORKS to fill the path: " & swChainPatternFeatureData.InstanceCount
    swFeature.ModifyDefinition swChainPatternFeatureData, swModel, Nothing

    'Examine the graphics area

End Sub

```

Create and Modify Distance Chain Pattern Feature Example (VB.NET)

This example shows how to create and modify a distance chain pattern feature.

```

'-----
' Preconditions:
' 1. Verify that the assembly to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified assembly.
' 2. Selects entities to create a distance chain pattern feature.
' 3. Creates the distance chain pattern feature.
' 4. After examining the graphics area and FeatureManager design
'    tree, press F5.
' 5. Modifies the distance chain pattern feature.
' 6. Examine the graphics area to verify step 5.
' 7. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save
' changes.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureManager As FeatureManager
        Dim swFeature As Feature
        Dim swChainPatternFeatureData As ChainPatternFeatureData
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2019\samples\tutorial\api\distance linkage.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors,

        'Select entities to create distance chain pattern feature
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("Belt Drive Path", "SKETCH", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Mount Base-1@distance linkage", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)
        swModel.ViewZoomTo2(1.03935160466665, -0.0995602294424154, 0.846177052265487, 1.16810340243326, -0.261325308687639, 0.846177
        status = swModelDocExt.SelectByID2("", "FACE", 0.187298652259415, -0.193039676915248, 0.113419833821467, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Right Plane@Mount Base-1@distance linkage", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("Belt Drive Path", "SKETCH", 0, 0, 0, False, 2, Nothing, 0)
        status = swModelDocExt.SelectByID2("Mount Base-1@distance linkage", "COMPONENT", 0, 0, 0, True, 1, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "FACE", 0.187298652259415, -0.193039676915248, 0.113419833821467, True, 256, Nothing, 0)
        status = swModelDocExt.SelectByID2("Right Plane@Mount Base-1@distance linkage", "PLANE", 0, 0, 0, True, 16384, Nothing, 0)

        'Create distance chain pattern feature
        swFeatureManager = swModel.FeatureManager
        swChainPatternFeatureData = swFeatureManager.CreateDefinition(swFeatureNameID_e.swFmLocalChainPattern)
        swFeature = swFeatureManager.CreateFeature(swChainPatternFeatureData)
        swModel.ClearSelection2(True)
        swModel.ViewZoomToFit2()

        Stop
        'Examine the graphics area and FeatureManager design tree
        'Press F5

        'Modify distance chain pattern feature
        swChainPatternFeatureData = swFeature.GetDefinition
        status = swChainPatternFeatureData.AccessSelections(swModel, Nothing)
        swChainPatternFeatureData.Spacing = 0.078
        swChainPatternFeatureData.FillPath = True
        Debug.Print("Number of instances calculated by SOLIDWORKS to fill the path: " & swChainPatternFeatureData.InstanceCount)
        swFeature.ModifyDefinition(swChainPatternFeatureData, swModel, Nothing)

        'Examine the graphics area

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Create and Modify Dome Feature Example (C#)

This example shows how to create and modify a dome feature.

```
//-----
// Preconditions:
// 1. Verify that the part document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part document.
// 2. Edits Sketch1, sketches an ellipse, and creates a boss feature.
// 3. Selects a face on the boss feature and inserts a dome feature.
// 4. Prints to the Immediate window some dome feature data.
// 5. Reverses the direction of the dome feature.
// 6. Examine the Immediate window, graphics area, and FeatureManager design tree.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace DomeFeatureData2CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchMgr = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            Feature swFeature = default(Feature);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            DomeFeatureData2 swDomeFeatureData = default(DomeFeatureData2);
            object[] faces = null;
            Face2 swFace = default(Face2);
            Body2 oneBody = default(Body2);
            string fileName = null;
            bool status = false;
            int errors = 0;
            int warnings = 0;

            //Open model document to which to add a dome feature
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\box.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);

            //Open sketch to which to add a sketch of an ellipse
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);
            swModel.EditSketch();
            swModel.ClearSelection2(true);

            //Sketch an ellipse
            swModel.ShowNamedView2("Top", 5);
            swSketchMgr = (SketchManager)swModel.SketchManager;
            swSketchSegment = (SketchSegment)swSketchMgr.CreateEllipse(-0.0761501034873036, 0.0490523248480422, 0, -0.0513492425103863, 0.0490523248480422, 0, -0.0761501034873036, 0.0545451329838695, 0);
            swModel.ClearSelection2(true);
            swSketchMgr.InsertSketch(true);
            swModel.ViewZoomToFit2();
            swModel.ShowNamedView2("Dimetric", 9);

            //Insert dome feature
            status = swModelDocExt.SelectByID2("", "FACE", -0.0930732824141103, 0.0299999999999727, -0.0482299571224303, true, 0, null, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("", "FACE", -0.0930732824141103, 0.0299999999999727, -0.0482299571224303, false, 1, null, 0);
            swModel.InsertDome(0.01, false, true);

            //Get and modify dome feature data
            status = swModelDocExt.SelectByID2("Dome1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            swDomeFeatureData = (DomeFeatureData2)swFeature.GetDefinition();
            status = swDomeFeatureData.AccessSelections(swModel, null);
            Debug.Print("Is dome feature elliptical? " + swDomeFeatureData.Elliptical);
            Debug.Print("Height of dome: " + swDomeFeatureData.Height);
            Debug.Print("Number of faces on dome feature: " + swDomeFeatureData.GetFaceCount());
            faces = (object[])swDomeFeatureData.Faces;
            foreach (object aFace in faces)
            {
                swFace = (Face2)aFace;
                oneBody = (Body2)swFace.GetBody();
                Debug.Print("Name of body for this dome feature face: " + oneBody.Name);
            }
            //Change direction of dome feature to concave
            swDomeFeatureData.ReverseDir = true;
            status = swFeature.ModifyDefinition(swDomeFeatureData, swModel, null);
        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
```

Create and Modify Dome Feature Example (VBA)

This example shows how to create and modify a dome feature.

```

'-----
' Preconditions:
' 1. Verify that the part document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Edits Sketch1, sketches an ellipse, and creates a boss feature.
' 3. Selects a face on the boss feature and
'    inserts a dome feature.
' 4. Prints to the Immediate window some
'    dome feature data.
' 5. Reverses the direction of the dome feature.
' 6. Examine the Immediate window, graphics area,
'    and FeatureManager design tree.
'
' NOTE: Because the part is used elsewhere, do not
' save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchMgr As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swFeature As SldWorks.Feature
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swDomeFeatureData As SldWorks.DomeFeatureData2
Dim faces As Variant
Dim aFace As Variant
Dim swFace As SldWorks.Face2
Dim oneBody As SldWorks.Body2
Dim fileName As String
Dim status As Boolean
Dim errors As Long
Dim warnings As Long

Sub main()

    Set swApp = Application.SldWorks

    'Open model document to which to add a dome feature
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\box.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Open sketch to which to add a sketch of an ellipse
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    swModel.EditSketch
    swModel.ClearSelection2 True

    'Sketch an ellipse
    swModel.ShowNamedView2 "Top", 5
    Set swSketchMgr = swModel.SketchManager
    Set swSketchSegment = swSketchMgr.CreateEllipse(-7.61501034873036E-02, 4.90523248480422E-02, 0, -5.13492425103863E-02, 4.90523248480422E-02, 0, -7.61501034873036E-02, 5.45451329838695E-02, 0)
    swModel.ClearSelection2 True
    swSketchMgr.InsertSketch True
    swModel.ViewZoomtofit2
    swModel.ShowNamedView2 "Dimetric", 9

    'Insert dome feature
    status = swModelDocExt.SelectByID("", "FACE", -9.30732824141103E-02, 2.9999999999727E-02, -4.82299571224303E-02, True, 0, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID("", "FACE", -9.30732824141103E-02, 2.9999999999727E-02, -4.82299571224303E-02, False, 1, Nothing, 0)
    swModel.InsertDome 0.01, False, True

    'Get and modify dome feature data
    status = swModelDocExt.SelectByID("Dome1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swDomeFeatureData = swFeature.GetDefinition
    status = swDomeFeatureData.AccessSelections(swModel, Nothing)
    Debug.Print "Is dome feature elliptical? " & swDomeFeatureData.Elliptical
    Debug.Print "Height of dome: " & swDomeFeatureData.Height
    Debug.Print "Number of faces on dome feature: " & swDomeFeatureData.GetFaceCount
    faces = swDomeFeatureData.faces
    For Each aFace In faces
        Set swFace = aFace
        Set oneBody = swFace.GetBody
        Debug.Print "Name of body for this dome feature face: " & oneBody.Name
    Next
    'Change direction of dome feature to concave
    swDomeFeatureData.ReverseDir = True

    status = swFeature.ModifyDefinition(swDomeFeatureData, swModel, Nothing)

End Sub

```

Create and Modify Dome Feature Example (VB.NET)

This example shows how to create and modify a dome feature.

```

'-----
' Preconditions:
' 1. Verify that the part document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Edits Sketch1, sketches an ellipse, and creates a boss feature.
' 3. Selects a face on the boss feature and
'    inserts a dome feature.
' 4. Prints to the Immediate window some
'    dome feature data.
' 5. Reverses the direction of the dome feature.
' 6. Examine the Immediate window, graphics area,
'    and FeatureManager design tree.
'
' NOTE: Because the part is used elsewhere, do not
' save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchMgr As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swFeature As Feature
        Dim swSelectionMgr As SelectionMgr
        Dim swDomeFeatureData As DomeFeatureData2
        Dim faces() As Object
        Dim aFace As Object
        Dim swFace As Face2
        Dim oneBody As Body2
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer

        'Open model document to which to add a dome feature
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\api\box.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        'Open sketch to which to add a sketch of an ellipse
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
        swModel.EditSketch()
        swModel.ClearSelection2(True)

        'Sketch an ellipse
        swModel.ShowNamedView2("Top", 5)
        swSketchMgr = swModel.SketchManager
        swSketchSegment = swSketchMgr.CreateEllipse(-0.0761501034873036, 0.0490523248480422, 0, -0.0513492425103863, 0.0490523248480422, 0, -0.07615010348
73036, 0.0545451329838695, 0)
        swModel.ClearSelection2(True)
        swSketchMgr.InsertSketch(True)
        swModel.ViewZoomtofit2()
        swModel.ShowNamedView2("Dimetric", 9)

        'Insert dome feature
        status = swModelDocExt.SelectByID2("", "FACE", -0.0930732824141103, 0.0299999999999727, -0.0482299571224303, True, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("", "FACE", -0.0930732824141103, 0.0299999999999727, -0.0482299571224303, False, 1, Nothing, 0)
        swModel.InsertDome(0.01, False, True)

        'Get and modify dome feature data
        status = swModelDocExt.SelectByID2("Dome1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swDomeFeatureData = swFeature.GetDefinition
        status = swDomeFeatureData.AccessSelections(swModel, Nothing)
        Debug.Print("Is dome feature elliptical? " & swDomeFeatureData.Elliptical)
        Debug.Print("Height of dome: " & swDomeFeatureData.Height)
        Debug.Print("Number of faces on dome feature: " & swDomeFeatureData.GetFaceCount)
        faces = swDomeFeatureData.Faces
        For Each aFace In faces
            swFace = aFace
            oneBody = swFace.GetBody
            Debug.Print("Name of body for this dome feature face: " & oneBody.Name)
        Next
        'Change direction of dome feature to concave
        swDomeFeatureData.ReverseDir = True
        status = swFeature.ModifyDefinition(swDomeFeatureData, swModel, Nothing)

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Create and Modify Move Face Feature Example (C#)

This example shows how to create a Move Face feature by translating a face on a part.

```
//-----
// Preconditions: Verify that the specified part exists.
//
// Postconditions:
// 1. Creates Move Face1 in the FeatureManager design tree.
// 2. Modifies the translation parameters of Move Face1.
//
// NOTE: Because the part is used elsewhere, do not save any changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace InsertMoveFace3_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        FeatureManager swFeatMgr;
        Feature swFeat;
        MoveFaceFeatureData swMoveFaceFeat;
        object transParams;
        bool boolstatus;
        double[] triadParams = new double[3];
        int fileerror;

        int filewarning;

        public void Main()
        {
            // Open the SOLIDWORKS Design document
            swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\assemblymates\\knee.sldprt", (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Sil

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;
            swFeatMgr = swModel.FeatureManager;

            // Translation parameters
            triadParams[0] = 0;
            triadParams[1] = 0.05;
            triadParams[2] = 0;

            transParams = triadParams;

            // Select face to move
            boolstatus = swModel.Extension.SelectByID("", "FACE", 0.04239074672171, 0.015874999999999, 0.3283508339712, false, 1, nu

            // Create the Move Face feature by
            // translating the selected face
            swFeat = (Feature)swFeatMgr.InsertMoveFace3((int)swMoveFaceType_e.swMoveFaceTypeTranslate, false, 0, 0, (transParams), r

            // Modify the Move Face feature
            swMoveFaceFeat = (MoveFaceFeatureData)swFeat.GetDefinition();

            // Roll back the Move Face feature
            swMoveFaceFeat.AccessSelections(swModel, null);

            triadParams[0] = 0;
            triadParams[1] = 0.1;
            triadParams[2] = 0;

            transParams = triadParams;

            swMoveFaceFeat.TriadTranslationParameters = (transParams);

            // Roll back the part with the modified Move Face feature
            swFeat.ModifyDefinition(swMoveFaceFeat, swModel, null);
        }

        public SldWorks swApp;
    }
}
```

Create and Modify Move Face Feature Example (VBA)

This example shows how to create a Move Face feature by translating a face on a part.

```
'-----
' Preconditions:
' 1. Verify that the specified part exists.
' 2. Press F8 repeatedly to step through the macro.
' 3. Observe the graphics area while stepping through the macro.
'
' Postconditions:
' 1. Creates Move Face1 in the FeatureManager design tree.
' 2. Modifies the translation parameters of Move Face1.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatMgr As SldWorks.FeatureManager
Dim swFeat As SldWorks.Feature
Dim swMoveFaceFeat As SldWorks.MoveFaceFeatureData
Dim transParams As Variant
Dim boolstatus As Boolean
Dim triadParams(0 To 2) As Double
Dim fileerror As Long
Dim filewarning As Long

Sub main()
    Set swApp = Application.SldWorks
    ' Open the SOLIDWORKS Design document
    swApp.OpenDoc6 "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\assemblymates\knee.sldprt", swDocPART,
swOpenDocOptions_Silent, "", fileerror, filewarning

    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    Set swFeatMgr = swModel.FeatureManager

    ' Translation parameters
    triadParams(0) = 0
    triadParams(1) = 0.05
    triadParams(2) = 0

    transParams = triadParams

    ' Select face to move
    boolstatus = swModel.Extension.SelectByID2("", "FACE", 0.04239074672171, 0.015874999999999, 0.3283508339712, False, 1, Nothing,
0)

    ' Create the Move Face feature by
    ' translating the selected face
    Set swFeat = swFeatMgr.InsertMoveFace3(swMoveFaceTypeTranslate, False, 0, 0, (transParams), Nothing, swEndCondBlind, 0)

    ' Modify the Move Face feature
    Set swMoveFaceFeat = swFeat.GetDefinition

    ' Roll back the Move Face feature
    swMoveFaceFeat.AccessSelections swModel, Nothing

    triadParams(0) = 0
    triadParams(1) = 0.1
    triadParams(2) = 0

    transParams = triadParams

    swMoveFaceFeat.TriadTranslationParameters = (transParams)

    ' Roll back the part with the modified Move Face feature
    swFeat.ModifyDefinition swMoveFaceFeat, swModel, Nothing
End Sub
```

Create and Modify Move Face Feature Example (VB.NET)

This example shows how to create a Move Face feature by translating a face on a part.

```

'-----
' Preconditions: Verify that the specified part exists.
'
' Postconditions:
' 1. Creates Move Face1 in the FeatureManager design tree.
' 2. Modifies the translation parameters of Move Face1.
'
' NOTE: Because the part is used, do not save changes.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

```

```
Partial Class SolidWorksMacro
```

```

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swFeatMgr As FeatureManager
    Dim swFeat As Feature
    Dim swMoveFaceFeat As MoveFaceFeatureData
    Dim transParams As Object
    Dim boolstatus As Boolean
    Dim triadParams(0 To 2) As Double
    Dim fileerror As Integer
    Dim filewarning As Integer

    Sub main()

        ' Open the SOLIDWORKS Design document
        swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\assemblymates\knee.sldprt", swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", filee

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swFeatMgr = swModel.FeatureManager

        ' Translation parameters
        triadParams(0) = 0
        triadParams(1) = 0.05
        triadParams(2) = 0

        transParams = triadParams

        ' Select face to move
        boolstatus = swModel.Extension.SelectByID2("", "FACE", 0.04239074672171, 0.015874999999999, 0.3283508339712, False, 1, Nothir

        ' Create the Move Face feature by
        ' translating the selected face
        swFeat = swFeatMgr.InsertMoveFace3(swMoveFaceType_e.swMoveFaceTypeTranslate, False, 0, 0, (transParams), Nothing, swEndCondi

        ' Modify the Move Face feature
        swMoveFaceFeat = swFeat.GetDefinition

        ' Roll back the Move Face feature
        swMoveFaceFeat.AccessSelections(swModel, Nothing)

        triadParams(0) = 0
        triadParams(1) = 0.1
        triadParams(2) = 0

        transParams = triadParams

        swMoveFaceFeat.TriadTranslationParameters = (transParams)

        ' Roll back the part with the modified Move Face feature
        swFeat.ModifyDefinition(swMoveFaceFeat, swModel, Nothing)

    End Sub

    Public swApp As SldWorks
End Class

```

Create and Modify Variable-pitch Helix Example (C#)

This example shows how to create and modify a variable-pitch helix.

```
//-----
// Preconditions:
// 1. Specified part template exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens a new part document.
// 2. Selects Front Plane, creates a circle, and
//    and uses the circle to create a variable-pitch
//    helix feature named Helix/Spiral1.
// 3. Gets whether the Helix/Spiral1 feature is a
//    variable-pitch helix. If so:
//    a. Prints to the Immediate window the number
//       of regions.
//    b. Prints to the Immediate window
//       each region's diameter, pitch, height,
//       and revolution.
//    c. If second region of variable-pitch helix
//       is defined by height and revolution:
//       1. Modifies region's diameter, height, and
//          revolution values.
//       2. Prints to the Immediate window the status
//          of modifications made in previous step.
//    d. Deletes the last region in the Helix/Spiral1 feature and
//       prints the status of the deletion to the Immediate window.
//    e. Adds a new last region to the Helix/Spiral1 feature and
//       prints the status of the addition to the Immediate window.
//    - or -
//    Prints to the Immediate window that the Helix/Spiral1
//    feature is not a variable-pitch helix.
// 4. Examine the Immediate window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace HelixCSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            PartDoc swPart = default(PartDoc);
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExt swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchMgr = default(SketchManager);
            FeatureManager swFeatureMgr = default(FeatureManager);
            Feature swFeat = default(Feature);
            HelixFeatureData swHelixFeatData = default(HelixFeatureData);
            bool status = false;
            int i = 0;
            int helixType = 0;
            int helixStatus = 0;
            int[] helixRegionArray = new int[1];

            swPart = (PartDoc)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SolidWorks 2015\\templates\\Part.prtdot", 0, 0, 0);
            swModel = (ModelDoc2)swPart;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Select plane on which to create circle
            //for variable-pitch helix
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", -0.0571253507530972, 0.0536428819342089, 0.00349118658744337, false, 0, null, 0);
            swSketchMgr = (SketchManager)swModel.SketchManager;
            swSketchMgr.InsertSketch(true);
            //Create circle for variable-pitch helix
            swSketchMgr.CreateCircle(0.007581, 0.053509, 0.0, 0.013533, 0.016475, 0.0);
            // Create a variable-pitch helix using the just-sketched circle
            swFeatureMgr = (FeatureManager)swModel.FeatureManager;
            status = swFeatureMgr.InsertVariablePitchHelix(false, true, (int)swHelixDefinedBy_e.swHelixDefinedByHeightAndRevolution, 4.712388980385);
            status = swFeatureMgr.AddVariablePitchHelixFirstPitchAndDiameter(0.053, 0.05382625271268);
            status = swFeatureMgr.AddVariablePitchHelixSegment(0.0265, 0.05382625271268, 0.5);
            status = swFeatureMgr.AddVariablePitchHelixSegment(0.03975, 0.05382625271268, 0.75);
            status = swFeatureMgr.AddVariablePitchHelixSegment(0.046375, 0.05382625271268, 0.875);
            status = swFeatureMgr.AddVariablePitchHelixSegment(0.053, 0.05382625271268, 1);
            swFeat = swFeatureMgr.EndVariablePitchHelix();
            //Get variable-pitch helix
            swHelixFeatData = (HelixFeatureData)swFeat.GetDefinition();
            if (swHelixFeatData.VariablePitch)
            {
                Debug.Print(" Number of regions: " + swHelixFeatData.PitchCount);
                for (i = 1; i <= swHelixFeatData.PitchCount; i++)
                {
                    Debug.Print(" Region " + i + ":");
                    Debug.Print(" Diameter: " + swHelixFeatData.GetRegionParameterAtIndex(i, (int)swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Diameter));
                    Debug.Print(" Pitch: " + swHelixFeatData.GetRegionParameterAtIndex(i, (int)swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Pitch));
                    Debug.Print(" Height: " + swHelixFeatData.GetRegionParameterAtIndex(i, (int)swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Height));
                    Debug.Print(" Revolutions: " + swHelixFeatData.GetRegionParameterAtIndex(i, (int)swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Revolution));
                }
                //Modify region 2 of variable-pitch helix
                //defined by height and revolution
                helixType = swHelixFeatData.DefinedBy;

                switch (helixType)
                {
                    case (int)swHelixDefinedBy_e.swHelixDefinedByHeightAndRevolution:
                        if (i >= 2)
                        {
                            //Cannot change pitch
                            //Can change diameter, height, and revolution
                            //Revolution must be smaller than previous region's
                        }
                    }
                }
            }
        }
    }
}
```

```

        //revolution and less than next region's revolution
        Debug.Print("");
        Debug.Print("Helix defined by height and revolution:");
        Debug.Print("    Region modified: 2");
        helixStatus = swHelixFeatData.SetRegionParameterAtIndex(2, (int)swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Diameter, 0.052);
        Debug.Print("    Diameter modified (0 = success): " + helixStatus);
        helixStatus = swHelixFeatData.SetRegionParameterAtIndex(2, (int)swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Height, 0.025);
        Debug.Print("    Height modified (0 = success): " + helixStatus);
        helixStatus = swHelixFeatData.SetRegionParameterAtIndex(2, (int)swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Revolution, 0.45);
        Debug.Print("    Revolution modified (0 = success): " + helixStatus);
    }
    else
    {
        Debug.Print("Less than three regions in Helix/Spiral1 feature.");
    }
    break;

case (int)swHelixDefinedBy_e.swHelixDefinedByHeightAndPitch:
    //Cannot change revolution
    //TODO: Add code for variable-pitch helix defined by height and pitch
    break;

case (int)swHelixDefinedBy_e.swHelixDefinedByPitchAndRevolution:
    //Cannot change height
    //TODO: Add code for variable-pitch helix defined by pitch and revolution
    break;
}

//Delete last region in the Helix/Spiral1 feature
i = i - 1;
helixRegionArray[0] = (int)i;
Debug.Print("");
status = swHelixFeatData.DeleteRecord(helixRegionArray);
Debug.Print("Last region in Helix/Spiral1 deleted; i.e., region " + i + " was deleted: " + status);

//Add new region to end of Region parameters table
double[] record = new double[4];
//Height
record[0] = 0.055;
//Number of revolutions
record[1] = 1;
//0 indicates that this value cannot be specified
//for this type of variable-pitch helix (Height and Revolution)
//Instead, SOLIDWORKS calculates it
record[2] = 0;
//Diameter
record[3] = 0.05382625271268;
status = swHelixFeatData.AddLastRecord(record);
Debug.Print("New region 5 added as last record to Helix/Spiral1: " + status);

status = swFeat.ModifyDefinition(swHelixFeatData, swModel, null);
}
else
{
    Debug.Print("Helix is not variable pitch.");
}
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Create and Modify Variable-pitch Helix Example (VBA)

This example shows how to create and modify a variable-pitch helix.

```

'-----
' Preconditions:
' 1. Specified part template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Selects Front Plane, creates a circle, and
'    and uses the circle to create a variable-pitch
'    helix feature named Helix/Spiral1.
' 3. Gets whether the Helix/Spiral1 feature is a
'    variable-pitch helix. If so:
'    a. Prints to the Immediate window the number
'       of regions.
'    b. Prints to the Immediate window
'       each region's diameter, pitch, height,
'       and revolution.
'    c. If region 2 of the variable-pitch helix
'       is defined by height and revolution:
'       1. Modifies region's diameter, height, and
'          revolution values.
'       2. Prints to the Immediate window the status
'          of modifications made in previous step.
'    d. Deletes the last region in the Helix/Spiral1 feature and
'       prints the status of the deletion to the Immediate window.
'    e. Adds a new last region to the Helix/Spiral1 feature and
'       prints the status of the addition to the Immediate window.
'    - or -
'    Prints to the Immediate window that the Helix/Spiral1
'    feature is not a variable-pitch helix.
' 4. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swPart As SldWorks.PartDoc
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchMgr As SldWorks.SketchManager
Dim swFeatureMgr As SldWorks.FeatureManager
Dim swFeat As SldWorks.Feature
Dim swHelixFeatData As SldWorks.HelixFeatureData
Dim status As Boolean
Dim i As Long
Dim helixType As Long
Dim helixStatus As Long
Dim helixRegionArray(0) As Long

Sub main()

    Set swApp = Application.SldWorks

    Set swPart = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)
    Set swModel = swPart
    Set swModelDocExt = swModel.Extension

    'Select plane on which to create circle
    'for variable-pitch helix
    status = swModelDocExt.SelectByID("Front Plane", "PLANE", -5.71253507530972E-02, 5.36428819342089E-02, 3.49118658744337E-03, False, 0, Nothing, 0)
    Set swSketchMgr = swModel.SketchManager
    swSketchMgr.InsertSketch (True)
    'Create circle for variable-pitch helix
    swSketchMgr.CreateCircle 0.007581, 0.053509, 0#, 0.013533, 0.016475, 0#
    'Create a variable-pitch helix using the just-sketched circle
    Set swFeatureMgr = swModel.FeatureManager
    status = swFeatureMgr.InsertVariablePitchHelix(False, True, swHelixDefinedBy_e.swHelixDefinedByHeightAndRevolution, 4.712388980385)
    status = swFeatureMgr.AddVariablePitchHelixFirstPitchAndDiameter(0.053, 0.05382625271268)
    status = swFeatureMgr.AddVariablePitchHelixSegment(0.0265, 0.05382625271268, 0.5)
    status = swFeatureMgr.AddVariablePitchHelixSegment(0.03975, 0.05382625271268, 0.75)
    status = swFeatureMgr.AddVariablePitchHelixSegment(0.046375, 0.05382625271268, 0.875)
    status = swFeatureMgr.AddVariablePitchHelixSegment(0.053, 0.05382625271268, 1)
    Set swFeat = swFeatureMgr.EndVariablePitchHelix()

    'Get variable-pitch helix
    Set swHelixFeatData = swFeat.GetDefinition
    If swHelixFeatData.VariablePitch Then
        Debug.Print (" Number of regions: " & swHelixFeatData.PitchCount)
        For i = 1 To swHelixFeatData.PitchCount
            Debug.Print (" Region " & i & ":")
            Debug.Print (" Diameter: " & swHelixFeatData.GetRegionParameterAtIndex(i,
swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Diameter))
            Debug.Print (" Pitch: " & swHelixFeatData.GetRegionParameterAtIndex(i,
swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Pitch))
            Debug.Print (" Height: " & swHelixFeatData.GetRegionParameterAtIndex(i,
swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Height))
            Debug.Print (" Revolutions: " & swHelixFeatData.GetRegionParameterAtIndex(i,
swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Revolution))
        Next i

        'Modify region 2 of variable-pitch helix
        'defined by height and revolution
        helixType = swHelixFeatData.DefinedBy
    End If
End Sub

```

```

Select Case helixType
    Case swHelixDefinedBy_e.swHelixDefinedByHeightAndRevolution
        If i >= 2 Then
            'Cannot change pitch
            'Can change diameter, height, and revolution
            'Revolution must be smaller than previous region's
            'revolution and less than next region's revolution
            Debug.Print ("")
            Debug.Print ("Helix defined by height and revolution:")
            Debug.Print ("    Region modified: 2")
            helixStatus = swHelixFeatData.SetRegionParameterAtIndex(2,
swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Diameter, 0.052)
            Debug.Print ("    Diameter modified (0 = success): " & helixStatus)
            helixStatus = swHelixFeatData.SetRegionParameterAtIndex(2,
swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Height, 0.025)
            Debug.Print ("    Height modified (0 = success): " & helixStatus)
            helixStatus = swHelixFeatData.SetRegionParameterAtIndex(2,
swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Revolution, 0.45)
            Debug.Print ("    Revolution modified (0 = success): " & helixStatus)
        Else
            Debug.Print "Less than three regions in Helix/Spiral1 feature.")
        End If

    Case swHelixDefinedBy_e.swHelixDefinedByHeightAndPitch
        'Cannot change revolution
        'TODO: Add code for variable-pitch helix defined by height and pitch

    Case swHelixDefinedBy_e.swHelixDefinedByPitchAndRevolution
        'Cannot change height
        'TODO: Add code for variable-pitch helix defined by pitch and revolution

End Select

'Delete last region in the Helix/Spiral1 feature
helixRegionArray(0) = (i - 1)
Debug.Print ("")
status = swHelixFeatData.DeleteRecord(helixRegionArray)
Debug.Print "Last region in Helix/Spiral1 deleted; i.e., region " & i - 1 & " was deleted: " & status

'Add new region to end of Region parameters table
Dim record(3) As Double
'Height
record(0) = 0.055
'Number of revolutions
record(1) = 1
'0 indicates that this value cannot be specified
'for this type of variable-pitch helix (Height and Revolution)
'Instead, SOLIDWORKS calculates it
record(2) = 0
'Diameter
record(3) = 0.05382625271268
status = swHelixFeatData.AddLastRecord(record)
Debug.Print "New region 5 added as last record to Helix/Spiral1: " & status

status = swFeat.ModifyDefinition(swHelixFeatData, swModel, Nothing)

Else
    Debug.Print "Helix is not variable pitch."

End If

End Sub

```

Create and Modify Variable-pitch Helix Example (VB.NET)

This example shows how to create and modify a variable-pitch helix.

```
.....
' Preconditions:
' 1. Specified part template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Selects Front Plane, creates a circle, and
'    and uses the circle to create a variable-pitch
'    helix feature named Helix/Spiral1.
' 3. Gets whether the Helix/Spiral1 feature is a
'    variable-pitch helix. If so:
'    a. Prints to the Immediate window the number
'       of regions.
'    b. Prints to the Immediate window
'       each region's diameter, pitch, height,
'       and revolution.
'    c. If region 2 of the variable-pitch helix
'       is defined by height and revolution:
'       1. Modifies region's diameter, height, and
'          revolution values.
'       2. Prints to the Immediate window the status
'          of modifications made in previous step.
'    d. Deletes the last region in the Helix/Spiral1 feature and
'       prints the status of the deletion to the Immediate window.
'    e. Adds a new last region to the Helix/Spiral1 feature and
'       prints the status of the addition to the Immediate window.
'    or
'    Prints to the Immediate window that the Helix/Spiral1
'    feature is not a variable-pitch helix.
' 4. Examine the Immediate window.
.....
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swPart As PartDoc
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchMgr As SketchManager
        Dim swFeatureMgr As FeatureManager
        Dim swFeat As Feature
        Dim swHelixFeatData As HelixFeatureData
        Dim status As Boolean
        Dim i As Integer
        Dim helixType As Integer
        Dim helixStatus As Integer
        Dim helixRegionArray(0) As Integer

        swPart = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)
        swModel = swPart
        swModelDocExt = swModel.Extension

        'Select plane on which to create circle
        'for variable-pitch helix
        status = swModelDocExt.SelectByID2("Front Plane", "PLANE", -0.0571253507530972, 0.0536428819342089, 0.00349118658744337, False, 0, Nothing, 0)
        swSketchMgr = swModel.SketchManager
        swSketchMgr.InsertSketch(True)
        'Create circle for variable-pitch helix
        swSketchMgr.CreateCircle(0.007581, 0.053509, 0.0#, 0.013533, 0.016475, 0.0#)
        'Create a variable-pitch helix using the just-sketched circle
        swFeatureMgr = swModel.FeatureManager
        status = swFeatureMgr.InsertVariablePitchHelix(False, True, swHelixDefinedBy_e.swHelixDefinedByHeightAndRevolution, 4.712388980385)
        status = swFeatureMgr.AddVariablePitchHelixFirstPitchAndDiameter(0.053, 0.05382625271268, 0.5)
        status = swFeatureMgr.AddVariablePitchHelixSegment(0.0265, 0.05382625271268, 0.75)
        status = swFeatureMgr.AddVariablePitchHelixSegment(0.03975, 0.05382625271268, 0.875)
        status = swFeatureMgr.AddVariablePitchHelixSegment(0.046375, 0.05382625271268, 0.875)
        status = swFeatureMgr.AddVariablePitchHelixSegment(0.053, 0.05382625271268, 1)
        swFeat = swFeatureMgr.EndVariablePitchHelix()
        'Get variable-pitch helix
        swHelixFeatData = swFeat.GetDefinition
        If swHelixFeatData.VariablePitch Then
            Debug.Print(" Number of regions: " & swHelixFeatData.PitchCount)
            For i = 1 To swHelixFeatData.PitchCount
                Debug.Print(" Region " & i & " :")
                Debug.Print(" Diameter: " & swHelixFeatData.GetRegionParameterAtIndex(i, swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Diameter))
                Debug.Print(" Pitch: " & swHelixFeatData.GetRegionParameterAtIndex(i, swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Pitch))
                Debug.Print(" Height: " & swHelixFeatData.GetRegionParameterAtIndex(i, swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Height))
                Debug.Print(" Revolutions: " & swHelixFeatData.GetRegionParameterAtIndex(i, swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Revolution))
            Next i
            'Modify Region 2 of variable-pitch helix
            'defined by height and revolution
            helixType = swHelixFeatData.DefinedBy
            Select Case helixType
                Case swHelixDefinedBy_e.swHelixDefinedByHeightAndRevolution
                    If i >= 2 Then
                        'Cannot change pitch
                        'Can change diameter, height, and revolution
                        'Revolution must be smaller than previous region's
                        'revolution and less than next region's revolution
                        Debug.Print("")
                        Debug.Print("Helix defined by height and revolution:")
                        Debug.Print(" Region modified: 2")
                        helixStatus = swHelixFeatData.SetRegionParameterAtIndex(2, swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Diameter, 0.052)
                        Debug.Print(" Diameter modified (0 = success): " & helixStatus)
                        helixStatus = swHelixFeatData.SetRegionParameterAtIndex(2, swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Height, 0.025)
                        Debug.Print(" Height modified (0 = success): " & helixStatus)
                        helixStatus = swHelixFeatData.SetRegionParameterAtIndex(2, swVariablePitchHelixRegionParameter_e.swVariablePitchHelixRegionParameter_Revolution, 0.45)
                        Debug.Print(" Revolution modified (0 = success): " & helixStatus)
                    Else

```

```

        Debug.Print("Less than three regions in Helix/Spiral1 feature.")
    End If

    Case swHelixDefinedBy_e.swHelixDefinedByHeightAndPitch
        'Cannot change revolution
        'TODO: Add code for variable-pitch helix defined by height and pitch

    Case swHelixDefinedBy_e.swHelixDefinedByPitchAndRevolution
        'Cannot change height
        'TODO: Add code for variable-pitch helix defined by pitch and revolution
    End Select

    'Delete last region in the Helix/Spiral1 feature
    helixRegionArray(0) = (i - 1)

    Debug.Print("")
    status = swHelixFeatData.DeleteRecord(helixRegionArray)
    Debug.Print("Last region in Helix/Spiral1 deleted; i.e., region " & i - 1 & " was deleted: " & status)

    'Add new region to end of Region parameters table
    Dim record(3) As Double
    'Height
    record(0) = 0.055
    'Number of revolutions
    record(1) = 1
    '0 indicates that this value cannot be specified
    'for this type of variable-pitch helix (Height and Revolution)
    'Instead, SOLIDWORKS calculates it
    record(2) = 0
    'Diameter
    record(3) = 0.05382625271268
    status = swHelixFeatData.AddLastRecord(record)
    Debug.Print("New region 5 added as last record to Helix/Spiral1: " & status)

    status = swFeat.ModifyDefinition(swHelixFeatData, swModel, Nothing)
Else
    Debug.Print("Helix is not variable pitch.")
End If

```

End Sub

```

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Create and Name Planes Example (VBA)

This example shows how to create five planes that are offset from each other and change each of their names.

```
'-----
' Preconditions: Open a part that has three planes and
' one of the planes is named Plane1.
'
' Postconditions:
' 1. Creates five offset planes: MyPlane1, MyPlane2,
'   MyPlane3, MyPlane4, and MyPlane5.
' 2. Examine the FeatureManager design tree and graphics
'   area.
'-----
Option Explicit

Public Sub CreateOffsetPlanes()

    Dim swApp As SldWorks.SldWorks
    Dim Part As SldWorks.PartDoc
    Dim Model As SldWorks.ModelDoc2
    Dim planeName As String
    Dim newPlaneName As String
    Dim planeFeature As SldWorks.Feature
    Dim i As Long
    Dim planeCount As Long

    Set swApp = CreateObject("SldWorks.Application")

    ' Get the active document
    Set Model = swApp.ActiveDoc
    If Model Is Nothing Then
        Exit Sub
    End If

    ' Set name of plane for SelectByID for
    ' first pass through the loop
    newPlaneName = "Plane1"

    ' Currently three planes in the part
    planeCount = 3

    ' If your model has more than three planes or you are not
    ' sure how many planes exist in your part, then immediately
    ' after loading the part you can traverse the FeatureManager design
    ' tree (see IPartDoc::FirstFeature) and count the reference plane
    ' features using IFeature::GetTypeNames to determine which
    ' features are reference planes and add them up

    ' Loop five times and create five planes offset from each other
    For i = 1 To 5
        Model.ClearSelection
        ' Select the plane from which to offset
        Model.SelectByID newPlaneName, "PLANE", 0, 0, 0
        ' Create plane offset from selected plane
        Model.CreatePlaneAtOffset 0.1, 0
        ' Increment the plane number
        planeCount = planeCount + 1
        ' Increment the plane name
        planeName = "Plane" & planeCount
        newPlaneName = "MyPlane" & i
        ' Get the Plane feature by its name
        Set planeFeature = Model.FeatureByName(planeName)
        ' Change the Plane name
        planeFeature.Name = (newPlaneName)

        Model.EditRebuild
    Next

End Sub
```

Select and Rename Object Example (VBA)

The example shows how to verify the name of the edge and then change it, if it doesn't already have a name.

NOTE: You could also select the edge programmatically using `IFace2::GetEdges` or `ICoEdge::GetEdge`.

```
'-----
' Preconditions:
' 1. Open a part.
' 2. Select an edge on the part.
'
' Postconditions:
' 1. Verifies that the user made a selection.
' 2. Gets the selected object and gets whether the selected
'    object is an edge.
' 3. If an edge, renames the edge if the edge doesn't already
'    have a name and the name is unique to the part.
' 4. Click OK to close the message box.
'-----
Option Explicit

Sub amin()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swPart As SldWorks.PartDoc
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swEntity As SldWorks.Entity
    Dim swSelObj As Object
    Dim edgeName As String
    Dim messageString As String
    Dim ret As Boolean

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swPart = swModel
    Set swSelMgr = swModel.SelectionManager
    If (swSelMgr.GetSelectedObjectCount <= 0) Then
        Set swSelObj = swSelMgr.GetSelectedObject6(1, -1)
        Set swEntity = swSelObj
        If (swSelMgr.GetSelectedObjectType(1) = swSelEDGES) Then
            edgeName = swPart.GetEntityName(swEntity)
            If (edgeName = "") Then
                ret = swPart.SetEntityName(swEntity, "NewEdgeName")
                If ret Then
                    swApp.SendMsgToUser ("Successfully set name of edge")
                Else
                    swApp.SendMsgToUser ("Error setting name of edge")
                End If
            Else
                messageString = "Edge already has name of " + edgeName
                swApp.SendMsgToUser (messageString)
            End If
        Else
            swApp.SendMsgToUser ("Select an edge and run the macro again")
        End If
    End If
End Sub
```

Create Angular Dimension Example (C#)

This example shows how to add an angular display dimension to a sketch.

```
// Preconditions:  
// 1. Open public_documents\samples\tutorial\api\box.sldprt.  
// 2. Press F8 repeatedly to step through the macro.  
//  
// Postconditions:  
// 1. Edits Sketch1.  
// 2. Creates an angular dimension.  
// 3. Displays the dimension for:  
// * Angle defined by the selected sketch segment and an extension line  
//   that is drawn to the right of the selected sketch point  
//   (Direction = swSmartDimensionDirection_Right)  
// * Explementary angle  
// * Vertically opposite angle  
//  
// NOTE: Because the model is used elsewhere, do not save changes to it.  
// -----  
using Microsoft.VisualBasic;  
using System;  
using System.Collections;  
using System.Collections.Generic;  
using System.Data;  
using System.Diagnostics;  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
namespace AddDimension_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
  
        ModelDoc2 Part;  
        DisplayDimension myDisplayDim;  
  
        bool boolstatus;  
  
        public void Main()  
        {  
  
            Part = (ModelDoc2)swApp.ActiveDoc;  
            System.Diagnostics.Debugger.Break();  
            boolstatus = Part.Extension.SelectByID("Sketch1", "SKETCH", 0, 0, 0, false, 0, null, 0);  
            Part.EditSketch();  
  
            boolstatus = Part.Extension.SelectByID("Line2", "SKETCHSEGMENT", -0.0672289031567236, 0.0180603779387131, 0.0184698306f  
            boolstatus = Part.Extension.SelectByID("Point1", "SKETCHPOINT", -0.0811067833265636, 0.0389478433654258, 0, true, 0, nu  
  
            myDisplayDim = (DisplayDimension)Part.Extension.AddDimension(-0.0456250220540824, 0, 0.00150965590938767, (int)swSmartDi  
            myDisplayDim.ExplementaryAngle();  
            myDisplayDim.VerticallyOppositeAngle();  
  
            Part.SketchManager.InsertSketch(true);  
  
        }  
  
        public SldWorks swApp;
```

Create Angular Dimension Example (VBA)

This example shows how to add an angular display dimension to a sketch.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\box.sldprt.
' 2. Press F8 repeatedly to step through the macro.
'
' Postconditions:
' 1. Edits Sketch1.
' 2. Creates an angular dimension.
' 3. Displays the dimension for:
'   * Angle defined by the selected sketch segment and an extension line
'     that is drawn to the right of the selected sketch point
'     (Direction = swSmartDimensionDirection_Right)
'   * Explementary angle
'   * Vertically opposite angle
'
' NOTE: Because the model is used elsewhere, do not save changes to it.
'-----

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim myDisplayDim As SldWorks.DisplayDimension
Dim boolstatus As Boolean

Option Explicit
Sub main()
    Set swApp = Application.SldWorks

    Set Part = swApp.ActiveDoc
    boolstatus = Part.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    Part.EditSketch

    boolstatus = Part.Extension.SelectByID2("Line2", "SKETCHSEGMENT", -6.72289031567236E-02, 1.80603779387131E-02,
1.84698306816742E-02, False, 0, Nothing, 0)
    boolstatus = Part.Extension.SelectByID2("Point1", "SKETCHPOINT", -8.11067833265636E-02, 3.89478433654258E-02, 0, True, 0,
Nothing, 0)

    Set myDisplayDim = Part.Extension.AddDimension(-4.56250220540824E-02, 0, 1.50965590938767E-03, swSmartDimensionDirection_Right)
    myDisplayDim.ExplementaryAngle
    myDisplayDim.VerticallyOppositeAngle

    Part.SketchManager.InsertSketch True

End Sub

```

```

Preconditions:
1. Open public_documents\samples\tutorial\api\box.sldprt.
2. Press F8 repeatedly to step through the macro.

Postconditions:
1. Edits Sketch1.
2. Creates an angular dimension.
3. Displays the dimension for:
    * Angle defined by the selected sketch segment and an extension line
      that is drawn to the right of the selected sketch point
      (Direction = swSmartDimensionDirection_Right)
    * Explementary angle
    * Vertically opposite angle

NOTE: Because the model is used elsewhere, do not save changes to it.
-----

```

Partial Class SolidWorksMacro

```
Sub main()
```

```
boolstatus = Part.Extension.SelectByID2("Line2", "SKETCHSEGMENT", -0.0672289031567236, 0.0180603779387131, 0.018469830681674  
boolstatus = Part.Extension.SelectByID2("Point1", "SKETCHPOINT", -0.0811067833265636, 0.0389478433654258, 0, True, 0, Nothi
```

Part.SketchManager.InsertSketch(True)

Public swApp As SldWorks

End Class

Create Variable Radius Asymmetric Elliptical Fillet Example (C#)

This example shows how to create a variable radius asymmetric elliptical fillet and get its data.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\block.sldprt.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Creates a variable radius asymmetric elliptical fillet, VarFillet1,
//    in the FeatureManager design tree.
// 2. Inspect the Immediate window.
//
// NOTE: Because the model is used elsewhere, do not save changes.
// -----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace FeatureFillet3_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        SelectionMgr swSelMgr;
        Feature myFeature;
        Edge swedge;
        Vertex ver1;
        Vertex ver2;
        VariableFilletFeatureData2 swFeatData;
        int Fillet_ProfileTyp;
        double[] dists26 = new double[2];
        double AsyRadius1;
        double AsyRadius2;
        double AsyRadius3;
        double AsyRadius4;
        bool boolstatus;
        double[] radiis = new double[2];
        object radiiArray0;
        object conicRhosArray0;
        object setBackArray0;
        object pointArray0;
        object pointRhoArray0;
        object dist2Array0;
        object pointDist2Array0;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            radiis[0] = 0.008;
            radiis[1] = 0.009;
            radiiArray0 = radiis;
            dists26[0] = 0.006;
            dists26[1] = 0.007;
            dist2Array0 = dists26;

            conicRhosArray0 = 0;
            setBackArray0 = 0;
            pointArray0 = 0;
            pointRhoArray0 = 0;
            pointDist2Array0 = 0;

            boolstatus = swModel.Extension.SelectByID2("", "EDGE", 1.66068305868521E-02, -4.40742070395572E-06, -1.49970056632469E-02, true, 1, null, 0);

            if (boolstatus == false)
                ErrorMsg(swApp, "Failed to select edge");

            myFeature = (Feature)swModel.FeatureManager.FeatureFillet3((int)swFeatureFilletOptions_e.swFeatureFilletAsymmetric +
            (int)swFeatureFilletOptions_e.swFeatureFilletKeepFeatures + (int)swFeatureFilletOptions_e.swFeatureFilletAttachEdges +
            (int)swFeatureFilletOptions_e.swFeatureFilletUniformRadius + (int)swFeatureFilletOptions_e.swFeatureFilletPropagate, 0, 0, 0,
            (int)swFeatureFilletType_e.swFeatureFilletType_VariableRadius, (int)swFilletOverflowType_e.swFilletOverflowType_Default,
            (int)swFeatureFilletProfileType_e.swFeatureFilletCircular, (radiiArray0), (dist2Array0), (conicRhosArray0),
            (setBackArray0), (pointArray0), (pointDist2Array0), (pointRhoArray0));
            if (myFeature == null)
                ErrorMsg(swApp, "Failed to create fillet");

            swFeatData = (VariableFilletFeatureData2)myFeature.GetDefinition();
        }
    }
}
```

```

        if (swFeatData == null)
            ErrorMsg(swApp, "Failed to get definition for fillet feature");

        boolstatus = swFeatData.AccessSelections(swModel, null);
        if (boolstatus == false)
            ErrorMsg(swApp, "Failed to access selections for fillet feature");

        boolstatus = swFeatData.AsymmetricFillet;
        if (boolstatus == false)
            ErrorMsg(swApp, "Fillet is not asymmetric");
        Debug.Print("Variable size fillet is asymmetric? " + boolstatus);

        swedge = (Edge)swFeatData.GetFilletEdgeAtIndex(0);
        if (swedge == null)
            ErrorMsg(swApp, "Failed to get edge");

        ver1 = (Vertex)swedge.GetStartVertex();
        if (ver1 == null)
            ErrorMsg(swApp, "Failed to get start vertex of edge");

        ver2 = (Vertex)swedge.GetEndVertex();
        if (ver2 == null)
            ErrorMsg(swApp, "Failed to get end vertex of edge");

        AsyRadius1 = swFeatData.GetRadius2(ver1, out boolstatus);
        if (AsyRadius1 != 0.008)
            ErrorMsg(swApp, "Radius R1 at vertex V1 is wrong");
        Debug.Print("Radius R1 at vertex V1: " + AsyRadius1);

        AsyRadius2 = swFeatData.GetDistance(ver1);
        if (AsyRadius2 != 0.006)
            ErrorMsg(swApp, "Radius R2 at vertex V1 is wrong");
        Debug.Print("Radius R2 at vertex V1: " + AsyRadius2);

        AsyRadius3 = swFeatData.GetRadius2(ver2, out boolstatus);
        if (AsyRadius3 != 0.009)
            ErrorMsg(swApp, "Radius R1 for vertex V2 is wrong");
        Debug.Print("Radius R1 at vertex V2: " + AsyRadius3);

        AsyRadius4 = swFeatData.GetDistance(ver2);
        if (AsyRadius4 != 0.007)
            ErrorMsg(swApp, "Radius R2 for vertex V2 is wrong");
        Debug.Print("Radius R2 at vertex V2: " + AsyRadius4);

        Fillet_ProfileTyp = swFeatData.ConicTypeForCrossSectionProfile;
        if (Fillet_ProfileTyp != 0)
            ErrorMsg(swApp, "Profile type is not elliptical");
        Debug.Print("Fillet profile type as defined in swFeatureFilletProfileType_e (0 = Elliptical): " + Fillet_ProfileTyp);

        swFeatData.ReleaseSelectionAccess();
    }

    public void ErrorMsg(SldWorks SwApp, string Message)
    {
        SwApp.SendMsgToUser2(Message, 0, 0);
        SwApp.RecordLine("**** WARNING - General");
        SwApp.RecordLine("**** " + Message);
        SwApp.RecordLine("");
    }

    public SldWorks swApp;
}
}

```

Create Variable Radius Asymmetric Elliptical Fillet Example (VBA)

This example shows how to create a variable radius asymmetric elliptical fillet and get its data.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\block.sldprt.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a variable radius asymmetric elliptical fillet, VarFillet1,
'    in the FeatureManager design tree.
' 2. Inspect the Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
Option Explicit
Dim SwApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim myFeature As SldWorks.Feature
Dim swedge As SldWorks.Edge
Dim ver1 As Vertex
Dim ver2 As Vertex
Dim swFeatData As SldWorks.VariableFilletFeatureData2
Dim Fillet_ProfileTyp As Integer
Dim dists26(1) As Double
Dim AsyRadius1 As Double
Dim AsyRadius2 As Double
Dim AsyRadius3 As Double
Dim AsyRadius4 As Double
Dim boolstatus As Boolean
Dim radiis(1) As Double
Dim radiiArray0 As Variant
Dim conicRhosArray0 As Variant
Dim setBackArray0 As Variant
Dim pointArray0 As Variant
Dim pointRhoArray0 As Variant
Dim dist2Array0 As Variant
Dim pointDist2Array0 As Variant

Sub main()
    Set SwApp = Application.SldWorks

    Set swModel = SwApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager

    radiis(0) = 0.008
    radiis(1) = 0.009
    radiiArray0 = radiis
    dists26(0) = 0.006
    dists26(1) = 0.007
    dist2Array0 = dists26

    conicRhosArray0 = 0
    setBackArray0 = 0
    pointArray0 = 0
    pointRhoArray0 = 0
    pointDist2Array0 = 0

    boolstatus = swModel.Extension.SelectByID2("", "EDGE", 1.66068305868521E-02, -4.40742070395572E-06, -1.49970056632469E-02,
True, 1, Nothing, 0)
    If boolstatus = False Then ErrorMsg SwApp, "Failed to select edge"

    Set myFeature = swModel.FeatureManager.FeatureFillet3(swFeatureFilletAsymmetric + swFeatureFilletKeepFeatures +
swFeatureFilletAttachEdges + swFeatureFilletUniformRadius + swFeatureFilletPropagate, 0, 0, 0, swFeatureFilletType_VariableRadius,
swFilletOverflowType_Default, swFeatureFilletCircular, (radiiArray0), (dist2Array0), (conicRhosArray0), (setBackArray0),
(pointArray0), (pointDist2Array0), (pointRhoArray0))
    If myFeature Is Nothing Then ErrorMsg SwApp, "Failed to create fillet"

    Set swFeatData = myFeature.GetDefinition()
    If swFeatData Is Nothing Then ErrorMsg SwApp, "Failed to get definition for fillet feature"

    boolstatus = swFeatData.AccessSelections(swModel, Nothing)
    If boolstatus = False Then ErrorMsg SwApp, "Failed to access selections for fillet feature"

    boolstatus = swFeatData.AsymmetricFillet
    If boolstatus = False Then ErrorMsg SwApp, "Fillet is not asymmetric"
    Debug.Print "Variable size fillet is asymmetric? " & boolstatus

    Set swedge = swFeatData.GetFilletEdgeAtIndex(0)
    If swedge Is Nothing Then ErrorMsg SwApp, "Failed to get edge"

    Set ver1 = swedge.GetStartVertex
    If ver1 Is Nothing Then ErrorMsg SwApp, "Failed to get start vertex of edge"

```

```

Set ver2 = swedge.GetEndVertex
If ver2 Is Nothing Then ErrorMsg SwApp, "Failed to get end vertex of edge"

AsyRadius1 = swFeatData.GetRadius2(ver1, True)
If AsyRadius1 <> 0.008 Then ErrorMsg SwApp, "Radius R1 at vertex V1 is wrong"
Debug.Print "Radius R1 at vertex V1: " & AsyRadius1

AsyRadius2 = swFeatData.GetDistance(ver1)
If AsyRadius2 <> 0.006 Then ErrorMsg SwApp, "Radius R2 at vertex V1 is wrong"
Debug.Print "Radius R2 at vertex V1: " & AsyRadius2

AsyRadius3 = swFeatData.GetRadius2(ver2, True)
If AsyRadius3 <> 0.009 Then ErrorMsg SwApp, "Radius R1 for vertex V2 is wrong"
Debug.Print "Radius R1 at vertex V2: " & AsyRadius3

AsyRadius4 = swFeatData.GetDistance(ver2)
If AsyRadius4 <> 0.007 Then ErrorMsg SwApp, "Radius R2 for vertex V2 is wrong"
Debug.Print "Radius R2 at vertex V2: " & AsyRadius4

Fillet_ProfileTyp = swFeatData.ConicTypeForCrossSectionProfile
If Fillet_ProfileTyp <> 0 Then ErrorMsg SwApp, "Profile type is not elliptical"
Debug.Print "Fillet profile type as defined in swFeatureFilletProfileType_e (0 = Elliptical): " & Fillet_ProfileTyp

swFeatData.ReleaseSelectionAccess

End Sub

Function ErrorMsg(SwApp As SldWorks.SldWorks, Message As String)
    SwApp.SendMsgToUser2 Message, 0, 0
    SwApp.RecordLine "**** WARNING - General"
    SwApp.RecordLine "**** " & Message
    SwApp.RecordLine ""
End Function

```

Create Variable Radius Asymmetric Elliptical Fillet Example (VB.NET)

This example shows how to create a variable radius asymmetric elliptical fillet and get its data.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\block.sldprt.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates a variable radius asymmetric elliptical fillet, VarFillet1,
'    in the FeatureManager design tree.
' 2. Inspect the Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swSelMgr As SelectionMgr
    Dim myFeature As Feature
    Dim swedge As Edge
    Dim ver1 As Vertex
    Dim ver2 As Vertex
    Dim swFeatData As VariableFilletFeatureData2
    Dim Fillet_ProfileTyp As Integer
    Dim dists26(1) As Double
    Dim AsyRadius1 As Double
    Dim AsyRadius2 As Double
    Dim AsyRadius3 As Double
    Dim AsyRadius4 As Double
    Dim boolstatus As Boolean
    Dim radiis(1) As Double
    Dim radiiArray0 As Object
    Dim conicRhosArray0 As Object
    Dim setBackArray0 As Object
    Dim pointArray0 As Object
    Dim pointRhoArray0 As Object
    Dim dist2Array0 As Object
    Dim pointDist2Array0 As Object

    Sub main()

        swModel = SwApp.ActiveDoc
        swSelMgr = swModel.SelectionManager

        radiis(0) = 0.008
        radiis(1) = 0.009
        radiiArray0 = radiis
        dists26(0) = 0.006
        dists26(1) = 0.007
        dist2Array0 = dists26

        conicRhosArray0 = 0
        setBackArray0 = 0
        pointArray0 = 0
        pointRhoArray0 = 0
        pointDist2Array0 = 0

        boolstatus = swModel.Extension.SelectByID("", "EDGE", 0.0166068305868521, -0.00000440742070395572, -0.0149970056632469,
True, 1, Nothing, 0)

        If boolstatus = False Then ErrorMsg(SwApp, "Failed to select edge")

        myFeature = swModel.FeatureManager.FeatureFillet3(swFeatureFilletOptions_e.swFeatureFilletAsymmetric +
swFeatureFilletOptions_e.swFeatureFilletKeepFeatures + swFeatureFilletOptions_e.swFeatureFilletAttachEdges +
swFeatureFilletOptions_e.swFeatureFilletUniformRadius + swFeatureFilletOptions_e.swFeatureFilletPropagate, 0, 0, 0,
swFeatureFilletType_e.swFeatureFilletType_VariableRadius, swFilletOverFlowType_e.swFilletOverFlowType_Default,
swFeatureFilletProfileType_e.swFeatureFilletCircular, (radiiArray0), (dist2Array0), (conicRhosArray0), (setBackArray0),
(pointArray0), (pointDist2Array0), (pointRhoArray0))
        If myFeature Is Nothing Then ErrorMsg(SwApp, "Failed to create fillet")

        swFeatData = myFeature.GetDefinition()
        If swFeatData Is Nothing Then ErrorMsg(SwApp, "Failed to get definition for fillet feature")

        boolstatus = swFeatData.AccessSelections(swModel, Nothing)
        If boolstatus = False Then ErrorMsg(SwApp, "Failed to access selections for fillet feature")

        boolstatus = swFeatData.AsymmetricFillet
        If boolstatus = False Then ErrorMsg(SwApp, "Fillet is not asymmetric")
        Debug.Print("Variable size fillet is asymmetric? " & boolstatus)
    End Sub
End Class

```

```

swedge = swFeatData.GetFilletEdgeAtIndex(0)
If swedge Is Nothing Then ErrorMsg(SwApp, "Failed to get edge")

ver1 = swedge.GetStartVertex
If ver1 Is Nothing Then ErrorMsg(SwApp, "Failed to get start vertex of edge")

ver2 = swedge.GetEndVertex
If ver2 Is Nothing Then ErrorMsg(SwApp, "Failed to get end vertex of edge")

AsyRadius1 = swFeatData.GetRadius2(ver1, True)
If AsyRadius1 <> 0.008 Then ErrorMsg(SwApp, "Radius R1 at vertex V1 is wrong")
Debug.Print("Radius R1 at vertex V1: " & AsyRadius1)

AsyRadius2 = swFeatData.GetDistance(ver1)
If AsyRadius2 <> 0.006 Then ErrorMsg(SwApp, "Radius R2 at vertex V1 is wrong")
Debug.Print("Radius R2 at vertex V1: " & AsyRadius2)

AsyRadius3 = swFeatData.GetRadius2(ver2, True)
If AsyRadius3 <> 0.009 Then ErrorMsg(SwApp, "Radius R1 for vertex V2 is wrong")
Debug.Print("Radius R1 at vertex V2: " & AsyRadius3)

AsyRadius4 = swFeatData.GetDistance(ver2)
If AsyRadius4 <> 0.007 Then ErrorMsg(SwApp, "Radius R2 for vertex V2 is wrong")
Debug.Print("Radius R2 at vertex V2: " & AsyRadius4)

Fillet_ProfileType = swFeatData.ConicTypeForCrossSectionProfile
If Fillet_ProfileType <> 0 Then ErrorMsg(SwApp, "Profile type is not elliptical")
Debug.Print("Fillet profile type as defined in swFeatureFilletProfileType_e (0 = Elliptical): " & Fillet_ProfileType)

swFeatData.ReleaseSelectionAccess()

```

End Sub

```

Sub ErrorMsg(ByVal SwApp As SldWorks, ByVal Message As String)
    SwApp.SendMsgToUser2(Message, 0, 0)
    SwApp.RecordLine("*** WARNING - General")
    SwApp.RecordLine("*** " & Message)
    SwApp.RecordLine("")
End Sub

```

End Sub

Public swApp As SldWorks

End Class

Create Attribute Example (VBA)

This example shows how to create an instance of attribute and display that attribute in the FeatureManager design tree.

```
'-----
' Preconditions:
' 1. Verify that the specified part document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Selects Cut-Extrude3.
' 3. Adds an attribute to the selected feature.
' 4. Rebuilds the part.
' 5. Expand Cut-Extrude3 in the FeatureManager design tree
'    to verify that the att 1 attribute added to that feature.
' 6. Examine the Immediate window.
'
' NOTES:
' * Attribute definition is as follows:
'   Name = "XML_string"
'   Parameter:
'     Name = "att_name"
'     Type = swParamTypeString
'   Parameter:
'     Name = "att_value"
'     Type = swParamTypeDouble
' * Attribute is not shown in FeatureManager design tree
'   until you rebuild, because attributes
'   are normally created hidden and refreshing the
'   FeatureManager design tree is an expensive operation.
' * Attributes are allocated by name, and the names must be
'   unique across the model.
' * The selected object must present the IEntity interface.
' * Because this part document is used elsewhere, do not save changes.
'-----
Option Explicit

Sub main()

    '1 = invisible
    '0 = visible
    Const CreateVisible As Long = 0
    Const AttDefName As String = "XML_string"
    Const AttLenDimName As String = "att_name"
    Const AttLenValueName As String = "att_value"
    Const AttRootName As String = "att"
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As Object
    Dim swEnt As SldWorks.Entity
    Dim swAttDef As SldWorks.AttributeDef
    Dim swAtt As SldWorks.Attribute
    Dim swParamName As SldWorks.Parameter
    Dim swParamValue As SldWorks.Parameter
    Dim AttName As String
    Dim i As Long
    Dim bRet As Boolean
    Dim fileName As String
    Dim errors As Long
    Dim warnings As Long

    Set swApp = CreateObject("SldWorks.Application")

    'Open part document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\smartcomponents\bearing.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Select feature
    Set swModelDocExt = swModel.Extension
    bRet = swModelDocExt.SelectByID2("Cut-Extrude3", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swEnt = swFeat

    'Set attribute
    Set swAttDef = swApp.DefineAttribute(AttDefName)
    bRet = swAttDef.AddParameter(AttLenDimName, swParamTypeString, 0#, 0)
    bRet = swAttDef.AddParameter(AttLenValueName, swParamTypeDouble, 0#, 0)

    bRet = swAttDef.Register

    While swAtt Is Nothing
        ' Get a unique attribute name
        i = i + 1
        AttName = AttRootName + Str(i)
        Set swAtt = swAttDef.CreateInstance5(swModel, swEnt, AttName, CreateVisible, swThisConfiguration)
    Wend

    Set swParamName = swAtt.GetParameter(AttLenDimName)
    Set swParamValue = swAtt.GetParameter(AttLenValueName)

    bRet = swParamName.SetString2(AttName & " - " & AttLenDimName, swAllConfiguration, "")
    bRet = swParamValue.SetDouble2(i * 10, swAllConfiguration, "")
```

```
Debug.Print "File = " & swModel.GetPathName
If Not swAtt Is Nothing Then
    Debug.Print " " & AttDefName & "(" & i - 1 & ") = " & AttName
Else
    Debug.Print " Attribute not created."
End If

swModel.ForceRebuild3 True

End Sub
```

Create Auto Route Example (C#)

This example shows how to connect two points using the Auto Route interface.

```
//-----
// Preconditions:
// 1. In SOLIDWORKS Design, click Tools > Add-Ins > SOLIDWORKS Routing.
// 2. In the IDE, right-click the project, select Add Reference,
//    browse install_dir\api\redist, and click
//    SolidWorks.Interop.SWRoutingLib.dll.
// 3. In Tools > Options > System Options > Routing > Routing File Locations,
//    add the locations of your SOLIDWORKS Design routing files.
// 4. Open a routing assembly that contains Route1 and two sketch points.
// 5. Modify the swModel.Extension.SelectByID2 parameters to select two sketch
//    points through which to auto route.
// 6. Select the assembly that contains Route1 in the FeatureManager design tree.
// 7. Open an Immediate Window.
//
// Postconditions: Auto Route connects the selected sketch points.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.SWRoutingLib;
using System;
using System.Diagnostics;
namespace CreateAutoRoute_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            AssemblyDoc swTopLevelAssembly = default(AssemblyDoc);
            RouteManager rtRouteManager = default(RouteManager);
            AutoRoute autoRoute = default(AutoRoute);
            int resultCode = 0;
            Boolean boolstatus = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swTopLevelAssembly = (AssemblyDoc)swModel;

            // Get the RouteManager from the top-level assembly
            rtRouteManager = (RouteManager)swTopLevelAssembly.GetRouteManager();
            if (rtRouteManager == null)
            {
                Debug.Print("No RouteManager found in top-level document");
                return;
            }

            rtRouteManager.EditRoute();
            autoRoute = (AutoRoute)rtRouteManager.GetAutoRoute();
            autoRoute.ShowGuidelines();
            boolstatus = swModel.Extension.SelectByID2("Point3", "SKETCHPOINT", -0.457835, 0.10795, -0.2032, true, 0, null, 0);
            boolstatus = swModel.Extension.SelectByID2("Point8", "SKETCHPOINT", -0.218948, 0.0508, -0.470281, true, 0, null, 0);
            resultCode = autoRoute.CreatePointToPointAutoRoute(2);
            rtRouteManager.ExitRoute();

            swTopLevelAssembly.EditAssembly();
        }

        public SldWorks swApp;
    }
}
```

Create Auto Route Example (VBA)

This example shows how to connect two points using the Auto Route interface.

```

'-----
' Preconditions:
' 1. In SOLIDWORKS Design, click Tools > Add-Ins > SOLIDWORKS Routing.
' 2. In the IDE, click Tools > References and select
' SOLIDWORKS version Routing Type Library.
' 3. In Tools > Options > System Options > Routing > Routing File Locations,
' add the locations of your SOLIDWORKS Design routing files.
' 4. Open a routing assembly that contains Route1 and two sketch points.
' 5. Modify the swModel.Extension.SelectByID2 parameters to select two sketch
' points through which to auto route.
' 6. Select the assembly that contains Route1 in the FeatureManager design tree.
' 7. Open an Immediate Window.
'
' Postconditions: Auto Route connects the selected sketch points.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----

```

```

Dim swApp As SldWorks.SldWorks
Option Explicit

```

```

Sub main()

```

```

    Dim swModel As SldWorks.ModelDoc2
    Dim swTopLevelAssembly As SldWorks.AssemblyDoc
    Dim rtRouteManager As SwRoutingLib.RouteManager
    Dim autoRoute As SwRoutingLib.autoRoute
    Dim resultCode, boolstatus As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swTopLevelAssembly = swModel

    ' Get the RouteManager from the top-level assembly
    Set rtRouteManager = swTopLevelAssembly.GetRouteManager
    If rtRouteManager Is Nothing Then
        Debug.Print "No RouteManager found in top-level document: " & swTopLevelAssembly.GetPathName
        Exit Sub
    End If

    rtRouteManager.EditRoute
    Set autoRoute = rtRouteManager.GetAutoRoute
    autoRoute.ShowGuidelines
    boolstatus = swModel.Extension.SelectByID2("Point3", "SKETCHPOINT", -0.457835, 0.10795, -0.2032, True, 0, Nothing, 0)
    boolstatus = swModel.Extension.SelectByID2("Point8", "SKETCHPOINT", -0.218948, 0.0508, -0.470281, True, 0, Nothing, 0)
    resultCode = autoRoute.CreatePointToPointAutoRoute(swOrthogonalMode)
    rtRouteManager.ExitRoute
    swTopLevelAssembly.EditAssembly

```

```

End Sub

```

Create Auto Route Example (VB.NET)

This example shows how to connect two points using the Auto Route interface.

```

'-----
' Preconditions:
' 1. In SOLIDWORKS Design, click Tools > Add-Ins > SOLIDWORKS Routing.
' 2. In the IDE, right-click the project, select Add Reference,
'    browse install_dir\api\redist, and click
'    SolidWorks.Interop.SWRoutingLib.dll.
' 3. In Tools > Options > System Options > Routing > Routing File Locations,
'    add the locations of your SOLIDWORKS Design routing files.
' 4. Open a routing assembly that contains Route1 and two sketch points.
' 5. Modify the swModel.Extension.SelectByID2 parameters to select two sketch
'    points through which to auto route.
' 6. Select the assembly that contains Route1 in the FeatureManager design tree.
' 7. Open an Immediate Window.
'
' Postconditions: Auto Route connects the selected sketch points.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.SWRoutingLib
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```

    Sub main()
        Dim swModel As ModelDoc2
        Dim swTopLevelAssembly As AssemblyDoc
        Dim rtRouteManager As RouteManager
        Dim autoRoute As autoRoute
        Dim resultCode, boolstatus As Integer

        swModel = swApp.ActiveDoc
        swTopLevelAssembly = swModel

        ' Get the RouteManager from the top-level assembly
        rtRouteManager = swTopLevelAssembly.GetRouteManager
        If rtRouteManager Is Nothing Then
            Debug.Print("No RouteManager found in top-level document: " & swTopLevelAssembly.GetPathName)
            Exit Sub
        End If

        rtRouteManager.EditRoute()
        autoRoute = rtRouteManager.GetAutoRoute
        autoRoute.ShowGuidelines()
        boolstatus = swModel.Extension.SelectByID2("Point3", "SKETCHPOINT", -0.457835, 0.10795, -0.2032, True, 0, Nothing, 0)
        boolstatus = swModel.Extension.SelectByID2("Point8", "SKETCHPOINT", -0.218948, 0.0508, -0.470281, True, 0, Nothing, 0)
        resultCode = autoRoute.CreatePointToPointAutoRoute(2)
        rtRouteManager.ExitRoute()
        swTopLevelAssembly.EditAssembly()
    End Sub

```

```
End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Create Base-Flange Feature Using Gauge Table Example (VBA)

This examples shows how to create a sheet metal part with a base-flange feature using a gauge table.

```

'-----
' Preconditions:
' 1. Verify that the specified part template and sheet
'    metal gauge table files exist.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a part document containing a sketch.
' 2. Sets some values for the base flange feature.
' 3. Gets and prints the sheet metal gauge table thickness names
'    and available bend radii to the Immediate window.
' 4. Using the sketch, creates a sheet metal part with
'    a base flange feature.
' 5. Examine the FeatureManager design tree and Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swFeatMgr As SldWorks.FeatureManager
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swSketchManager As SldWorks.SketchManager
    Dim featDef As SldWorks.BaseFlangeFeatureData
    Dim sketchLines As Variant
    Dim thicknessNames As Variant
    Dim radii As Variant
    Dim override As Boolean
    Dim boolstatus As Boolean
    Dim feat As Object
    Dim i As Long
    Dim j As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)

    swModel.ClearSelection2 True

    Set swModelDocExt = swModel.Extension
    boolstatus = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstToRectEntity,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, False)
    boolstatus = swModelDocExt.SetUserPreferenceToggle(swUserPreferenceToggle_e.swSketchAddConstLineDiagonalType,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, True)
    Set swSketchManager = swModel.SketchManager
    sketchLines = swSketchManager.CreateCornerRectangle(0, 0, 0, 0.107090305712461, -6.06697840413517E-02, 0)
    swSketchManager.InsertSketch True

    Set swFeatMgr = swModel.FeatureManager
    Set featDef = swFeatMgr.CreateDefinition(swFmBaseFlange)
    featDef.BendRadius = 0.02
    featDef.D1OffsetDistance = 0.04
    featDef.D1OffsetType = 1
    featDef.D1ReverseOffset = True
    featDef.D2OffsetDistance = 0.001
    featDef.D2OffsetType = 1
    featDef.D2ReverseOffset = True
    featDef.OffsetDirections = 1
    featDef.ReverseDirection = False
    featDef.ReverseThickness = False

    ' All of the following values depend on the previous values;
    ' you must set these values in sequence
    featDef.UseGaugeTable = True

    ' Set the path to the sheet metal gauge tables
    featDef.GaugeTablePath = "c:\Program Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\Sheet Metal Gauge Tables\sample table - steel - english units.xls"

    thicknessNames = featDef.GetTableThicknesses()

    If (IsEmpty(thicknessNames)) Then
        MsgBox "Invalid path to sheet metal gauge table file."
        End
    End If

    Debug.Print "Sheet metal gauge table thickness names and available bend radii:"
    For i = 0 To UBound(thicknessNames)
        Debug.Print " " & thicknessNames(i)
        radii = featDef.GetTableRadii(thicknessNames(i))
        For j = 0 To UBound(radii)
            Debug.Print " " & radii(j)
        Next
    Next i

    ' Use the values from the previously called method to set the following values
    Debug.Print ""
    featDef.ThicknessTableName = thicknessNames(3)
    Debug.Print "Table thickness name for this sheet metal part: " & thicknessNames(3)
    'Convert meters to inches by multiplying value by 39.37
    Debug.Print " Thickness: " & (featDef.TableThickness * 39.37) & " inches"

```

```

' Get updated radii for new table, which was previously set
radii = featDef.GetTableRadii(thicknessNames(3))
featDef.TableRadius = radii(3)
'Convert meters to inches by multiplying value by 39.37
Debug.Print " Bend radius: " & (radii(3) * 39.37) & " inches"

' Set override values
override = False
If (override = True) Then
    featDef.OverrideRadius = True
    featDef.OverrideThickness = True
    featDef.OverrideKFactor = True
Else
    featDef.OverrideRadius = False
    featDef.OverrideThickness = False
    featDef.OverrideKFactor = False
End If

' If above override value is true, then use following properties to set override value
If (override = True) Then
    featDef.Thickness = "0.06"
    featDef.BendRadius = 0.012
    featDef.KFactor = 0.75
End If

swModel.ShowNamedView2 "Trimetric", 8
boolstatus = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)

' Create the feature
Set feat = swFeatMgr.CreateFeature(featDef)

End Sub

```

Create Base Flange Feature Example (C#)

This example shows how to create a base flange feature.

```
// =====
// Preconditions:
// 1. Ensure the specified part template exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates a flange profile sketch.
// 2. Creates Base-Flange1 in the FeatureManager design tree.
// 3. Inspect the Immediate window, graphics area,
// and FeatureManager design tree.
// =====
```

```
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Diagnostics;
```

```
namespace Macro1_CSharp
```

```
{
    public partial class SolidWorksMacro
    {
        private ModelDoc2 Part;
        private PartDoc swPart;
        private ModelDoc2 swModel;
        private Feature swSKFeat;
        private SketchSegment skSegment;
        private BaseFlangeFeatureData swBaseFlangeFeat;
        private BaseFlangeFeatureData baseFlangeFeatData;
        private CustomBendAllowance cba;
        private object[] var;
        private Feature parent;
        private Feature SHFeat;
        private SheetMetalFeatureData smFeatData;
        private CustomBendAllowance cba1;
        private bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            double swSheetWidth;
            swSheetWidth = 0d;
            double swSheetHeight;
            swSheetHeight = 0d;
            Part = (ModelDoc2)swApp.NewDocument(@"C:\ProgramData\SolidWorks\SOLIDWORKS 2022\templates\Part.prtdot", 0,
            swSheetWidth, swSheetHeight);
            swPart = (PartDoc)Part;
            swModel = (ModelDoc2)swApp.ActiveDoc;
            boolstatus = Part.Extension.SelectByID("Top", "PLANE", -0.0598881514598713d, 0.0393749830258702d,
            0.00485137895479469d, false, 0, null, 0);
            Part.SketchManager.InsertSketch(true);
            Part.ClearSelection2(true);
            skSegment = Part.SketchManager.CreateLine(-0.140779d, 0.050824d, 0d, -0.106481d, -0.06735d, 0d);
            skSegment = Part.SketchManager.CreateLine(-0.106481d, -0.06735d, 0d, 0.084966d, -0.049265d, 0d);
            skSegment = Part.SketchManager.CreateLine(0.084966d, -0.049265d, 0d, 0.143274d, 0.063608d, 0d);
            Part.ClearSelection2(true);
```

```

Part.SketchManager.InsertSketch(true);
swSKFeat = (Feature)((SelectionMgr)(swModel.SelectionManager)).GetSelectedObject6(1, -1);
Debug.Print("Flange profile name : " + swSKFeat.Name + " and type : " + swSKFeat.GetTypeName2());
swBaseFlangeFeat =
(BaseFlangeFeatureData)swModel.FeatureManager.CreateDefinition((int)swFeatureNameID_e.swFmBaseFlange);
cba = (CustomBendAllowance)swBaseFlangeFeat.GetCustomBendAllowance();
cba.Type = (int)swBendAllowanceTypes_e.swBendAllowanceDirect;
cba.BendAllowance = 0.05d;
swBaseFlangeFeat.D1EndConditionType = 1;
swBaseFlangeFeat.D1EndConditionDistance = 0.02d;
swBaseFlangeFeat.ReverseDirection = true;
swBaseFlangeFeat.OffsetDirections = 2;
swBaseFlangeFeat.D2EndConditionType = 1;
swBaseFlangeFeat.D2EndConditionDistance = 0.05d;
swBaseFlangeFeat.OverrideDefaultSheetMetalParameters = true;
swBaseFlangeFeat.Thickness = 0.035d;

// Initialize the base flange feature
// Initialize(
// UseMaterialSheetMetalParameters=False,
// UseDefaultBendAllowance=False,
// CustomBendAllowance,
// UseDefaultBendRelief=False,
// ReliefType=swSheetMetalReliefTypes_e.swSheetMetalReliefRectangular,
// UseReliefRatio=True,
// ReliefRatio=0.8m,
// ReliefWidth,
// ReliefDepth)

swBaseFlangeFeat.Initialize(false, false, cba, false, (int)swSheetMetalReliefTypes_e.swSheetMetalReliefRectangular,
true, 0.8d, 0d, 0d);
SHFeat = swModel.FeatureManager.CreateFeature(swBaseFlangeFeat);
baseFlangeFeatData = (BaseFlangeFeatureData)SHFeat.GetDefinition();
Debug.Print("Use material sheet metal parameters? " + baseFlangeFeatData.UseMaterialSheetMetalParameters);
Debug.Print("Use default bend allowance? " + baseFlangeFeatData.UseDefaultBendAllowance);
Debug.Print("Use default bend relief? " + baseFlangeFeatData.UseDefaultBendRelief);
Debug.Print("Use relief ratio? " + baseFlangeFeatData.UseReliefRatio);
Debug.Print("Relief type as defined by swSheetMetalReliefTypes_e: " + baseFlangeFeatData.ReliefType);
Debug.Print("Relief width: " + baseFlangeFeatData.ReliefWidth);
Debug.Print("Relief depth: " + baseFlangeFeatData.ReliefDepth);
Debug.Print("Relief ratio: " + baseFlangeFeatData.ReliefRatio);

// Modify the relief ratio and override default AutoRelief in the parent sheet metal feature
var = (object[])SHFeat.GetParents();
parent = (Feature)var[1];
Debug.Print("Parent type: " + parent.GetTypeName2());
smFeatData = (SheetMetalFeatureData)parent.GetDefinition();
cba1 = smFeatData.GetCustomBendAllowance();
Debug.Print("Custom bend allowance type as defined in swBendAllowanceTypes_e: " + cba1.Type);
Debug.Print("Bend allowance: " + cba1.BendAllowance);
Debug.Print("Result code for override of AutoRelief as defined by swSheetMetalModifierError_e: " +
smFeatData.SetOverrideDefaultParameter2((int)swSheetMetalOverrideDefaultParameters_e.swSheetMetalOverrideDefaultParameters_AutoRelief
true));
smFeatData.ReliefRatio = 0.7d;
Debug.Print("Base flange successfully modified? " + parent.ModifyDefinition(smFeatData, swModel, null));
Debug.Print("Base flange feature name : " + SHFeat.Name + " and type : " + SHFeat.GetTypeName2());
}

// The SldWorks swApp variable is pre-assigned for you.
public SldWorks swApp;

}
}

```

